Expo Snips

Complete Software, Product and Architecture Documentation

React Native / Expo Screen Generator — CLI, VS Code Extension and Registry

Document title	Complete Software, Product and Architecture Documentation
Document version	1.0
Date	17 October 2026
Prepared by	Generated from Repository Analysis
Repository	`C:\Dev\expo-snips`
Repository remote	`github.com/unkown812/expo-snips`

Technology Stack

Language	TypeScript 5.5 (strict, ES2020 target)
Runtime	Node.js $\geq$ 18
Repository type	pnpm workspace monorepo (`packages/*`)
Template engine	Handlebars 4.7
CLI framework	commander 12, prompts, chalk 4
Editor integration	VS Code Extension API $\geq$ 1.80.0
Visual Studio Code build	esbuild 0.21 (CJS, Node 18 target)
Registry	Static JSON catalog (24 screen recipes)
Hosting	GitHub Pages (registry), npm registry (packages)
Databases / backend	None (no server application, no database)

Confidentiality notice. This document was reverse-engineered from the repository source code. It contains no credentials, secrets or personal data. It describes the code exactly as found; where functionality is absent it is marked **NOT APPLICABLE** rather than omitted silently.

Document Control

Purpose of this document set

This documentation set describes the **Expo Snips** repository located at `C:\Dev\expo-snips`. It was produced by a complete reverse-engineering analysis of the source tree: package manifests, TypeScript sources, registry data, Handlebars templates, build scripts, GitHub Actions workflows and Markdown documentation. No application source code was modified.

Source of truth

The repository is the only source of truth. Every material statement in this document set is traceable to a file, symbol, configuration key or commit. Where the repository does not contain evidence for a statement, the statement is either omitted or explicitly labelled **UNKNOWN**.

Evidence classification scheme

Each material claim in this documentation carries a status label:

- **IMPLEMENTED** — present and verifiable in the repository source.
- **INFERRED** — a reasonable conclusion drawn from repository evidence, but not stated explicitly by the code.
- **RECOMMENDED** — a suggestion made by this document; it is *not* a description of existing behaviour.
- **UNKNOWN** — no evidence could be found in the repository.
- **NOT APPLICABLE** — the subject does not exist in this product (for example, a database, because the product has no data store).

The full scheme and a worked example are given in Appendix C.

Scope of analysis

Table 1: Analysis scope and artefacts inspected

Area	Artefacts inspected
Root workspace	`package.json`, `pnpm-workspace.yaml`, `tsconfig.base.json`, `.gitignore`
Packages	`packages/core`, `packages/cli`, `packages/registry`, `packages/vscode-extension`
Registry data	`packages/registry/data/**` (24 recipes), `packages/registry/templates/**` (48 templates)
Built registry	`registry/index.json`, `registry/recipes/**`, `packages/registry/bundle.json`
Scripts	`scripts/build-registry.mjs`, `scripts/lib/build-recipes.mjs`, `scripts/validate.mjs`, `scripts/serve.mjs`
CI/CD	`.github/workflows/deploy-registry.yml`, `.github/workflows/publish.yml`
Documentation	`documents/ProjectDescription.md`, `packages/vscode-extension/README.md`, `CHANGELOG.md`, `LICENSE`
Version control	`git log` (10 commits), branches `master`, `origin/gh-pages`

Explicitly excluded

- Generated dependency trees (`node_modules/`) were excluded, except where a manifest was needed to identify a dependency.
- Build output directories (`dist/`, `out/`, `*.vsix`) were treated as generated artefacts.
- Git-ignored generated sources (`packages/*/src/generated/`) were inspected only to confirm their build-time origin.

Terminology note

The product uses "screen recipes" (JSON definitions) rendered through Handlebars templates. Throughout this document, *recipe* means a registry entry such as `auth/login`, and *generated screen* means the resulting files written into a developer's project.

Contents

Document Control i

1 Project Overview 1
- 1.1 Introduction . . . 1
- 1.2 Repository at a glance . . . 1
- 1.3 Product category . . . 1
- 1.4 Problems addressed . . . 2
- 1.5 Solution . . . 2
- 1.6 Primary users . . . 2
- 1.7 High-level capabilities . . . 2
- 1.8 What this product is not . . . 3
- 1.9 Navigating this document . . . 3

2 Executive Summary 4
- 2.1 What the product is . . . 4
- 2.2 The problem . . . 4
- 2.3 The solution . . . 4
- 2.4 How value is delivered . . . 4
- 2.5 Technology and architecture in one paragraph . . . 5
- 2.6 Current maturity . . . 5
- 2.7 Key limitations . . . 5
- 2.8 Summary judgement . . . 5

3 Project Description 7
- 3.1 Product identity . . . 7
- 3.2 Product category and positioning . . . 7
- 3.3 Problem statement . . . 7
- 3.4 Solution . . . 8
- 3.5 Target audience . . . 8
- 3.6 Personas . . . 8
- 3.7 Value proposition . . . 9
- 3.8 Feature inventory . . . 9
- 3.9 Use cases . . . 10
- 3.10 Primary workflow . . . 11
- 3.11 Call-to-action strategy (for a future product site) . . . 11
- 3.12 Visual identity . . . 11
- 3.13 SEO, AEO and GEO summary . . . 11

4 Business Requirements Document (BRD) 12
- 4.1 Purpose . . . 12
- 4.2 Business problem . . . 12
- 4.3 Business objectives . . . 12
- 4.4 Stakeholders . . . 13
- 4.5 User groups . . . 13
- 4.6 Business processes . . . 13

4.7 Business requirements . . . 13
4.8 Expected outcomes . . . 14
4.9 Success metrics . . . 14
4.10 Constraints . . . 15
4.11 Assumptions . . . 15
4.12 Risks . . . 15

5 Product Requirements Document (PRD) 17
5.1 Product vision . . . 17
5.2 Problem statement . . . 17
5.3 Goals and objectives . . . 17
5.4 Personas . . . 17
5.5 User stories . . . 17
5.6 Use cases . . . 18
5.7 Features . . . 19
5.8 Functional requirements . . . 19
5.9 Non-functional requirements . . . 19
5.10 Acceptance criteria . . . 19
5.11 Success metrics . . . 20
5.12 Constraints and dependencies . . . 20
5.13 Risks . . . 20
5.14 Future scope . . . 21

6 Functional Requirements 22
6.1 FR-1 — Catalog resolution . . . 22
6.2 FR-2 — Project stack detection . . . 22
6.3 FR-3 — Option resolution and validation . . . 23
6.4 FR-4 — Conditional template rendering . . . 23
6.5 FR-5 — File writing . . . 23
6.6 FR-6 — UI primitive resolution . . . 24
6.7 FR-7 — Dependency installation . . . 24
6.8 FR-8 — Manifest maintenance . . . 24
6.9 FR-9 — Update with diff preview . . . 25
6.10 FR-10 — Interactive selection . . . 25
6.11 FR-11 — Project bootstrap . . . 25
6.12 FR-12 — CLI information commands . . . 25
6.13 FR-13 — Editor integration . . . 25
6.14 FR-14 — Variant selection . . . 26
6.15 Requirements traceability . . . 26

7 Non-Functional Requirements 27
7.1 Compatibility . . . 27
7.2 Performance . . . 27
7.3 Reliability and fault tolerance . . . 27
7.4 Security (summary) . . . 28
7.5 Usability . . . 28
7.6 Accessibility . . . 28
7.7 Maintainability . . . 28
7.8 Portability and distribution . . . 28
7.9 Observability . . . 29
7.10 Privacy . . . 29
7.11 Localisation . . . 29

7.12 Portability of generated code . . . 29

8 Technical Requirements **30**
8.1 Technology stack . . . 30
8.2 Frameworks and platforms . . . 30
8.3 Runtime requirements and entry points . . . 30
8.4 Architecture requirements . . . 31
8.5 Interfaces and APIs . . . 31
8.6 Data requirements . . . 31
8.7 Authentication and authorization requirements . . . 32
8.8 Infrastructure requirements . . . 32
8.9 Storage requirements . . . 32
8.10 Monitoring requirements . . . 32
8.11 Testing requirements . . . 32
8.12 Third-party integration requirements . . . 32

9 System Architecture **34**
9.1 Overview . . . 34
9.2 Deployment topology . . . 35
9.3 Communication paths . . . 35
9.4 Architectural characteristics . . . 36
9.5 Key architectural decisions . . . 36
9.6 Constraints and limitations of the architecture . . . 37

10 Software Architecture **38**
10.1 Package structure . . . 38
10.2 Module boundaries . . . 40
10.3 Dependency graph . . . 42
10.4 Build pipeline . . . 42
10.5 Code-size profile . . . 43

11 High-Level Design **44**
11.1 Design intent . . . 44
11.2 End-to-end generation flow . . . 45
11.3 Major services and how they communicate . . . 46
11.4 Data model at a high level . . . 46
11.5 Cross-cutting concerns . . . 47
11.6 Design trade-offs . . . 47

12 Low-Level Design **48**
12.1 `@expo-snips/core` . . . 48
12.2 `@expo-snips/cli` — utilities . . . 49
12.3 The recipe primitive graph . . . 51
12.4 `expo-snippets` — VS Code extension . . . 52
12.5 State . . . 52
12.6 Data access patterns . . . 53
12.7 Dependencies between modules . . . 53

13 Client and Tooling Architecture **54**
13.1 Surface comparison . . . 54
13.2 Command-line interface architecture . . . 54
13.3 VS Code extension architecture . . . 55

13.4 Shared client concerns . . . 56
13.5 Presentation layer of generated screens . . . 57

14 CLI Reference **58**
14.1 Invocation . . . 58
14.2 Command summary . . . 58
14.3 `expo-snips init` . . . 58
14.4 `expo-snips list` . . . 59
14.5 `expo-snips add` . . . 59
14.6 `expo-snips update` . . . 60
14.7 `expo-snips status` . . . 61
14.8 Exit codes and error messages . . . 61
14.9 Version discrepancy note . . . 62

15 Screen Registry and Recipe Model **63**
15.1 Concept . . . 63
15.2 Recipe schema . . . 63
15.3 Catalog statistics . . . 65
15.4 Variants . . . 65
15.5 Template organisation . . . 65
15.6 Build and distribution . . . 67
15.7 Validation . . . 68

16 API Documentation **69**
16.1 Registry HTTP interface . . . 69
16.2 Programmatic interfaces . . . 72
16.3 Failure and error semantics summary . . . 73

17 Authentication **75**
17.1 Product authentication . . . 75
17.2 Authentication templates generated for user applications . . . 75
17.3 Validation model in generated screens . . . 76
17.4 Authentication state and flow . . . 77
17.5 Password handling . . . 78
17.6 Session, token and MFA handling . . . 78
17.7 Recommendations for users of the generated auth screens . . . 78

18 Authorization **79**
18.1 Product authorization . . . 79
18.2 Effective privilege boundaries . . . 79
18.3 A UI access consideration worth recording . . . 79
18.4 Recommendation . . . 80

19 User Flows **81**
19.1 CLI journey . . . 82
19.2 VS Code journey . . . 84
19.3 Variant selection flow . . . 85
19.4 Update flow . . . 85
19.5 Error and recovery flows . . . 85

20 Data Flows **87**
20.1 Principal data flow: generation . . . 88

20.2 Registry resolution flow . . . 90
20.3 Primitive resolution flow . . . 91
20.4 Update flow . . . 93
20.5 Migration and data at rest . . . 94
20.6 Manifest lifecycle . . . 94
20.7 Data volumes . . . 95

21 UI / UX **96**
21.1 Tool experience — terminal . . . 96
21.2 Tool experience — VS Code . . . 96
21.3 Generated screen experience . . . 97
21.4 Responsive design . . . 98
21.5 Animations and interaction feedback . . . 98
21.6 Accessibility . . . 98
21.7 Dark mode . . . 99

22 Design System **100**
22.1 Scope . . . 100
22.2 Colour palette . . . 100
22.3 shadcn-style design tokens . . . 102
22.4 Typography . . . 103
22.5 Spacing . . . 103
22.6 Border radius . . . 104
22.7 Elevation, borders and shadows . . . 104
22.8 Component inventory . . . 104
22.9 Icons . . . 105
22.10 Breakpoints and responsive units . . . 105
22.11 Animation . . . 105
22.12 Design-system gap summary . . . 105

23 Security **107**
23.1 Method and scope . . . 107
23.2 Secrets . . . 107
23.3 Injection . . . 107
23.4 Trust boundary: the registry . . . 109
23.5 Generated-code findings . . . 111
23.6 Input validation . . . 112
23.7 Cross-site, CORS and web concerns . . . 112
23.8 Dependency posture . . . 113
23.9 Positive security controls . . . 113
23.10 Data integrity finding . . . 113
23.11 Security summary . . . 114

24 Performance **115**
24.1 Method and evidence boundary . . . 115
24.2 Measured artefact sizes . . . 115
24.3 Workload sizes . . . 115
24.4 Algorithmic characteristics . . . 115
24.5 Observed efficiency measures . . . 116
24.6 Optimisation opportunities . . . 116
24.7 Perceived performance (user experience) . . . 117
24.8 Summary . . . 117

25 Scalability **118**
25.1 Scope of the question . . . 118
25.2 Catalog growth . . . 118
25.3 Consumer base growth . . . 119
25.4 Team growth . . . 119
25.5 Known scalability limits . . . 119
25.6 Summary . . . 120

26 Testing **121**
26.1 Current state . . . 121
26.2 The validator as the de-facto quality gate . . . 121
26.3 Rendering is untested . . . 122
26.4 Test coverage by area . . . 122
26.5 Recommended test strategy . . . 122
26.6 Summary . . . 123

27 Deployment **124**
27.1 Deployment inventory . . . 124
27.2 Registry deployment . . . 124
27.3 npm package deployment . . . 125
27.4 VS Code extension deployment . . . 125
27.5 Runtime requirements and prerequisites . . . 125
27.6 What is absent . . . 126
27.7 Deployment summary . . . 126

28 CI/CD **127**
28.1 Overview . . . 127
28.2 Workflow inventory . . . 127
28.3 Pipeline detail: `publish.yml` . . . 128
28.4 Pipeline detail: `deploy-registry.yml` . . . 128
28.5 What the pipelines do not do . . . 128
28.6 Consequences . . . 129
28.7 Recommended pipeline . . . 129
28.8 CI/CD summary . . . 129

29 Integrations **130**
29.1 Integration map . . . 130
29.2 Build-time library integrations . . . 130
29.3 Libraries the generated screens depend on . . . 131
29.4 The remote registry as an integration . . . 132
29.5 Integration risks . . . 132
29.6 Summary . . . 132

30 Configuration **133**
30.1 Configuration surfaces . . . 133
30.2 Environment variables . . . 133
30.3 CLI options . . . 133
30.4 VS Code settings . . . 134
30.5 Project detection (configuration by inference) . . . 135
30.6 Recipe-level configuration . . . 135
30.7 Project state: the manifest . . . 135
30.8 Build and development configuration . . . 136

30.9 Configuration summary . . . 136

31 SEO — Search Engine Optimisation 137
31.1 Applicability . . . 137
31.2 In-repository SEO signals . . . 137
31.3 Strengths . . . 138
31.4 Gaps . . . 138
31.5 Summary . . . 139

32 AEO — Answer Engine Optimisation 140
32.1 Applicability . . . 140
32.2 Content available to answer engines . . . 140
32.3 Answer-readiness of the content . . . 140
32.4 Factual accuracy as an AEO risk . . . 141
32.5 Summary . . . 142

33 GEO — Generative Engine Optimisation 143
33.1 Applicability and scope . . . 143
33.2 Corpus footprint . . . 143
33.3 Generative-readiness properties . . . 143
33.4 Representation risks . . . 144
33.5 Why GEO matters for this product specifically . . . 144
33.6 Summary . . . 145

34 Known Issues 146
34.1 Purpose and method . . . 146
34.2 Issue register . . . 146
34.3 Author-acknowledged limitations . . . 149
34.4 Corrections required to prior documentation . . . 149
34.5 Issue density by area . . . 150
34.6 Summary . . . 150

35 Technical Debt 151
35.1 Definition and method . . . 151
35.2 Debt register . . . 151
35.3 The dominant debt: two clients, one engine, duplicated orchestration . . . 152
35.4 Debt heat map . . . 153
35.5 Priority . . . 153
35.6 Summary . . . 153

36 Roadmap 155
36.1 Basis . . . 155
36.2 Repository-stated roadmap . . . 155
36.3 Near-term recommendations . . . 155
36.4 Mid-term recommendations . . . 156
36.5 Long-term / architectural directions . . . 157
36.6 Sequencing . . . 157
36.7 Summary . . . 158

37 Development Guidelines 159
37.1 Purpose . . . 159
37.2 Repository layout and tooling . . . 159

37.3 TypeScript conventions . . . 159
37.4 Extension conventions (INFERRED) . . . 160
37.5 Recipe and template conventions (INFERRED + IMPLEMENTED) . . . 160
37.6 Versioning and change control (INFERRED) . . . 161
37.7 Recommended additions to these guidelines . . . 161
37.8 Summary . . . 161

38 Contribution Guide **162**
38.1 Basis and honesty note . . . 162
38.2 Prerequisites (INFERRED) . . . 162
38.3 Getting started (INFERRED) . . . 162
38.4 Contribution paths . . . 162
38.5 Change and release process (INFERRED) . . . 163
38.6 What to include in a contribution . . . 163
38.7 Missing policy areas . . . 164
38.8 Contribution-friendly features already present . . . 164
38.9 Summary . . . 164

39 Glossary **165**
39.1 Purpose . . . 165
39.2 Domain terms . . . 165
39.3 Ecosystem and technology terms . . . 166
39.4 Documentation and status terms . . . 167
39.5 Abbreviations used in this document . . . 167

40 References **169**
40.1 Purpose . . . 169
40.2 Primary repository sources . . . 169
40.3 Documentation sources . . . 170
40.4 External references . . . 170
40.5 Verified facts ledger . . . 171

A Catalog Reference **172**
A.1 Overview . . . 172
A.2 Recipe identity . . . 172
A.3 Supported options . . . 173
A.4 Catalog observations . . . 174
A.5 Tags . . . 174

B Environment and Toolchain **175**
B.1 Application runtime environment . . . 175
B.2 TypeScript and build configuration . . . 175
B.3 Declared dependency versions . . . 175
B.4 Environment variables . . . 176
B.5 Documentation build environment . . . 176
B.6 Version landscape (a source of confusion) . . . 177

C Classification and Coverage **178**
C.1 Evidence labels . . . 178
C.2 Coverage of requested document types . . . 178
C.3 Additional chapters written . . . 180
C.4 Applicability summary . . . 180

C.5 Reading guide 180

D File Index **182**
D.1 Purpose 182
D.2 Root 182
D.3 `packages/core` — shared engine and types 182
D.4 `packages/cli` — command-line interface 183
D.5 `packages/registry` — recipes, templates and schema 184
D.6 `packages/vscode-extension` — editor integration 184
D.7 `scripts` — workspace tooling 185
D.8 `.github/workflows` — automation 185
D.9 Generated and derived artefacts 185

List of Figures

9.1 System architecture: clients, shared engine, catalog sources and target project . . 34
9.2 Registry resolution sequence with bundled-catalog fallback 36

10.1 Component architecture of the pnpm workspace 39

11.1 High-level generation pipeline from command to persisted manifest 45

12.1 Recursive UI primitive resolution when `ui=shadcn` 51

13.1 VS Code extension interaction flow 55

15.1 Recipe data flows into the built registry and into the embedded client bundles . . 67

17.1 Authentication flow encoded in the generated templates (placeholder backend) . 77

19.1 CLI journey from installation to keeping screens up to date 82
19.2 VS Code journey using the inline completion trigger 84

20.1 Generation data flow: from command input to persisted manifest 88
20.2 Sequence for resolving a recipe, including fallback 90
20.3 Recursive primitive resolution data flow 91
20.4 Update flow with diff preview and confirmation 93

28.1 Continuous delivery pipelines present in the repository 127

List of Tables

1 Analysis scope and artefacts inspected ii

1.1 Repository identity and metrics (all values verified from the working tree) 1
1.2 Capability summary 2

3.1 Product identity 7
3.2 Target audience groups 8
3.3 Core features 9
3.4 Feature priority assignment 10
3.5 Use cases mapped to recipes 10

4.1 Business problems and repository evidence 12
4.2 Stakeholders 13
4.3 Business processes and how they are performed 13
4.4 Business requirements 14
4.5 Business risks 15

5.1 Goals and how they are served 17
5.2 Implemented user stories 18
5.3 Use cases 18
5.4 Acceptance criteria 20

6.6 Traceability from requirements to source files 26

7.1 Compatibility requirements 27

8.1 Technology stack (declared in package manifests) 30
8.3 Interface inventory 31
8.4 Integrations 32

9.1 System elements 34
9.2 Communication paths 35
9.3 Architectural characteristics 36
9.4 Architectural decisions and rationale 37

10.1 Package inventory 40
10.5 Runtime and build dependencies 42

11.1 Pipeline stages 46
11.2 Service-equivalent components and their interactions 46
11.3 Cross-cutting concerns and their handling 47
11.4 Principal design trade-offs 47

12.1 CLI utility functions 49
12.2 `add` command steps 50
12.3 Extension functions 52
12.4 State inventory 52
12.5 Module-level import graph (workspace) 53

13.1 CLI versus VS Code extension . . . 54
13.2 VS Code command identifiers . . . 56

14.1 Command summary . . . 58
14.2 `init` options . . . 58
14.3 `add` options . . . 59
14.4 `update` options . . . 60
14.5 Failure conditions and messages . . . 61

15.1 Recipe fields . . . 63
15.2 File entry fields . . . 64
15.3 Option enumerations (from the schema and the core types) . . . 64
15.4 Catalog composition (verified) . . . 65
15.6 Build outputs derived from the recipe sources . . . 68

16.1 Registry base URL resolution . . . 69
16.4 Public exports of the core package . . . 72
16.5 Exports of the registry package . . . 72
16.6 Extension commands and settings contributions . . . 73
16.7 Error semantics across interfaces . . . 73

17.1 Authentication recipes . . . 75

18.1 Effective privilege boundaries . . . 79

19.1 CLI journey steps . . . 83
19.2 VS Code journey steps . . . 85
19.3 Recovery flows . . . 85

20.1 Data transformations in the generation flow . . . 89
20.2 Manifest lifecycle events . . . 94
20.3 Typical data volumes (measured from the repository) . . . 95

21.1 Terminal interaction design . . . 96
21.2 Editor interaction design . . . 96
21.3 States provided by generated screens . . . 97
21.4 Accessibility finding . . . 98

22.1 Colour palette derived from template literals . . . 100
22.2 Recommended semantic colour mapping . . . 102
22.3 Tokens referenced by shadcn-style templates . . . 102
22.4 Type scale (Frontend) . . . 103
22.5 Spacing scale observed in templates . . . 103
22.6 Radius scale . . . 104
22.7 Design-system components demonstrated by recipes . . . 104
22.8 Design-system gaps . . . 105

23.1 Secret-related findings . . . 107
23.2 Injection findings . . . 108
23.3 Remote-content findings . . . 110
23.4 Findings in generated screens . . . 111
23.5 Input validation posture . . . 112
23.7 Findings by severity . . . 114

24.1 Measured artefact sizes . . . 115
24.2 Complexity of the principal routines . . . 116
24.3 Performance findings . . . 116
24.4 Performance posture . . . 117

25.1 Catalog growth analysis . . . 118
25.2 Scalability limits and risks . . . 119
25.3 Scalability posture . . . 120

26.1 Checks performed by `validate.mjs` . . . 121
26.2 Coverage assessment by area . . . 122
26.3 Testing posture . . . 123

27.1 Deployment inventory . . . 124
27.2 Runtime prerequisites . . . 125
27.3 Deployment posture . . . 126

28.1 GitHub Actions workflows . . . 127
28.2 Missing pipeline capabilities . . . 128
28.3 CI/CD posture . . . 129

29.1 External integrations . . . 130
29.2 Third-party libraries used by the product . . . 131
29.3 Dependencies declared by recipes . . . 131

30.1 Environment variables . . . 133
30.2 CLI generation options . . . 134
30.3 VS Code settings . . . 134
30.4 Detection probes . . . 135
30.5 Build-related configuration . . . 136
30.6 Configuration posture . . . 136

31.1 SEO-relevant metadata found in the repository . . . 137
31.2 SEO findings . . . 138
31.3 SEO posture . . . 139

32.1 Answer-engine-relevant content . . . 140
32.2 AEO findings . . . 140
32.3 AEO posture . . . 142

33.1 Sources a generative system could learn from . . . 143
33.2 GEO findings . . . 144
33.3 GEO posture . . . 145

34.1 Known issues . . . 146
34.2 Documentation corrections . . . 149
34.3 Issues by category . . . 150

35.1 Technical debt register . . . 151
35.2 Debt by theme and interest . . . 153

36.1 Items the repository itself lists as future work . . . 155
36.2 Near-term recommendations . . . 156
36.3 Mid-term recommendations . . . 156

36.4 Suggested sequencing . . . 157

37.1 TypeScript settings and their implications . . . 159
37.2 Template naming examples . . . 161

38.1 Contribution expectations (INFERRED) . . . 163

39.1 Glossary of domain terms . . . 165
39.2 Glossary of technology terms . . . 166
39.3 Glossary of documentation terms . . . 167
39.4 Abbreviations . . . 167

40.1 Manifest and configuration sources . . . 169
40.2 Source code sources . . . 169
40.3 Data, scripts and automation sources . . . 170
40.4 Existing documentation consulted . . . 170
40.5 External references . . . 171
40.6 Verified figures . . . 171

A.1 All recipes: identity and shape . . . 172
A.2 Options each recipe supports . . . 173

B.1 Required runtime environment for the product . . . 175
B.2 Compiler and bundler settings . . . 175
B.3 Third-party dependencies by package . . . 176
B.4 Environment variables . . . 176
B.5 Tooling used to produce this documentation . . . 176
B.6 Version lines . . . 177

C.1 Evidence labels . . . 178
C.2 Requested document types and their handling . . . 178
C.3 Areas that are not applicable to this product . . . 180

D.1 Root-level files . . . 182
D.2 Core package files . . . 182
D.3 CLI package files . . . 183
D.4 Registry package files . . . 184
D.5 Extension package files . . . 184
D.6 Build and tooling scripts . . . 185
D.7 Workflows . . . 185
D.8 Generated artefacts and their disposition . . . 186

1 Project Overview

1.1 Introduction

Expo Snips is a developer tooling product that generates complete React Native / Expo screens from a short identifier. It ships as two independent developer surfaces that share a single generation engine:

- a command-line interface published as `@expo-snips/cli`;
- a Visual Studio Code extension published as `expo-snippets` (display name "Expo Snips").

Both surfaces render versioned "screen recipes" from a registry into the developer's existing project, then optionally install the recipe's npm dependencies and record the installation in a project-level manifest file.

Status: IMPLEMENTED — see `packages/cli/package.json`, `packages/vscode-extension/package.json` and `packages/core/src`.

1.2 Repository at a glance

Table 1.1: Repository identity and metrics (all values verified from the working tree)

Attribute	Value
Root package name	`expo-snips` (private workspace root)
Workspace root version	0.1.0
Package manager	pnpm 11.9 (workspace declared in `pnpm-workspace.yaml`)
Workspace packages	4 (`packages/*`)
Languages	TypeScript (all packages), JavaScript (build scripts, `.mjs`)
Node.js requirement	$\geq$ 18 (`engines.node`)
TypeScript version	5.5
Screen recipes	24 (`packages/registry/data`)
Handlebars templates	48 (`packages/registry/templates`)
Automated test files	0
CI workflows	2 (`deploy-registry.yml`, `publish.yml`)
Root README	none
Default branch	`master` (plus `origin/gh-pages`)
Commit count at analysis time	10

1.3 Product category

Development tooling / code generation. Expo Snips is not a SaaS application, not a hosted platform and not a component library. It is a local developer tool that writes files into

the user's own repository.

Status: IMPLEMENTED — the CLI writes to `process.cwd()` and the extension writes to the VS Code workspace root; there is no server component (`packages/cli/src/utils/writeFiles.ts`, `packages/vscode-extension/src/generator.ts`).

1.4 Problems addressed

The repository's own documentation (`packages/vscode-extension/README.md`, lines 27–39) states the problem: building Expo / React Native apps requires re-writing the same screens repeatedly — authentication flows, app chrome, e-commerce views, onboarding and empty/error states — each requiring multiple files, careful styling and several library integrations.

The templates in the repository confirm this scope: recipes generate a screen file, form/child components and hooks, with validation, loading states, error handling and keyboard/safe-area handling (`packages/registry/templates/auth/login/*`).

1.5 Solution

A versioned catalog of screen recipes, rendered by a shared Handlebars engine that adapts output to the target project's detected stack (styling, router, UI primitives, icon library, TypeScript or JavaScript).

Status: IMPLEMENTED — `packages/core/src/generate.ts` (`getFilesForRecipe`, conditional `when` filtering) and `packages/cli/src/utils/detectProject.ts`.

1.6 Primary users

- Expo / React Native application developers building mobile apps.
- Developers who prefer a terminal workflow (CLI) or an editor-native workflow (VS Code completion, command palette, status bar).
- Teams that want a consistent, version-tracked catalog of screen implementations (via the manifest and the `update` command).

Status: IMPLEMENTED (developer as the only actor) / INFERRED (team and agency usage, which the repository supports technically but does not demonstrate).

1.7 High-level capabilities

Table 1.2: Capability summary

Capability	Description
Screen generation	Render a recipe into one or more project files.
Stack adaptation	Detect router, styling, UI, icons and TS/JS; render conditionally.
Interactive selection	Prompt for unresolved options (CLI) or fall back to settings (VS Code).
Dependency installation	Run the project's package manager to add recipe dependencies.

Capability	Description
Primitive resolution	Recursively add shadcn-style UI primitives when `ui=shadcn`.
Manifest tracking	Record installs in `.expo-snips.json`.
Versioned updates	Re-render tracked screens and preview a line diff before writing.
Catalog distribution	Build and serve the registry as static JSON; bundle the catalog offline.
Editor integration	Inline `snip:` completion, command palette entry and status bar item.
Project bootstrap	Create a new Expo project through `create-expo-app`.

1.8 What this product is not

To prevent misreading of later sections, the following are explicitly **not** present in the repository:

- No backend service and no REST/GraphQL API server (**NOT APPLICABLE**).
- No database, ORM, migrations or schema files (**NOT APPLICABLE**). The only persisted state is the `.expo-snips.json` manifest.
- No user accounts, authentication, authorization or sessions in the product itself (**NOT APPLICABLE**). Authentication *templates* are generated for the user's app; see Chapter 17.
- No web application, marketing site or dashboard.
- No tests, linters or formatter configuration.
- No Docker, Terraform or other infrastructure-as-code.

1.9 Navigating this document

Chapter 2 is written for non-technical stakeholders. Chapters 4–5 cover business and product requirements. Chapters 9–13 cover architecture and design. Chapter 14 is the operational command reference, Chapter 16 documents every callable interface, and Chapters 34–35 record verified problems and debt.

2 Executive Summary

2.1 What the product is

Expo Snips is a free developer tool that writes complete Expo / React Native screens into an existing mobile project from a single short command or an editor trigger. A developer types `expo-snips add auth/login` in a terminal, or `snip:` inside VS Code, and the tool produces the screen file, its supporting form component and any hooks — complete with form validation, loading and error states, and keyboard handling.

Status: IMPLEMENTED.

2.2 The problem

Teams building React Native apps repeatedly hand-write the same screens: login, registration, one-time-code entry, settings, product listings, onboarding flows and empty/error states. Each one involves several files and several libraries, and it is easy to omit the details that make a screen feel finished (validation messages, loading spinners, keyboard avoidance, safe areas).

Status: INFERRED from the product's stated rationale (`packages/vscode-extension/README.md`) and from the recipe coverage in `packages/registry/data`.

2.3 The solution

A catalog of 24 versioned "screen recipes". Each recipe declares the stacks it supports, the npm dependencies it needs, and the files it produces. A shared Handlebars engine renders those files to match the target project.

Status: IMPLEMENTED.

2.4 How value is delivered

1. **Whole screens, not snippets.** A recipe such as `auth/login` emits up to seven file variants depending on the project's stack (`app/login/index.tsx`, `screens/LoginScreen.tsx`, `components/LoginForm.tsx`, `hooks/useLogin.ts` or `.js`).
2. **Stack-aware output.** The tool reads `package.json` and `tsconfig.json` to detect the router, styling solution, UI primitives, icon library and language, then renders conditionally.
3. **Best-practice patterns.** Auth recipes use `react-hook-form` with `zod` validation and `@hookform/resolvers`.
4. **No dependency drift.** Recipe dependencies are installed with the project's own package manager (npm, pnpm, yarn or bun).
5. **Reviewable updates.** Installs are recorded in `.expo-snips.json`; `expo-snips update` shows a line-level diff before applying changes.
6. **Works offline.** The catalog is bundled into both the CLI and the extension, so no network access is required.

Status: IMPLEMENTED — each item is traceable to source (see the corresponding chapters).

2.5 Technology and architecture in one paragraph

Expo Snips is a pnpm workspace monorepo with four packages: a shared generation engine (`@expo-snips/core`, Handlebars + TypeScript types), a CLI (`@expo-snips/cli`, commander + prompts + chalk), a registry package (`@expo-snips/registry`, recipe data + 48 templates + a JSON Schema) and a VS Code extension (`expo-snippets`, esbuild-bundled). The catalog is compiled from source recipe files into a static JSON registry that is published to GitHub Pages, and into offline bundles embedded in the CLI and the extension. There is no backend, no database and no user account system.

Status: IMPLEMENTED.

2.6 Current maturity

Classification: MVP / early-stage.

- All packages are version 0.1.0 (extension 0.1.3).
- The generation engine, CLI, registry and VS Code extension are all implemented and functional.
- Infrastructure for distribution exists (GitHub Actions for registry deployment and npm publishing).
- There are no automated tests, no linters and no root README.
- There is no evidence of production adoption, usage metrics, customers or testimonials anywhere in the repository.

Status: IMPLEMENTED (versions, CI, absence of tests); **UNKNOWN** (adoption — not evidenced).

2.7 Key limitations

- Generated screens target **Expo Router** by default; only `auth/login` and `auth/register` support react-navigation, and only in their native-UI form.
- `tamagui`, `vector-icons` and `custom` UI are declared in types, schema and VS Code settings but no recipe implements them.
- Generated authentication hooks contain *placeholder* logic (a 1.5-second mock delay and an alert) and must be replaced by the developer.
- The shadcn-style templates rely on design tokens (`bg-background`, `text-foreground`, ...) for which no Tailwind / NativeWind theme configuration is shipped in the repository.
- The VS Code extension exposes no update or status command; those exist only in the CLI.

Status: IMPLEMENTED (each limitation is cited in Chapter 34).

2.8 Summary judgement

Expo Snips is a coherent, well-factored developer tool with a genuinely useful core idea — stack-aware whole-screen generation from a versioned registry — implemented across two delivery surfaces. Its principal risks are not architectural but of completeness and hardening: no tests,

several declared but unimplemented options, and placeholder business logic in generated code that must be clearly communicated to users.

Status: INFERRED (an assessment by this document, based on the evidence gathered).

3 Project Description

3.1 Product identity

Table 3.1: Product identity

Attribute	Value
Product name	Expo Snips
CLI package name	`@expo-snips/cli`
CLI binary	`expo-snips`
Extension package name	`expo-snippets` (display name "Expo Snips")
Extension publisher	`ukown812`
Engine package name	`@expo-snips/core`
Tagline	"Beautiful React Native screens, one command away."
Category	Developer tooling — screen generator / scaffolding
License (core, cli)	MIT
License (registry)	none declared; package is `private`
License (extension)	"Private License" field with a proprietary `LICENSE` file

Sources: `packages/cli/package.json` (description, bin), `packages/cli/src/index.ts` (program name and description), `packages/vscode-extension/package.json` (name, displayName, publisher, license), `packages/vscode-extension/LICENSE`.

Note on licensing inconsistency. The VS Code extension's `README.md` badge and footer describe the project as MIT-licensed and reference `github.com/expo-snips/expo-snips`, while `package.json` declares a "Private License", the bundled `LICENSE` file is proprietary, and the declared repository is `github.com/unkown812/expo-snips.git`. This contradiction is recorded as a known issue (Chapter 34).

3.2 Product category and positioning

Expo Snips sits between a code-snippet library (which inserts a line or a block) and a full project starter (which creates an entire application). It generates *whole screens* that fit into an existing project.

Status: IMPLEMENTED (behaviour) / INFERRED (positioning statement, drawn from the tool's actual output granularity).

3.3 Problem statement

Developers building Expo / React Native apps repeatedly implement the same screens by hand. The repository's own README enumerates the affected categories (`packages/vscode-extension/README.md`, lines 27–39):

- authentication screens — login, registration, forgot password, OTP — each needing validation, error handling, loading states, secure inputs and keyboard avoidance;
- app chrome — home dashboard, settings, profile, notifications, search;
- e-commerce views — product grids and product detail pages;
- onboarding flows and permission requests;
- empty, error and retry states.

3.4 Solution

A registry of versioned recipes plus a conditional template engine. The engine selects the correct file variants for the detected stack and renders them.

Status: IMPLEMENTED — `packages/core/src/generate.ts` (`getFilesForRecipe`, `matchesCondition`, `resolveTemplateString`).

3.5 Target audience

Table 3.2: Target audience groups

Group	Description and evidence
Primary	Expo / React Native application developers. The tool writes Expo Router routes under `app/` by default and detects Expo-specific dependencies.
Secondary	Teams and agencies standardising screens across projects. Supported by the manifest, versioned recipes and the `update` command.
Secondary	VS Code-centric developers. Supported by the completion provider, command palette and status bar item.
Contributor	Recipe authors. Supported by the JSON Schema (`packages/registry/schema.json`) and the validation script (`scripts/validate.mjs`).

3.6 Personas

Persona 1 — The Solo Expo Developer

- **Context:** building a new Expo app; needs auth, onboarding and core screens quickly.
- **Goal:** reach a working, consistent app without hand-writing boilerplate.
- **Relevant capability:** `expo-snips add auth/login`, stack detection, automatic dependency install, the manifest.
- **Status:** INFERRED (persona is a modelling device; the supporting capability is IMPLEMENTED).

Persona 2 — The Team / Agency Developer

- **Context:** multiple developers producing screens across projects.
- **Goal:** identical, best-practice screens and a way to pull in catalog improvements.

- **Relevant capability:** deterministic rendering from the same engine, `expo-snips status`, `expo-snips update` with diff preview.
- **Status:** INFERRED (persona) / IMPLEMENTED (capability).

Persona 3 — The VS Code Power User

- **Context:** prefers to stay inside the editor.
- **Goal:** generate screens without switching to a terminal.
- **Relevant capability:** `snip:` completion, command palette entry, status bar item, "Open main file" action.
- **Status:** IMPLEMENTED.

3.7 Value proposition

Generate complete, validated, keyboard-safe React Native screens in seconds, matched to the stack already in your project.

Supporting propositions, each evidenced:

- Whole screens, not snippets — recipes emit multiple files.
- Stack-aware by construction — conditional templates on styling, router, UI, icons and language.
- Best practices embedded — `react-hook-form` + `zod` validation, loading/error states, keyboard avoidance, safe areas.
- Zero-config dependency setup — package-manager detection and install.
- Versioned and updateable — `.expo-snips.json` manifest plus diff-previewed updates.
- Works from terminal or editor — one shared engine.

3.8 Feature inventory

Core features

Table 3.3: Core features

Feature	Evidence
Screen recipe catalog	`registry/index.json`; 24 recipes under `packages/registry/data/**`.
One-command generation	`packages/cli/src/commands/add.ts`.
Stack auto-detection	`packages/cli/src/utils/detectProject.ts`; `packages/vscode-extension/src/project.ts`.
Conditional template rendering	`packages/core/src/generate.ts`; `packages/registry/templates/**`.
Dependency installation	`packages/cli/src/utils/installDeps.ts`; `packages/vscode-extension/src/generator.ts`.
Primitive resolution (`ui=shadcn`)	`packages/cli/src/utils/primitives.ts`.
Manifest tracking	`packages/cli/src/utils/manifest.ts`; `.expo-snips.json`.

Feature	Evidence
Versioned update with diff	`packages/cli/src/commands/update.ts`; `packages/cli/src/utils/diff.ts`.
VS Code integration	`packages/vscode-extension/src/extension.ts`, `commands.ts`, `completionProvider.ts`.
Offline catalog	Bundled modules `src/generated/registry-data.ts` in both the CLI and the extension.
Project bootstrap	`packages/cli/src/commands/init.ts`.
Configurable registry	`packages/cli/src/utils/registry.ts`; `packages/vscode-extension/src/registry.ts`.
Screen variants	`ScreenVariant` type; variants defined for `app/home`, `auth/login`, `onboarding/welcome` (id `glass`).

Feature priority (this document's assessment)

Table 3.4: Feature priority assignment

Priority	Features
P0 — Core	Recipe catalog, `add` generation, stack detection, conditional templates, dependency install, manifest, `status`/`update`, VS Code completion and commands.
P1 — Important	`init`, shadcn primitives, configurable registry, offline bundled catalog.
P2 — Secondary	Screen variants (implemented for 3 recipes, not exercised by default).
P3 — Maintainer	Registry build/deploy automation, npm publish workflow.

Status: RECOMMENDED (the priority ranking is this document's interpretation, not a repository artefact).

3.9 Use cases

Table 3.5: Use cases mapped to recipes

Use case	Recipes
Authentication	`auth/login`, `auth/register`, `auth/forgot-password`, `auth/otp-verification`
Onboarding	`onboarding/welcome`, `onboarding/features`, `onboarding/permissions`
App chrome	`app/home`, `app/settings`, `app/profile`, `app/notifications`, `app/search`
Commerce	`ecommerce/products`, `ecommerce/product-details`
Content	`content/article-reader`

Use case	Recipes
Chat	`chat/conversations`
Calendar	`calendar/schedule`
Analytics	`dashboard/analytics`
Maps	`map/location`
Data states	`utility/empty`, `utility/error`
UI primitives	`ui/button`, `ui/input`, `ui/utils`

3.10 Primary workflow

Figures 19.1 and 13.1 show the two journeys. In both, the sequence is: resolve catalog, detect project stack, resolve unresolved options, render templates, write files, install dependencies, update the manifest.

3.11 Call-to-action strategy (for a future product site)

The product has *no* sign-up, trial, account, billing or demo flow anywhere in the repository. An honest conversion path therefore drives installation and first generation rather than registration.

Status: RECOMMENDED (the CTA wording is a suggestion). The statement that no account or billing flow exists is IMPLEMENTED/verified — a repository-wide search found no pricing, payment, subscription or account code.

3.12 Visual identity

The repository contains no brand assets (no logo files, SVGs, or colour variables) and no marketing site. The only brand mark is a VS Code status bar item labelled with the editor's `sparkle` codicon and the text "Expo Snips" (`packages/vscode-extension/src/extension.ts`, line 10).

Visual identity therefore has to be derived from the generated screens, which is what Chapter 22 does.

3.13 SEO, AEO and GEO summary

Because there is no website, there is no sitemap, robots file, canonical URL, structured data or Open Graph metadata in the repository. Discoverability documentation in Chapters 31–33 therefore separates *current implementation* (essentially: package metadata and published READMEs) from *recommended improvements*.

4 Business Requirements Document (BRD)

4.1 Purpose

This chapter records the business context that the repository supports. The repository is a developer tool with no commercial surface, so business requirements are necessarily modest and are stated only where the code provides evidence or where an assumption is clearly labelled.

4.2 Business problem

Table 4.1: Business problems and repository evidence

Problem	Evidence
Repetitive screen construction consumes developer time	Recipe catalog exists precisely to remove repeated screen authoring; `packages/registry/data/**`.
Inconsistent screen quality across a team	A single shared registry plus deterministic rendering produces identical output; `packages/core/src/generate.ts`.
Dependency setup drift	Automated install using the project's package manager; `packages/cli/src/utils/installDeps.ts`.
Screens fall behind catalogue improvements	Versioned recipes, manifest tracking and diff-previewed updates; `packages/cli/src/commands/update.ts`.

Status: INFERRED — these are the business consequences the design implies. The repository contains no business case document, no market analysis and no customer research.

4.3 Business objectives

Each objective below is expressed as something the current code can already do, so that no objective is aspirational:

1. Ship a reusable catalog of best-practice Expo / React Native screens (24 recipes present).
2. Make adding a screen a single, reversible action (`expo-snips add`, with no overwrite of unrelated files).
3. Adapt generated code to the developer's existing stack rather than requiring the developer to adapt (stack detection and conditional templates).
4. Keep installed screens maintainable over time (manifest, `status`, `update`).
5. Distribute the catalog and packages without manual release steps (GitHub Actions workflows).

Status: IMPLEMENTED (objectives 1–5 are all realised in code).

4.4 Stakeholders

Table 4.2: Stakeholders

Stakeholder	Interest
End users (developers)	Adding complete, correct screens quickly and consistently.
Recipe authors / contributors	An unambiguous way to define screens (JSON Schema, validation script).
Maintainers	Publishing the registry and npm packages reliably; keeping the catalog correct.
Organisation / copyright holder	Declared in `packages/vscode-extension/LICENSE` as "Jay Jogane" (2026). No other organisational information exists in the repository.

4.5 User groups

- **CLI users** — interact through `init`, `list`, `add`, `status`, `update`.
- **VS Code users** — interact through the completion trigger, the command palette and the status bar.
- **Catalog consumers** — anyone fetching `index.json` or individual recipe JSON documents from the registry.
- **Catalog producers** — maintainers adding recipes and running `pnpm validate` and `pnpm build:registry`.

Status: IMPLEMENTED.

4.6 Business processes

Table 4.3: Business processes and how they are performed

Process	Implementation
Publish the screen catalog	Push to `main`/`master` → `node scripts/build-registry.mjs` → deploy the `registry/` folder to GitHub Pages (`deploy-registry.yml`).
Publish the packages	Push a `v*` tag → `pnpm install` → `pnpm build` → publish `@expo-snips/core` and `@expo-snips/cli` to npm (`publish.yml`).
Add a screen recipe	Author a JSON file under `packages/registry/data/` plus `.hbs` templates; validate with `scripts/validate.mjs`; rebuild.
Distribute to users	Users install the CLI from npm or the extension from a `.vsix`; the catalog is bundled at build time and is also fetched from the registry when configured.

4.7 Business requirements

Table 4.4: Business requirements

ID	Requirement	Evidence / status
BR-01	The catalog must be distributable independently of the clients.	IMPLEMENTED — the registry is a folder of static JSON published to GitHub Pages; clients fall back to a bundled copy.
BR-02	Clients must work without network access.	IMPLEMENTED — bundled `REGISTRY_RECIPES` in both the CLI and the extension.
BR-03	Generated code must not overwrite unrelated files.	IMPLEMENTED — only recipe file paths are written; primitives skip files that already exist (`primitives.ts`).
BR-04	Installs must be inventoried per project.	IMPLEMENTED — `.expo-snips.json` with recipe id, version, options, variant, files and timestamp.
BR-05	Users must be able to review changes before applying updates.	IMPLEMENTED — `update` prints a line diff and requires confirmation unless `-yes`.
BR-06	The catalog must be extensible by third parties.	IMPLEMENTED — JSON Schema plus a self-hosted registry option (`-registry`, `EXPO_SNIPS_REGISTRY`, `expo-snips.registryUrl`).
BR-07	Release of packages must be automated.	IMPLEMENTED — `publish.yml` runs on `v*` tags.
BR-08	The VS Code extension should also be released automatically.	NOT IMPLEMENTED — the extension is packaged manually with `vsce` (`package.json` script "package"); no workflow builds it.
BR-09	No usage telemetry or analytics.	IMPLEMENTED (by absence) — no analytics, telemetry or tracking code exists.

4.8 Expected outcomes

- Developers can add a complete screen with one command.
- Generated code matches the project's router, styling, UI primitives, icon library and language.
- Screens remain updateable from the catalog.
- Maintainers can add recipes without changing engine code.

Status: IMPLEMENTED.

4.9 Success metrics

UNKNOWN. The repository contains no analytics, telemetry, usage counters, issue templates, download badges or any other measurable signal. No success metric can be reported without invention, and none is proposed here as fact.

Status: RECOMMENDED (a future measurement programme would be needed to establish

metrics such as number of generations, recipe coverage or update adoption — but this document does not invent targets).

4.10 Constraints

- Node.js ≥ 18 is required (`engines.node` in the CLI).
- Generated screens consume Expo Router by default; the catalog's react-navigation coverage is limited to `auth/login` and `auth/register`.
- The shadcn UI path forces `styling=nativewind` and `icons=lucide` (`add.ts`, lines 202–214 and `generator.ts`, lines 82–85).
- The registry is unauthenticated static hosting; there is no access control on published recipes.
- The VS Code extension targets VS Code ≥ 1.80.0.

4.11 Assumptions

- Users have an existing Expo / React Native project and a working package manager.
- Users will review generated code and replace placeholder business logic before shipping.
- "Icons = none" is an acceptable degradation path; templates guard icon imports with `{#if (eq icons "lucide")}`.

Status: INFERRED.

4.12 Risks

Table 4.5: Business risks

ID	Risk	Severity	Mitigation / note
R-01	Users treat placeholder auth hooks as production logic.	High	Templates carry a `// Replace with your actual authentication logic` comment and a commented `supabase` example; user-facing docs should state this plainly.
R-02	A remote registry is fully trusted: arbitrary recipe templates are rendered to disk and their npm dependencies are executed.	High	No signature or checksum verification exists. Mitigated only by defaulting to the bundled catalog.
R-03	Declared-but-unimplemented options (`tamagui`, `vector-icons`, `custom`) cause user confusion.	Medium	Document explicitly; see Chapter 34.
R-04	No automated tests means regressions in the engine or CLI are detectable only manually.	Medium	Add a test suite; see Chapter 26.

ID	Risk	Severity	Mitigation / note
R-05	Licensing inconsistency (MIT in two packages, proprietary elsewhere; two different repository URLs).	Medium	Reconcile licence and repository metadata before public distribution.
R-06	A user's existing files at recipe paths are overwritten by `add`.	Medium	`add` writes unconditionally via `fs.writeFile`; only primitives are skip-aware.

Status: INFERRED (severity and mitigation are this document's assessment, grounded in the cited code).

5 Product Requirements Document (PRD)

5.1 Product vision

A developer should never hand-write a login screen again. Expo Snips aims to make any common mobile screen available as a versioned, stack-aware recipe that can be generated into an existing project in one action and kept current thereafter.

Status: INFERRED (vision statement); the underlying capability is IMPLEMENTED.

5.2 Problem statement

See Section 2.3 of Chapter 3. In short: recurring screen boilerplate is slow, inconsistent and easy to get subtly wrong, and no existing category of tool (snippet library, project starter, component library) generates whole, wired screens into an existing stack.

5.3 Goals and objectives

Table 5.1: Goals and how they are served

Goal	Mechanism
Generate complete screens	Multi-file recipes (`files[]` with conditions).
Match the user's stack	`detectProject()` plus conditional template rendering.
Make output trustworthy	Best-practice libraries (react-hook-form, zod) and standard platform patterns.
Remove dependency friction	Package-manager detection and installation.
Support long-term maintenance	Manifest, `status`, diff-previewed `update`.
Work in terminal and editor	One engine, two surfaces.
Zero-config, offline-capable	Catalog bundled into both clients.

Status: IMPLEMENTED (mechanisms) / RECOMMENDED (goal phrasing).

5.4 Personas

See Section 2.6 of Chapter 3 for the three personas (solo developer, team/agency developer, VS Code power user).

5.5 User stories

Stories are written in the standard form and each is mapped to the code that satisfies it. Stories without an implementation are listed separately as future scope.

Table 5.2: Implemented user stories

ID	Story	Implementation
US-01	As a developer, I want to add a screen with one command so I do not hand-write boilerplate.	`add.ts`: `expo-snips add <screenId>`.
US-02	As a developer, I want the generated code to match my project's styling.	`detectProject.ts` + `#if (eq styling ...)` in templates.
US-03	As a developer, I want to choose a screen interactively if I do not know its id.	`add.ts` interactive `prompts` select when no argument is given.
US-04	As a developer, I want form validation in generated auth screens.	`zod` schema + `zodResolver` in `components.login-form.hbs`.
US-05	As a developer, I want loading and error states.	`isLoading` state and error rendering in the login template.
US-06	As a developer, I want keyboard handling.	`KeyboardAvoidingView` with platform-specific behaviour.
US-07	As a developer, I want the right dependencies installed automatically.	`installDeps.ts` / `generator.ts`.
US-08	As a developer, I want to see which screens my project uses.	`expo-snips status` reading `.expo-snips.json`.
US-09	As a developer, I want to update generated screens safely.	`update.ts` with `printDiff` and confirmation.
US-10	As a VS Code user, I want to add a screen without leaving the editor.	`completionProvider.ts`, `commands.ts`, status bar in `extension.ts`.
US-11	As a TypeScript user, I want TypeScript output.	Recipe `typescript` flag and `#if typescript` blocks; TS/JS file variants.
US-12	As a JavaScript user, I want JavaScript output.	`-javascript` flag; `hooks/useLogin.js` variant.
US-13	As a developer using shadcn-style UI, I want the primitives generated.	`primitives.ts` plus `ui/*` recipes.
US-14	As a developer, I want to start a brand-new Expo project.	`init.ts` running `create-expo-app`.
US-15	As a maintainer, I want to validate recipes before publishing.	`scripts/validate.mjs` against `schema.json`.

5.6 Use cases

Table 5.3: Use cases

ID	Use case	Flow
UC-01	Add an authentication screen	`add auth/login` → detect → render → install → manifest.
UC-02	Add an app-chrome screen	`add app/home`.

ID	Use case	Flow
UC-03	Generate a glassmorphism variant	`add app/home -variant glass` (variant templates exist for `app/home`, `auth/login`, `onboarding/welcome`).
UC-04	Generate shadcn UI	`add auth/register -ui shadcn` (forces NativeWind and lucide; adds primitives).
UC-05	Generate JavaScript output	`add auth/login -javascript`.
UC-06	Generate through react-navigation	`add auth/login -router react-navigation` (supported only for `auth/login` and `auth/register`).
UC-07	Inspect installed screens	`status`.
UC-08	Update installed screens	`update` or `update <id> -yes`.
UC-09	Use a custom registry	`add app/home -registry http://localhost:3000`.
UC-10	Bootstrap a project	`init -name myapp`.

5.7 Features

The reference list of features is the recipe and command inventory in Section 3.10 of Chapter 3. The catalog itself is tabulated in Appendix A.

5.8 Functional requirements

Functional requirements are numbered and detailed in Chapter 6. In the PRD they are summarised as:

- FR-1 Catalog resolution (bundled or remote).
- FR-2 Project stack detection.
- FR-3 Option resolution and validation.
- FR-4 Conditional template rendering.
- FR-5 File writing.
- FR-6 UI primitive resolution.
- FR-7 Dependency installation.
- FR-8 Manifest maintenance.
- FR-9 Update with diff preview.
- FR-10 Interactive selection.
- FR-11 Project bootstrap.

5.9 Non-functional requirements

See Chapter 7. Highlights: offline operation, no telemetry, cross-platform paths, deterministic output, and a 10-second network timeout.

5.10 Acceptance criteria

Acceptance criteria are stated against observable behaviour.

Table 5.4: Acceptance criteria

ID	Criterion
AC-01	Running `expo-snips add auth/login` in a TypeScript Expo Router project creates `app/login/index.tsx`, `components/LoginForm.tsx` and `hooks/useLogin.ts`.
AC-02	The generated screen contains a `KeyboardAvoidingView` and a `SafeAreaView`.
AC-03	Validation errors render for empty or malformed input because a `zod` schema is applied through `react-hook-form`.
AC-04	With `-ui shadcn`, `components/ui/button.tsx`, `components/ui/input.tsx` and `lib/utils.ts` are created unless they already exist.
AC-05	With `-no-install`, no package-manager command is executed.
AC-06	After generation, `.expo-snips.json` contains an install entry for the recipe with its version and options.
AC-07	`expo-snips status` lists every entry in the manifest.
AC-08	`expo-snips update` prints a diff and changes nothing when the user declines.
AC-09	In VS Code, typing `snip:` offers the recipe identifiers and selecting one generates the screen.
AC-10	With no network and no registry configured, `list` still reports the bundled catalog.
AC-11	`expo-snips add auth/login -styling tamagui` exits with an error because the recipe does not support it.

Status: IMPLEMENTED (each criterion is directly derivable from the cited source) — except AC-11, which is the intended validation path in `add.ts` lines 104–109.

5.11 Success metrics

UNKNOWN. See Section 4.9 of Chapter 4. No metric data exists in the repository.

5.12 Constraints and dependencies

- Runtime: Node.js $\geq$ 18.
- Engine dependency: `handlebars` ^4.7.8 (only runtime dependency of `@expo-snips/core`).
- CLI dependencies: `commander` ^12, `prompts` ^2.4, `chalk` ^4.1, `fs-extra` ^11.2.
- Extension build: `esbuild` ^0.21.5 and `@vscode/vsce` ^2.24 (development only).
- Generated screens depend on recipe-declared packages; for auth recipes that is `react-hook-form`, `zod`, `@hookform/resolvers` and `lucide-react-native`.

5.13 Risks

See Section 4.11 of Chapter 4 (R-01 . . . R-06).

5.14 Future scope

The following are *not* implemented and are listed as candidate future work. They are drawn from gaps in the repository and from the extension's own "Roadmap" section, which is itself an unimplemented checklist.

- VS Code commands for update and status (the CLI has them; the extension does not).
- Tamagui styling support, Vector Icons support, custom UI primitive support (declared in types/schema/settings; no recipes).
- Test infrastructure (`README.md` "Roadmap": "Testing infrastructure").
- CI for packaging the VS Code extension (`README.md` "Roadmap": "CI/CD pipeline for the VS Code extension").
- Additional recipes for react-navigation.
- A NativeWind/Tailwind theme providing the shadcn design tokens used by the `ui=shadcn` templates.

Status: RECOMMENDED / POTENTIAL FUTURE WORK — explicitly not committed plans.

6 Functional Requirements

Each requirement states observable behaviour, the implementing code, and a status label. All requirements below are IMPLEMENTED unless stated otherwise.

6.1 FR-1 — Catalog resolution

Aspect	Detail
Description	The tool resolves the screen catalog from a remote registry when one is configured, otherwise from the catalog bundled at build time.
Precedence	Explicit `-registry` flag, then `EXPO_SNIPS_REGISTRY` (CLI), or the `expo-snips.registryUrl` setting (extension); an empty value selects the bundled catalog.
Remote fetch	`GET {base}/index.json` for the index and `GET {base}/recipes/{id}.json` for a single recipe.
Timeout	10 000 ms via `AbortController`.
Fallback	Any failure, empty array or non-OK response falls back to the bundled catalog.
Evidence	`packages/cli/src/utils/registry.ts` (`getRegistryBase`, `resolveRecipeIndex`, `getRecipeById`); `packages/vscode-extension/src/registry.ts`.

Requirement FR-1.1 — `getRegistryBase` must normalise trailing slashes and trim whitespace. **Requirement FR-1.2** — the extension must cache a successful remote index for 5 minutes and back off from an unreachable registry for the same period.

6.2 FR-2 — Project stack detection

Detected value	Rule
Router	`expo-router` dependency → `expo-router`; else `@react-navigation/native` → `react-navigation`.
Styling	`nativewind` dependency → `nativewind`; otherwise `stylesheet`.
UI primitives	`@shadcn/ui` or `react-native-reusables` dependency → `shadcn`; otherwise `native`.
Icons	`lucide-react-native` dependency → `lucide`; otherwise `none`.
Language	Presence of `tsconfig.json` → TypeScript.

Evidence: `packages/cli/src/utils/detectProject.ts` (all rules) and the duplicated implementation in `packages/vscode-extension/src/project.ts`. Note that detection reads `dependencies` only — `devDependencies` are not inspected.

Requirement FR-2.1 — detection must return a partial result rather than throw when `package.json` is absent; the CLI then falls back to prompting and the extension to settings.

6.3 FR-3 — Option resolution and validation

Resolution order for each option (styling, router, UI, icons):

1. an explicit CLI flag, if provided;
2. if the recipe supports exactly one value, that value;
3. the detected value, if the recipe supports it;
4. otherwise (CLI) an interactive prompt listing only supported values, or (extension) a configurable default filtered through `pickSupported()`.

Any explicitly supplied value that the recipe does not support aborts the command with a coloured error message and sets the process exit code to 1 (`add.ts` lines 104–159).

Language resolution: `-javascript` forces JavaScript; otherwise `-typescript` or the detected value, defaulting to TypeScript. If a TypeScript-authored recipe is generated as JavaScript a warning is printed (`add.ts` lines 161–165).

Requirement FR-3.1 — selecting `ui=shadcn` must force `styling=nativewind` and `icons=lucide`, printing an explanatory notice (`add.ts` lines 202–214; `generator.ts` lines 82–85).

6.4 FR-4 — Conditional template rendering

Behaviour	Detail
File filtering	A file entry with a `when` clause is skipped unless every specified condition matches the generation options.
Template precedence	`variants[variant]` → `perStyling[styling]` → `template`.
Compilation	Handlebars compiles each template string with `eq` and `neq` helpers registered once at module load.
Render context	`options`, `styling`, `router`, `ui`, `icons`, `typescript`, `variant`.
Evidence	`packages/core/src/generate.ts`; `packages/core/src/context.ts`.

Requirement FR-4.1 — a file entry with no resolvable template string must be silently skipped rather than throwing.

6.5 FR-5 — File writing

Files are written relative to the project root (CLI: the current working directory; extension: the first workspace folder). Parent directories are created on demand. The CLI logs each created

path; the extension logs them in a progress notification and offers to open the first file.

Evidence: `packages/cli/src/utils/writeFiles.ts`; `packages/vscode-extension/src/generator.ts` lines 214–251.

6.6 FR-6 — UI primitive resolution

When the effective UI option is `shadcn`, the engine walks the recipe's `primitives` array recursively, marking ids already seen to avoid cycles, forcing `styling=nativewind` and `ui=shadcn` on the primitive generation context, skipping any file whose path already exists, and merging the primitives' dependencies into the install set.

Evidence: `packages/cli/src/utils/primitives.ts` and the mirrored implementation in `packages/vscode-extension/src/generator.ts` (lines 127–179).

The shipped primitive chain is `ui/button` → `ui/utils` and `ui/input` → `ui/utils` (`packages/registry/data/ui/*.json`).

6.7 FR-7 — Dependency installation

- Package manager detection: `pnpm-lock.yaml` → pnpm; `yarn.lock` → yarn; `bun.lockb` → bun; otherwise npm.
- Production dependencies are installed with `<manager> add`; dev dependencies with `<manager> add -D`.
- The CLI executes synchronously with inherited stdio (`execSync`, `stdio: "inherit"`).
- The extension sends the commands to a dedicated "Expo Snips" terminal, and only when `expo-snips.installDependencies` is true (default).
- Duplicate dependencies are removed with a `Set` before installation.

Evidence: `installDeps.ts`; `generator.ts` lines 108–125 and 265–267.

Requirement FR-7.1 — when `-no-install` is passed, no package-manager command may run.

6.8 FR-8 — Manifest maintenance

Aspect	Detail
File name	`.expo-snips.json`, written at the project root with two-space indentation.
Schema	`{ version: number, installs: ManifestEntry[] }`.
Entry fields	`recipeId`, `recipeVersion`, `installedAt` (ISO timestamp), `options` {styling, router, ui, icons, typescript}, optional `variant`, `files[]`.
Upsert key	`recipeId` together with `variant` — reinstalling the same recipe/variant replaces the entry.
Corrupt file handling	Invalid JSON yields an empty manifest rather than an error.
Evidence	`packages/cli/src/utils/manifest.ts`; `packages/vscode-extension/src/project.ts` lines 46–103.

6.9 FR-9 — Update with diff preview

`expo-snips update [screenId] [-registry <url>] [-yes]`:

1. read the manifest; if a `screenId` is supplied, filter to it and error if nothing matches;
2. re-fetch each recipe (skipping and warning on recipes no longer present in the catalog);
3. re-render using the stored options and variant, including shadcn primitives when applicable;
4. compare rendered content against files on disk; a missing file is treated as fully new content;
5. if nothing changed, report that everything is up to date;
6. otherwise print a line-level diff (longest-common-subsequence) and ask for confirmation unless `-yes` was given;
7. on confirmation, write the files and update each entry's `recipeVersion`.

Evidence: `packages/cli/src/commands/update.ts`; `packages/cli/src/utils/diff.ts`.

6.10 FR-10 — Interactive selection

With no screen argument, the CLI displays a select prompt built from the resolved index (id → name). Cancelling sets exit code 1. An empty index prints guidance and exits non-zero.

Evidence: `packages/cli/src/commands/add.ts` lines 72–92.

6.11 FR-11 — Project bootstrap

`expo-snips init -name <name> -template <template>` runs `npx create-expo-app@latest <name|.> -template <template> -yes`. It refuses to proceed when `app.json` already exists in the current directory.

Evidence: `packages/cli/src/commands/init.ts`.

Status: IMPLEMENTED. Note the caveat in Chapter 23 regarding shell-string construction.

6.12 FR-12 — CLI information commands

`list` prints the count and source of the catalog plus one line per recipe (id, name, tags). `status` prints each manifest entry with its version and optional variant, or a message when nothing is tracked.

Evidence: `packages/cli/src/commands/list.ts`; `packages/cli/src/commands/status.ts`.

6.13 FR-13 — Editor integration

Capability	Detail
Inline completion	Registered for TypeScript, TSX, JavaScript and JSX, triggered on `:`, matching `snip:` or `expo-snips:` immediately before the cursor. Each item inserts empty text and runs the generation command with the recipe id.
Command palette	`expo-snips.addScreen` shows a Quick Pick of the catalog.

Capability	Detail
Status bar	A right-aligned item showing `$(sparkle) Expo Snips` bound to the add-screen command.
Post-generation	An information message reports the file count and offers "Open main file".
Evidence	`completionProvider.ts`, `commands.ts`, `extension.ts`, `generator.ts`.

6.14 FR-14 — Variant selection

A recipe may declare variants. Three recipes do: `app/home`, `auth/login` and `onboarding/welcome`, each with a single `glass` variant. The CLI accepts `-variant <id>`; the extension shows a Quick Pick when a recipe declares one or more variants. Templates can override output per variant through the `variants` map on a file entry.

Evidence: `packages/core/src/types.ts` (`ScreenVariant`); `packages/registry/data/app/home.json`, `auth/login.json`, `onboarding/welcome.json`; `generator.ts` lines 92–106.

6.15 Requirements traceability

Table 6.6: Traceability from requirements to source files

Req.	Name	Primary source
FR-1	Catalog resolution	`cli/src/utils/registry.ts`
FR-2	Stack detection	`cli/src/utils/detectProject.ts`
FR-3	Option resolution	`cli/src/commands/add.ts`
FR-4	Template rendering	`core/src/generate.ts`
FR-5	File writing	`cli/src/utils/writeFiles.ts`
FR-6	Primitive resolution	`cli/src/utils/primitives.ts`
FR-7	Dependency install	`cli/src/utils/installDeps.ts`
FR-8	Manifest	`cli/src/utils/manifest.ts`
FR-9	Update	`cli/src/commands/update.ts`
FR-10	Interactive selection	`cli/src/commands/add.ts`
FR-11	Bootstrap	`cli/src/commands/init.ts`
FR-12	Info commands	`cli/src/commands/list.ts`, `status.ts`
FR-13	Editor integration	`vscode-extension/src/*.ts`
FR-14	Variants	`core/src/types.ts`, recipe data

7 Non-Functional Requirements

Non-functional requirements are stated only where the repository provides evidence, or are explicitly marked as RECOMMENDED.

7.1 Compatibility

Table 7.1: Compatibility requirements

Area	Requirement / evidence
Node.js	$\geq$ 18 (`engines.node` in `@expo-snips/cli`; esbuild target `node18`).
Language target	ES2020, CommonJS modules (`tsconfig.base.json`).
VS Code	$\geq$ 1.80.0 (`engines.vscode`).
Operating system	No OS-specific code; paths are built with `path.join` and the CLI uses `process.cwd()`, so Windows, macOS and Linux are supported.
Target projects	Expo / React Native with Expo Router (default); NativeWind or StyleSheet; TypeScript or JavaScript.

7.2 Performance

- A remote registry request is bounded by a 10-second abort timeout (`registry.ts` in both clients).
- The extension caches a successful remote index for 5 minutes and enters a 5-minute back-off after an unreachable registry, preventing repeated stalls (`packages/vscode-extension/src/registry.ts`).
- Rendering is in-memory Handlebars compilation per file; there is no incremental compilation or caching. This is INFERRED as adequate because recipes contain at most seven file entries, but no benchmark exists in the repository.
- The diff algorithm is a full $O(n \times m)$ longest-common-subsequence table (`diff.ts`), allocated per file compared.

Quantified performance targets (**UNKNOWN**) — the repository defines none.

7.3 Reliability and fault tolerance

- Catalog resolution degrades gracefully: remote failure falls back to the bundled catalog (FR-1).
- An unreadable or corrupt manifest is treated as empty (`manifest.ts`, lines 37–45).
- Update skips recipes that are no longer present in the registry with a warning instead of failing the whole operation (`update.ts`, lines 78–83).
- An unknown primitive id logs a warning and is skipped (`primitives.ts`, line 31).

Status: IMPLEMENTED.

7.4 Security (summary)

Detailed in Chapter 23. Non-functional expectations:

- No credentials are stored or transmitted by the product.
- Network access is read-only HTTP GET against JSON endpoints.
- Recipe content and recipe dependency names are treated as trusted input; there is no signature verification.

Status: IMPLEMENTED (behaviour) / RECOMMENDED (hardening measures).

7.5 Usability

- Coloured terminal output groups information, warnings and errors (`chalk`).
- Interactive prompts are used only when a value cannot be resolved.
- Post-generation guidance lists concrete next steps (`add.ts`, lines 275–278).
- The extension reports progress in a notification and offers a single follow-up action.

Status: IMPLEMENTED.

7.6 Accessibility

NOT APPLICABLE for a graphical user interface: the product has no web or mobile UI of its own; its interface is a command-line program and a VS Code extension surface built entirely from native VS Code components (Quick Pick, notification, status bar), which inherit the editor's own accessibility behaviour.

Accessibility of *generated screens* is the responsibility of the recipe authors. The templates verify: they use `Text` components for all copy and `Pressable` for interaction, but no recipe adds `accessibilityLabel`, `accessibilityRole` or `accessibilityHint`. This is recorded as a recommendation in Chapter 21 and as technical debt in Chapter 35.

7.7 Maintainability

- The engine, CLI, registry and extension are separated into distinct packages with a single dependency direction (CLI/extension/registry → core).
- The recipe format is captured in a JSON Schema and enforced by a validation script.
- TypeScript strict mode is enabled workspace-wide (`strict: true`).
- Counter-evidence: the stack-detection logic and the primitive-resolution logic are duplicated between the CLI and the extension; there is no test suite; and generated catalog modules are committed/ignored inconsistently.

Status: IMPLEMENTED (structure) / INFERRED (assessment).

7.8 Portability and distribution

- The CLI is published to npm with public access.
- The extension is packaged with `vsce` into a `.vsix`.

- The registry is plain static files, deployable to any static host.
- The catalog is additionally embedded in both clients, so distribution of the catalog does not depend on hosting availability.

Status: IMPLEMENTED.

7.9 Observability

The product emits console output only. There is no logging framework, no log levels, no log destination configuration, no metrics and no telemetry. Diagnostics are limited to `console.log`/`console.warn`/`console.error` with `chalk` colouring in the CLI.

Status: IMPLEMENTED (as a limitation); instrumentation is RECOMMENDED for future work but is not required for a local code-generation tool.

7.10 Privacy

- No personal data is collected, transmitted or stored by the product.
- The only persistent artefact is `.expo-snips.json` in the user's project, containing recipe ids, versions, timestamps and option values.
- No analytics or third-party trackers exist in the repository.

Status: IMPLEMENTED (verified by absence).

7.11 Localisation

NOT APPLICABLE. All user-facing strings are hard-coded English literals; there is no i18n framework, message catalogue or locale handling.

7.12 Portability of generated code

Generated screens target React Native only. They import from `react-native`, `expo-router` or `@react-navigation/native`, and the third-party libraries declared by each recipe. They are therefore portable across the platforms Expo supports, subject to the recipe's dependency set (for example `map/location` declares `react-native-maps`, which is not available on the web target without an additional provider).

8 Technical Requirements

8.1 Technology stack

Table 8.1: Technology stack (declared in package manifests)

Category	Technology	Detail
Language	TypeScript 5.5	Strict mode, ES2020 target, CommonJS, declaration + source maps.
Runtime	Node.js $\geq$ 18	Declared in `@expo-snips/cli` engines.
Workspace	pnpm workspaces	`pnpm-workspace.yaml` lists `packages/*`.
Engine	Handlebars 4.7	Only runtime dependency of `@expo-snips/core`.
CLI	commander 12	Command registration.
CLI prompts	prompts 2.4	Interactive selection and confirmation.
CLI output	chalk 4.1	Terminal colouring.
File I/O	fs-extra 11.2	`ensureDir`, `readJSON`, `writeJSON`, `pathExists`.
Extension bundler	esbuild 0.21	CJS bundle, `node18` target, `vscode` external.
Extension packaging	@vscode/vsce 2.24	Produces the `.vsix`.
Validation	Node built-ins	Hand-written Draft-07 subset validator in `scripts/validate.mjs`.
Registry serving (dev)	Node `http`	`scripts/serve.mjs`.

8.2 Frameworks and platforms

- **Target framework:** React Native, generated through Expo. Every recipe declares `framework: "react-native"` and `runtime: "expo"` (enforced by the JSON Schema enums).
- **Routing:** Expo Router is the primary target (`app/` directory conventions); react-navigation is available for two recipes.
- **Styling:** NativeWind (Tailwind-style utility classes) or React Native `StyleSheet`.
- **Editor platform:** VS Code Extension API.

8.3 Runtime requirements and entry points

Entry point	Runtime	Detail
`packages/cli/src/index.ts`	Node	CLI binary, shebang `#!/usr/bin/env node`, declares five subcommands.
`packages/vscode-extension/src/extension.ts`	VS Code	`activate()` registers commands, completion provider and status bar.
`packages/core/src/index.ts`	Node/VS Code	Re-exports types, context and engine.
`packages/registry/src/index.ts`	Node	Filesystem recipe loading; also exposes an HTTP fetch helper.
`scripts/build-registry.mjs`	Node	Builds the deployable registry folder.
`scripts/serve.mjs`	Node	Local static registry server on `PORT` (default 3000).

8.4 Architecture requirements

- A single shared generation engine must be used by all clients. The CLI and the extension both import `@expo-snips/core` and render with the same `getFilesForRecipe` function.
- The catalog must be consumable both as a bundle embedded in the clients and as static JSON over HTTP.
- Recipe definitions must not require filesystem access at render time; all `.hbs` templates are inlined into the emitted recipe JSON at build time (`scripts/lib/build-recipes.mjs`).
- There is no server-side state; all state is local to the user's project.

8.5 Interfaces and APIs

Table 8.3: Interface inventory

Interface	Definition
CLI	Five subcommands with documented flags; see Chapter 14.
Registry HTTP	`GET {base}/index.json`; `GET {base}/recipes/{url-encoded-id}.json`. See Chapter 16.
Programmatic	`@expo-snips/core` exports `renderTemplate`, `getFilesForRecipe`, `GenerationContext`, `ScreenRecipe`, `FileTemplate`, `ScreenVariant` and the option union types.
Extension commands	`expo-snips.addScreen` and `expo-snips.generateFromCompletion` (the latter is not declared in `contributes.commands`, only registered at runtime).
Configuration	Six VS Code settings; two environment variables; two secrets used in CI. See Chapter 30.

8.6 Data requirements

- **No database.** NOT APPLICABLE.

- **Persistent state:** a JSON manifest (`.expo-snips.json`) written to the user's project root. Its shape is defined in Section 5.8 of Chapter 6.
- **Catalog data:** JSON recipe documents plus inlined Handlebars template strings.
- **Schema:** `packages/registry/schema.json` (JSON Schema Draft-07).

8.7 Authentication and authorization requirements

The product itself requires no authentication and implements no authorization. **NOT APPLICABLE.** See Chapters 17 and 18 for why, and for the authentication templates that are generated *for* user applications.

8.8 Infrastructure requirements

- **Build:** Node 18+ and pnpm 9+.
- **Distribution:** npm registry for the two public packages; GitHub Pages for the registry; manual `.vsix` for the extension.
- **CI:** GitHub Actions, `ubuntu-latest` runners.
- **No containerisation, orchestration or cloud infrastructure is required or present.**

8.9 Storage requirements

None beyond local disk. NOT APPLICABLE for object storage, databases or caches (a 5-minute in-memory cache in the extension aside).

8.10 Monitoring requirements

UNKNOWN / not implemented. There is no monitoring, alerting, error reporting or health endpoint.

8.11 Testing requirements

Not implemented. There are no test files, no test runner and no test script. The only automated check in the repository is the recipe validator (`scripts/validate.mjs`), which verifies recipe structure against the JSON Schema and is exposed as `pnpm validate`. See Chapter 26.

8.12 Third-party integration requirements

Table 8.4: Integrations

Integration	Direction	Used for
npm registry	outbound (publish)	Publishing `@expo-snips/core` and `@expo-snips/cli`.
GitHub Pages	outbound (deploy)	Hosting the static registry.
GitHub Actions	build	CI/CD.

Integration	Direction	Used for
Registry HTTP endpoint	inbound (fetch)	Optional remote recipe source.
create-expo-app	outbound (exec)	`init` project bootstrap.
Package managers	outbound (exec)	Installing recipe dependencies.
VS Code API	host	Extension surface.

Detailed in Chapter 29.

9 System Architecture

9.1 Overview

Expo Snips is a two-client, one-engine system. There is no server tier: the only network-facing component is a set of static JSON documents, and both clients can operate entirely without it because the catalog is embedded at build time.

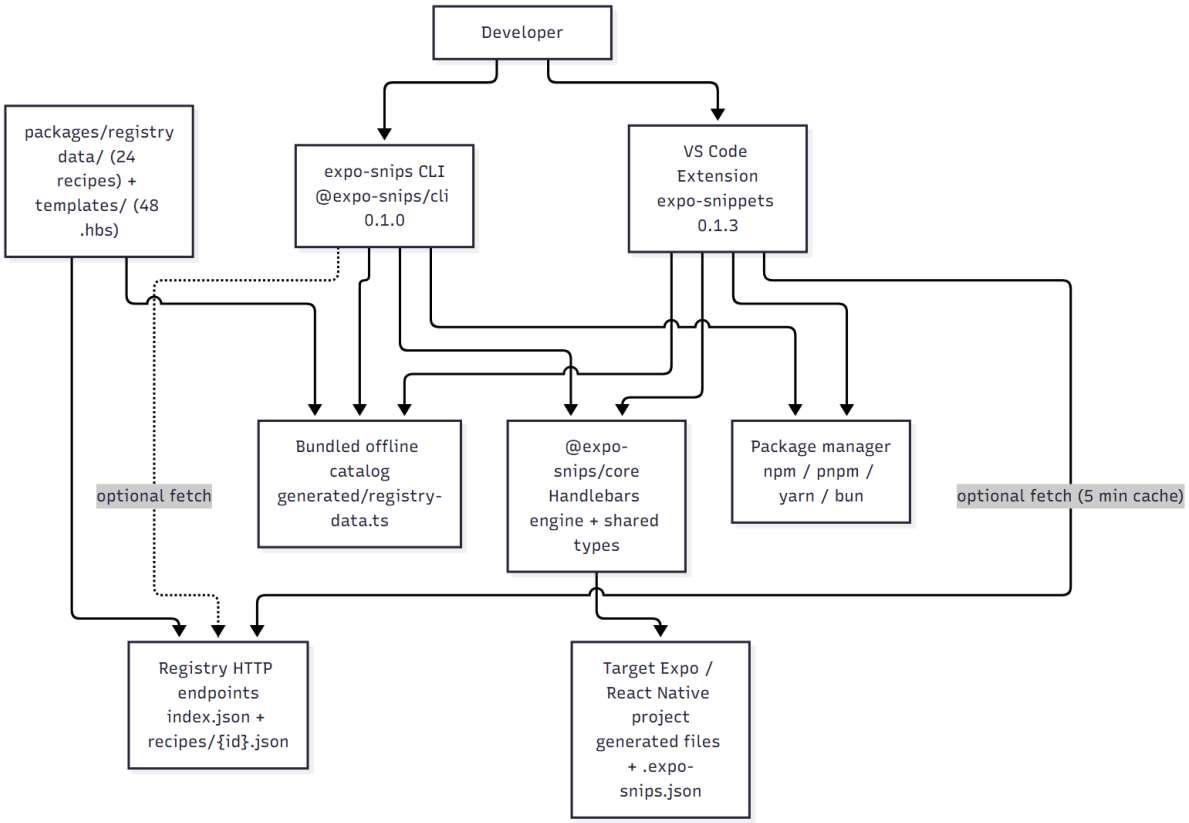


Figure 9.1: System architecture: clients, shared engine, catalog sources and target project

Table 9.1: System elements

Element	Responsibility and evidence
CLI client	Command parsing, interactive prompts, stack detection, orchestration and terminal reporting. `packages/cli/src`.
Extension client	Command palette, completion trigger, status bar, workspace detection, orchestration in an editor context. `packages/vscode-extension/src`.
Generation engine	Condition filtering and Handlebars rendering; pure functions with no I/O. `packages/core/src`.

Element	Responsibility and evidence
Offline catalog	Recipe arrays inlined into each client at build time (`src/generated/registry-data.ts`, git-ignored, produced by `scripts/prepare-registry.mjs`).
Registry service	Static JSON published over HTTP: `index.json` plus per-recipe documents. `registry/`; default base `https://registry.expo-snips.dev`.
Catalyst / build pipeline	Recipe source data and templates compiled into the registry folder and into the embedded bundles. `scripts/build-registry.mjs`, `scripts/lib/build-recipes.mjs`.
Target project	The user's Expo / React Native application, which receives generated files and the `.expo-snips.json` manifest.
Package manager	External process used to install recipe dependencies.

9.2 Deployment topology

- **Clients** execute on the developer's machine (Windows, macOS or Linux).
- **Registry** is static hosting. The repository's own workflow deploys it to GitHub Pages under a `registry/` path; the code also supports any base URL, including a local server started with `pnpm serve` (default `http://localhost:3000`).
- **Packages** are distributed through the public npm registry.
- **No runtime service, message broker, cache or database exists in the deployed system.**

9.3 Communication paths

Table 9.2: Communication paths

From	To	Mechanism
CLI	Registry	HTTPS GET, 10 s abort timeout, optional; falls back to embedded catalog.
Extension	Registry	HTTPS GET, 10 s abort timeout, 5-minute cache and back-off, optional.
CLI	Package manager	Child process (`execSync`, inherited stdio).
CLI	create-expo-app	Child process via `npx` (`init` only).
Extension	Package manager	Terminal command injection (`terminal.sendText`).
CLI/Extension	Target project	Local filesystem writes.
Clients	`@expo-snips/core`	In-process function calls (TypeScript module import).
Build pipeline	Registry	Local filesystem writes during CI.

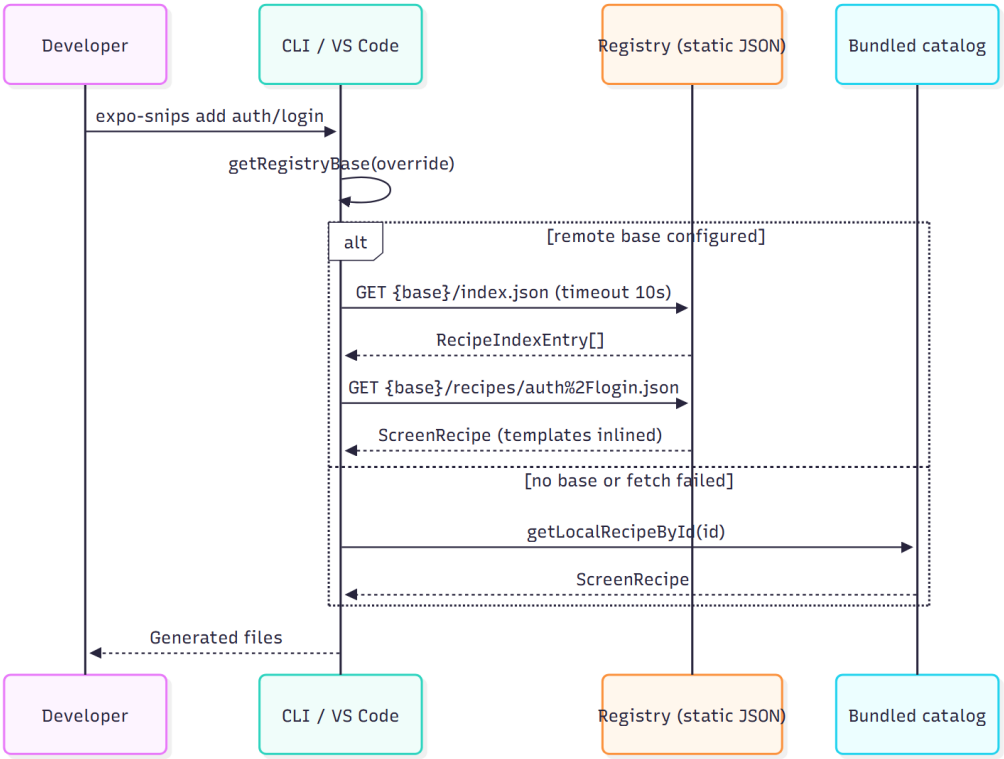


Figure 9.2: Registry resolution sequence with bundled-catalog fallback

9.4 Architectural characteristics

Table 9.3: Architectural characteristics

Characteristic	Assessment
Distribution style	Local-first, offline-capable.
State	Stateless process; persistence only in the user's project manifest.
Coupling	Low-to-moderate: clients depend on the engine and on the recipe schema; the engine has no dependency on either client.
Cohesion	High within each package; the recipe format is the stable contract.
Scalability	The catalog scales by adding files; there is no runtime scaling concern because there is no server.
Failure isolation	Catalog fetch failure is contained; a failed install aborts the CLI command after files have been written.
Trust boundary	The registry is trusted without verification — the principal architectural risk (see Chapter 23).

9.5 Key architectural decisions

Table 9.4: Architectural decisions and rationale

ID	Decision	Rationale / consequence
AD-01	Inline templates into recipe JSON at build time	Consumers never need filesystem access to templates; enables both HTTP delivery and embedded bundles. Evidenced by `build-recipes.mjs` and the emitted recipe files.
AD-02	Share one engine across clients	Guarantees identical output from CLI and editor. Both import `getFilesForRecipe`.
AD-03	Bundle the catalog in clients as well as hosting it	Guarantees offline operation; remote registry becomes an enhancement rather than a dependency.
AD-04	Recipes are data, not code	New screens need no engine change; validated by schema. Trade-off: template strings are embedded in JSON, which is awkward to edit and diff but trivially portable.
AD-05	Use Handlebars rather than a stronger typed generator	Minimal dependency footprint in the engine (one runtime dependency) at the cost of limited type safety inside templates (INFERRED).
AD-06	Force `nativewind` + `lucide` when `ui=shadcn`	The shadcn-style primitives depend on NativeWind utility classes and lucide icons; enforced rather than validated.
AD-07	Persist installs in a project-local manifest	Enables update and status workflows without any server-side account.

9.6 Constraints and limitations of the architecture

- No plugin or extension mechanism exists inside the clients; adding a screen means adding catalogue data, not extending the CLI.
- The engine's generated output depends on the recipe's `when` conditions being written correctly; there is no schema check that every supported option combination yields at least one file for every required path.
- Because templates are inlined into JSON, template content is escaped and difficult to review in the built registry (see the `auth/login` built recipe, where multi-line templates appear as single escaped lines).
- The extension duplicates the CLI's detection and primitive-resolution logic rather than importing it, so the two can drift.

Status: IMPLEMENTED (the first, second and fourth items are directly observable); INFERRED (the third).

10 Software Architecture

10.1 Package structure

The repository is a pnpm workspace with four packages. Dependencies flow in one direction only: every package may depend on `@expo-snips/core`; the core depends on nothing from the workspace.

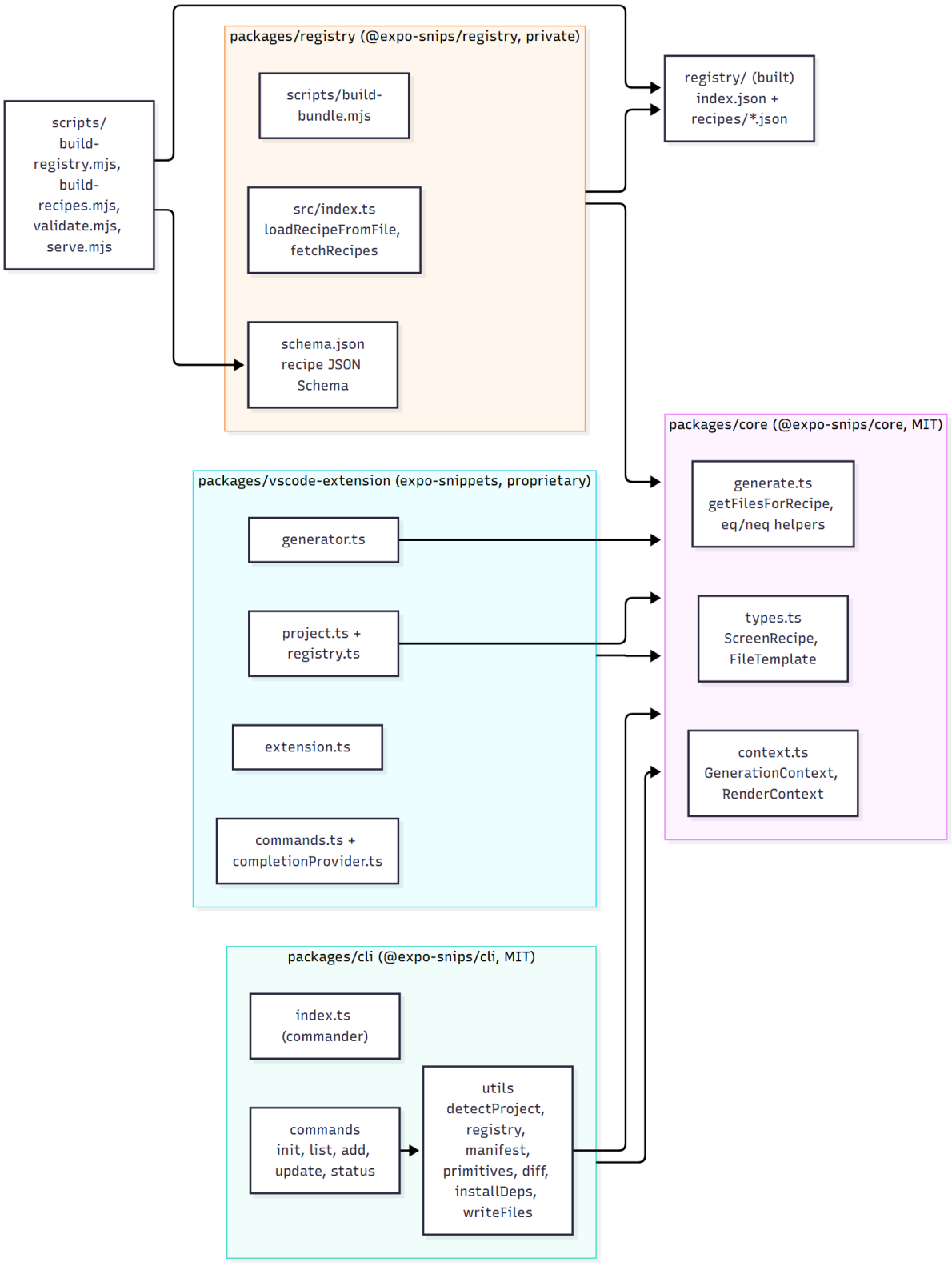


Figure 10.1: Component architecture of the pnpm workspace

Table 10.1: Package inventory

Package	Name	Visibility	Responsibility
`packages/core`	`@expo-snips/core`	public (MIT)	Shared types, generation context and the Handlebars rendering engine.
`packages/cli`	`@expo-snips/cli`	public (MIT)	Terminal client: commands, detection, orchestration, manifest, diff.
`packages/registry`	`@expo-snips/registry`	private	Recipe data (24), templates (48), JSON Schema, bundle builder, filesystem + HTTP loaders.
`packages/vscode-extension`	`expo-snippets`	proprietary	VS Code client: commands, completion, status bar, workspace detection, generation.

Workspace dependency declarations: `"@expo-snips/core": "workspace:*"` in `packages/cli/package.json` and `packages/registry/package.json`; a development dependency on `@expo-snips/core` in `packages/vscode-extension/package.json` (the engine is bundled into the extension by esbuild).

10.2 Module boundaries

`@expo-snips/core`

Module	Contents
`types.ts`	`StylingOption`, `RouterOption`, `UIOption`, `IconOption`, `Conditions`, `FileTemplate` (with `perStyling` and `variants` override maps), `ScreenVariant`, `ScreenRecipe`.
`context.ts`	`GenerationOptions`, `GenerationContext` (project root, options, optional variant), `RenderContext`.
`generate.ts`	Registers the `eq`/`neq` Handlebars helpers; exposes `renderTemplate`, `getFilesForRecipe`; privately `resolveTemplateString` (variant → per-styling → template precedence) and `matchesCondition`.
`index.ts`	Barrel re-export of the three modules above.

The core performs no I/O and has no knowledge of the CLI, the extension or the filesystem. This makes it the natural unit-test target, although no tests exist.

`@expo-snips/cli`

Module	Contents
`index.ts`	Creates the `Command` program, sets name/description/version, registers the five subcommands.
`commands/init.ts`	Project bootstrap via `create-expo-app`; refuses when `app.json` exists.
`commands/list.ts`	Prints the resolved catalog with source attribution.
`commands/add.ts`	The main generation command: registry resolution, detection, option resolution, prompts, rendering, writing, primitives, install, manifest.
`commands/status.ts`	Prints manifest entries.
`commands/update.ts`	Re-renders tracked screens, diffs, confirms, writes, bumps versions.
`utils/detectProject.ts`	`detectProject(dir)` — reads `package.json` and `tsconfig.json`.
`utils/registry.ts`	Registry base resolution, remote fetch with timeout, embedded-catalog fallback.
`utils/resolveRegistry.ts`	Wraps index resolution with user-facing source reporting.
`utils/manifest.ts`	Manifest read/write/upsert/version-bump.
`utils/primitives.ts`	Recursive primitive resolution.
`utils/installDeps.ts`	Package-manager detection and dependency installation.
`utils/writeFiles.ts`	Directory creation and file writing with per-file logging.
`utils/diff.ts`	LCS line diff and coloured printing.
`generated/registry-data.ts`	Build-time generated embedded catalog (git-ignored).

`@expo-snips/registry`

- `src/index.ts` — `loadRecipeFromFile` (inlines `templateFile`, variant and per-styling references), `getAllRecipes`, `getRecipeById`, `searchRecipes` and `fetchRecipes` (accepts an array, a `{recipes}` bundle or a single recipe object).
- `schema.json` — the recipe JSON Schema used by the validator.
- `data/**` — 24 recipe source files organised by category.
- `templates/**` — 48 Handlebars templates organised by category.
- `scripts/build-bundle.mjs` — writes `bundle.json` (a `{version, name, recipes}` bundle, 24 recipes).

`expo-snippets` (VS Code extension)

Module	Contents
`extension.ts`	`activate()`: registers commands and completion provider, creates the status bar item; `deactivate()` is empty.
`commands.ts`	`expo-snips.addScreen` (Quick Pick) and `expo-snips.generateFromCompletion`.

Module	Contents
`completionProvider.ts`	Completion provider for the four TS/JS language ids, triggered on `:`, matching `snip:` / `expo-snips:`.
`generator.ts`	`resolveOptions`, `pickVariant`, `detectPackageManager`, `installDependencies`, `resolvePrimitives`/`getPrimitives` and `generateScreen`.
`project.ts`	`getWorkspaceRoot`, `detectProject` and the manifest functions.
`registry.ts`	Embedded catalog, remote index/recipe fetch, 5-minute cache and back-off.

10.3 Dependency graph

Table 10.5: Runtime and build dependencies

Package	Runtime dependencies	Dev / build dependencies
core	handlebars ^4.7.8	typescript ^5.5
cli	@expo-snips/core (workspace), chalk ^4.1.2, commander ^12, fs-extra ^11.2, prompts ^2.4.2	@types/fs-extra, @types/node ^20, @types/prompts, typescript ^5.5
registry	@expo-snips/core (workspace)	@types/node ^20, typescript ^5.5
vscode-extension	(none at runtime; engine bundled)	@expo-snips/core, @types/fs-extra, @types/node ^20, @types/vscode ^1.80, @vscode/vsce ^2.24, esbuild ^0.21.5, fs-extra ^11.2, typescript ^5.5

10.4 Build pipeline

Package	Build command and behaviour
core	`tsc -p tsconfig.json` — emits `dist/` with declarations and source maps.
cli	`node scripts/prepare-registry.mjs && tsc -p tsconfig.json` — regenerates the embedded catalog, then compiles.
registry	`tsc` (library build) and `node scripts/build-bundle.mjs` (`build:bundle`).

Package	Build command and behaviour
vscode-extension	`node esbuild.mjs` — runs `prepareRegistry()` first, then bundles to `out/extension.js`.
root	`pnpm -r build`; also `validate`, `build:registry` and `serve` scripts.

10.5 Code-size profile

The hand-written TypeScript across the workspace (excluding type declarations and generated catalog modules) is approximately 1,600 lines. The largest single source files are `generator.ts` (268 lines), `add.ts` (279 lines) and `update.ts` (150 lines). No file is pathological in size; the largest codebase element by far is the recipe data and template set (24 recipes and 48 templates).

Status: IMPLEMENTED (measured) / INFERRED (assessment that no file is pathological).

11 High-Level Design

11.1 Design intent

The high-level design separates four concerns:

1. **Catalog** — what screens exist and what they need (data only).
2. **Engine** — how a recipe becomes files (pure transformation).
3. **Clients** — how a developer reaches the engine (I/O and UX).
4. **Delivery** — how catalog and packages reach the developer (build and CI).

This separation is what allows the same recipe to be rendered by a terminal program and by an editor extension with identical results.

11.2 End-to-end generation flow

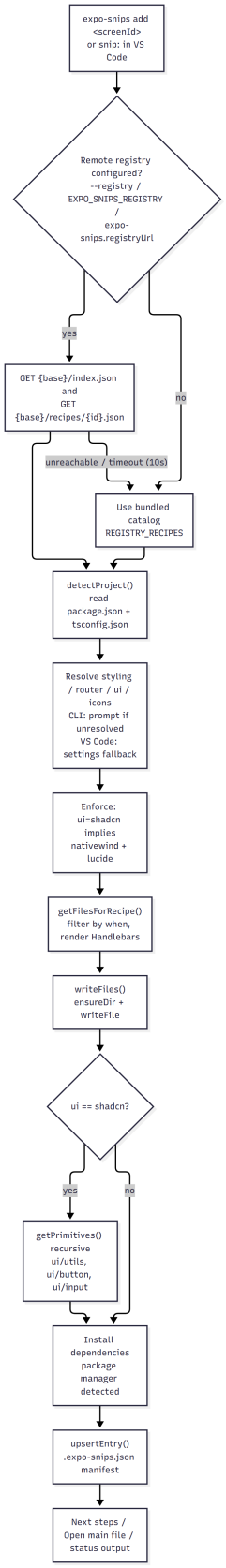


Figure 11.1: High-level generation pipeline from command to persisted manifest

The pipeline has seven stages, each implemented in a single, identifiable place:

Table 11.1: Pipeline stages

#	Stage	Implementation
1	Resolve input	Parse the screen id and flags; fetch or select the recipe (`add.ts`, `registry.ts`).
2	Detect environment	Read `package.json` / `tsconfig.json` (`detectProject.ts`).
3	Resolve options	Apply precedence rules and prompts; enforce shadcn constraints (`add.ts`).
4	Render	Filter file entries by `when`; select template; compile with Handlebars (`core/src/generate.ts`).
5	Persist files	Create directories and write content (`writeFiles.ts`).
6	Resolve primitives	Recursively emit shadcn primitives, skipping existing files (`primitives.ts`).
7	Finalise	Install dependencies and record the install (`installDeps.ts`, `manifest.ts`).

11.3 Major services and how they communicate

There are no long-running services. The system's "services" are libraries and one static endpoint.

Table 11.2: Service-equivalent components and their interactions

Component	Interaction style	Peer
Generation engine	In-process function calls	CLI client, extension client
Embedded catalog	In-memory module import	CLI client, extension client
Registry endpoint	HTTP GET, JSON, read-only	Both clients (optional)
Package manager	Child process / terminal text	CLI client, extension client
create-expo-app	Child process	CLI client
VS Code host	Extension API	Extension client

11.4 Data model at a high level

- **Recipe** — identity, version, supported stacks, dependencies, file templates (with conditions and overrides), optional variants, tags and primitive references. Defined by `ScreenRecipe` in `core/src/types.ts` and constrained by `registry/schema.json`.
- **Catalog index entry** — the recipe subset needed for browsing: id, name, description, version, typescript flag, tags, supported stacks.
- **Generated file** — a path plus rendered content.
- **Manifest entry** — the record that a recipe was installed into a project.

11.5 Cross-cutting concerns

Table 11.3: Cross-cutting concerns and their handling

Concern	Handling
Error reporting	Coloured console output (CLI); VS Code error/information messages (extension). Non-zero exit code on failure.
Configuration	CLI flags and `EXPO_SNIPS_REGISTRY`; six VS Code settings. No config files.
Logging	Untyped console output; no log levels or destinations.
Offline behaviour	Embedded catalog; graceful fallback on every network failure.
Versioning	Semver on recipes and packages; the manifest stores the installed recipe version.
Extensibility	New recipes and new templates require no code change; new options would require type, schema, CLI-choice and settings changes.

11.6 Design trade-offs

Table 11.4: Principal design trade-offs

Choice	Benefit	Cost
Templates in JSON (inlined)	Portable, dependency-free rendering; works offline and over HTTP.	Poor diffs and review ergonomics in the built registry; JSON escaping of large templates.
Handlebars	One dependency; familiar syntax; easy recipe authoring.	Unchecked at compile time; typos surface only at render time.
Option enforcement instead of validation	Simpler command code.	Silent reprioritisation of user choices when `ui=shadcn` is selected.
Unconditional file writes on `add`	Simple, predictable generation.	A pre-existing file at a recipe path is overwritten with no warning.
Duplicated detection logic in the extension	Avoids a shared dependency on CLI internals.	Two implementations can diverge.

12 Low-Level Design

This chapter documents the significant functions, their signatures, their inputs and outputs, and the state they touch. Signatures are reproduced from the source.

12.1 @expo-snips/core

renderTemplate(templateStr, context)

```
export function renderTemplate(templateStr: string, context: RenderContext): string
```

Compiles the template with Handlebars and executes it against the render context. Compilation happens per call; there is no cache.

getFilesForRecipe(recipe, context, resolveFile?)

```
export function getFilesForRecipe(
  recipe: ScreenRecipe,
  context: GenerationContext,
  resolveFile?: (file: FileTemplate) => string | null
): Array<{ path: string; content: string }>
```

Algorithm:

1. For each entry in `recipe.files`, evaluate `when` via `matchesCondition`; skip on mismatch.
2. Resolve the template string with `resolveTemplateString`: variant override, then per-styling override, then `template`.
3. If the template is still null and `templateFile` is set, delegate to the optional `resolveFile` callback (used when templates were not inlined).
4. Skip the entry if no template could be resolved.
5. Render and append `{path, content}`.

Only `path` and `content` are returned; conditions and metadata are discarded. Consequently two file entries targeting the same path (a common recipe pattern — one entry per supported stack) are filtered down to the single entry whose conditions match.

matchesCondition (private)

Evaluates five optional predicates in sequence — styling, router, ui, icons and typescript — returning false on the first mismatch. TypeScript comparison is an exact equality check, so `typescript: false` matches only JavaScript generation.

buildRenderContext (private)

Projects the generation context into the render context by duplicating the option values at the top level (`styling`, `router`, `ui`, `icons`, `typescript`) so templates can be written as `{#if (eq styling "nativewind")}` without an options prefix.

12.2 @expo-snips/cli — utilities

Table 12.1: CLI utility functions

Function	Behaviour
`detectProject(dir?)`	Reads `package.json` for four dependency probes and `tsconfig.json` for the language flag; returns `DetectedProject`. Defaults: ui `native`, icons `none`, styling `stylesheet` when no `package.json` exists.
`getRegistryBase(override?)`	Returns `(override || env || "").trim()` with trailing slashes removed.
`hasLocalRegistry()`	True when the embedded catalog is non-empty.
`getLocalRecipeById(id)`	Map lookup into the embedded catalog.
`fetchJSON<T>(url)`	`fetch` with a 10-second `AbortController` timeout; returns null on non-OK responses.
`resolveRecipeIndex(base?)`	Remote fetch first; falls back to the local index; reports `source` as `"remote"` or `"local"`.
`getRecipeIndex(base?)`	Index only.
`getRecipeById(id, base?)`	Remote recipe fetch, then embedded lookup.
`getRecipesByIds(ids, base?)`	Parallel resolve, filtering nulls.
`resolveRegistry(url?)`	Calls `resolveRecipeIndex` and prints a source message.
`readManifest(dir?)`	Returns `{version:1, installs:[]}` when the file is absent or unparseable.
`upsertEntry(entry, dir?)`	Replaces the entry matching (`recipeId`, `variant`) or appends.
`updateEntryVersions(entries, dir?)`	Rewrites `recipeVersion` for matching entries.
`getPrimitives(recipe, context, root, findRecipe)`	Recursive primitive resolution with a `Set` cycle guard; dedupes dependencies.
`installDependencies(deps, devDeps, dir?)`	Detects the package manager and runs `add` / `add -D`.
`writeFiles(files, root)`	`ensureDir` then `writeFile` per file, logging each path.
`diffLines(oldText, newText)`	Builds an $(n+1) \times (m+1)$ LCS table and walks it to emit add/remove/context hunks.
`printDiff(path, old, new)`	Prints coloured hunks; no output when nothing changed.
`hasChanges(old, new)`	Strict string inequality.

Manifest data structures

```
interface ManifestOptions { styling: string; router: string; ui: string; icons: string
    ; typescript: boolean; }
```

```
interface ManifestEntry { recipeId: string; recipeVersion: string; installedAt: string
    ; options: ManifestOptions; variant?: string; files: string[]; }
interface Manifest { version: number; installs: ManifestEntry[]; }
const MANIFEST_FILENAME = ".expo-snips.json";
```

The `add` command in detail

`add.ts` is the longest command and the best illustration of the CLI's control flow.

Table 12.2: `add` command steps

Lines	Step
68–70	Resolve the registry; prepare a `findRecipe` closure.
72–92	If no id was supplied, list the index in a select prompt; abort with exit code 1 if empty or cancelled.
94–100	Load the recipe; error with guidance if not found.
102	Detect the project.
104–117	Resolve styling; validate any explicit value; fall back to a detected value only when the recipe supports it.
119–159	Resolve router, ui and icons with the same pattern, including the "single supported value" shortcut.
161–165	Resolve language; warn when a TypeScript recipe is generated as JavaScript.
167–200	Prompt for any option still unresolved, offering only supported values and preselecting the detected one.
202–214	Enforce the shadcn constraints.
216–222	Build the generation context with `projectRoot: process.cwd()`.
224–230	Render; error when no files matched.
232–238	Announce the recipe and the effective option values.
238	Write files.
240–256	Resolve and write shadcn primitives; merge their dependencies; report skipped files.
258–260	Install dependencies unless `-no-install` was given.
262–272	Upsert the manifest entry.
274–278	Print success and next steps.

Observation. Styling resolution appears twice: lines 111–117 compute a detected styling, and lines 167–173 recompute the same value before prompting. The duplicate is harmless but redundant (recorded in Chapter 35).

12.3 The recipe primitive graph

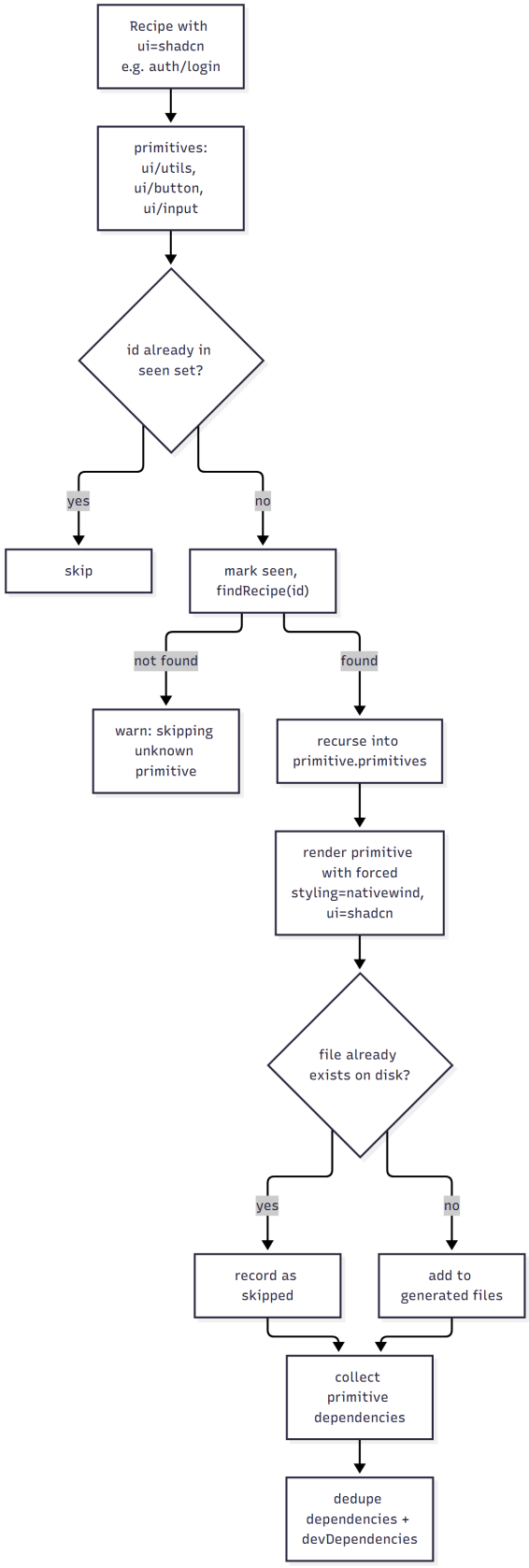


Figure 12.1: Recursive UI primitive resolution when `ui=shadcn`

12.4 expo-snippets — VS Code extension

Table 12.3: Extension functions

Function	Behaviour
`activate(context)`	Registers commands and the completion provider, creates a right-aligned status bar item (`$(sparkle) Expo Snips`) bound to `expo-snips.addScreen`, and pushes it to subscriptions.
`deactivate()`	Empty.
`registerCommands(context)`	Registers `expo-snips.addScreen` (index Quick Pick, with an "Open Settings" error action when the catalog is empty) and `expo-snips.generateFromCompletion(recipeId?)`.
`registerCompletionProvider(context)`	Language-filtered provider triggered on `:`; tests the line prefix against `/expo-snips:$\|snip:$/`; returns items whose `insertText` is empty and whose command carries the recipe id.
`pickSupported(supported, preferred, fallback)`	Returns the preferred value when supported, otherwise the first supported value, otherwise the fallback.
`resolveOptions(recipe, detected)`	Reads the four default settings and the language; applies `pickSupported`; enforces shadcn constraints.
`pickVariant(recipe)`	Returns undefined when no variants; otherwise a Quick Pick of variant labels.
`detectPackageManager(dir)`	Same lockfile probes as the CLI.
`installDependencies(dir, deps, devDeps)`	Creates and reveals an "Expo Snips" terminal and sends `add` commands.
`generateScreen(recipe)`	Resolves the workspace root, detects the project, resolves options and variant, renders, writes files under a progress notification, resolves primitives, upserts the manifest, shows an information message with an "Open main file" action, and installs dependencies when enabled.
`getWorkspaceRoot()`	Returns the first workspace folder's filesystem path.

Behavioural difference from the CLI. The extension never prompts for styling, router, UI or icons — it silently selects from the detected values and the configured defaults. The CLI prompts. This is the single largest behavioural divergence between the two surfaces.

12.5 State

Table 12.4: State inventory

State	Location	Lifetime
Remote index cache	`registry.ts` (extension)	Process; 5 minutes (`CACHE_TTL_MS`).

State	Location	Lifetime
Registry back-off timestamp	`remoteRetryAfter` (extension)	Process; 5 minutes after a failed fetch.
Embedded catalog maps	`localRecipes`, `localById` (both clients)	Module lifetime.
Generation context	Function local (`add.ts`, `generator.ts`)	Single generation.
Manifest	`.expo-snips.json` on disk	Permanent until deleted.
Generated files	The user's project	Permanent.

No global mutable state exists in the CLI. The extension holds the two cache variables above at module scope.

12.6 Data access patterns

- **Reads:** `fs.readJSON` / `fs.existsSync` / `fs.pathExists` for project detection and manifest reads; `fs.readFileSync` for diff comparison; `fetch` for the remote registry.
- **Writes:** `fs.ensureDir` + `fs.writeFile` for generated files; `fs.writeJSONSync` for the manifest.
- **Catalog access:** synchronous filesystem reads in `@expo-snips/registry` (build-time only) and in-memory lookups in the clients.

There is no transaction semantics, no locking and no atomic write; `writeFile` overwrites in place.

12.7 Dependencies between modules

Table 12.5: Module-level import graph (workspace)

Module	Imports from workspace
`core/src/*`	none
`cli/src/commands/*`	`@expo-snips/core`, `../utils/*`
`cli/src/utils/*`	`@expo-snips/core`, `./generated/registry-data`
`registry/src/index.ts`	`@expo-snips/core`
`vscode-extension/src/*`	`@expo-snips/core`, `./generated/registry-data`

13 Client and Tooling Architecture

The product has no web or mobile user interface of its own. "Frontend" here means the two developer-facing surfaces: the command-line interface and the VS Code extension. Both are thin orchestration layers over the shared engine documented in Chapter 10.

13.1 Surface comparison

Table 13.1: CLI versus VS Code extension

Aspect	CLI	VS Code extension
Package	`@expo-snips/cli`	`expo-snippets`
Version	0.1.0	0.1.3
Entry point	`src/index.ts` (commander)	`src/extension.ts` (`activate`)
Engine	imported `@expo-snips/core`	same, bundled by esbuild
Discovery	`list`; interactive select in `add`	completion, Quick Pick, status bar
Option resolution	prompts when unresolved	detection plus settings defaults, never prompts
Variant selection	`-variant` flag	Quick Pick when variants exist
Update and status	implemented	**not implemented**
Project root	`process.cwd()`	first workspace folder
Dependency install	`execSync`, blocking, inherited stdio	terminal command, non-blocking
Offline catalog	embedded at build	embedded at build
Registry fetch	every invocation (no cache)	cached 5 minutes with back-off
Failure reporting	coloured stderr, exit code	VS Code error/information notifications

13.2 Command-line interface architecture

The CLI is a single-process program. Startup cost is the commander registration plus a static module import of the embedded catalog.

- **Parsing:** commander defines the program and five subcommands.
- **Interaction:** `prompts` drives the select prompts in `add` and the confirm prompt in `update`.
- **Presentation:** `chalk` colours informational (cyan), success (green), warning (yellow) and error (red) output.
- **Exit codes:** 0 on success; 1 on any handled error (`process.exitCode = 1`).

13.3 VS Code extension architecture

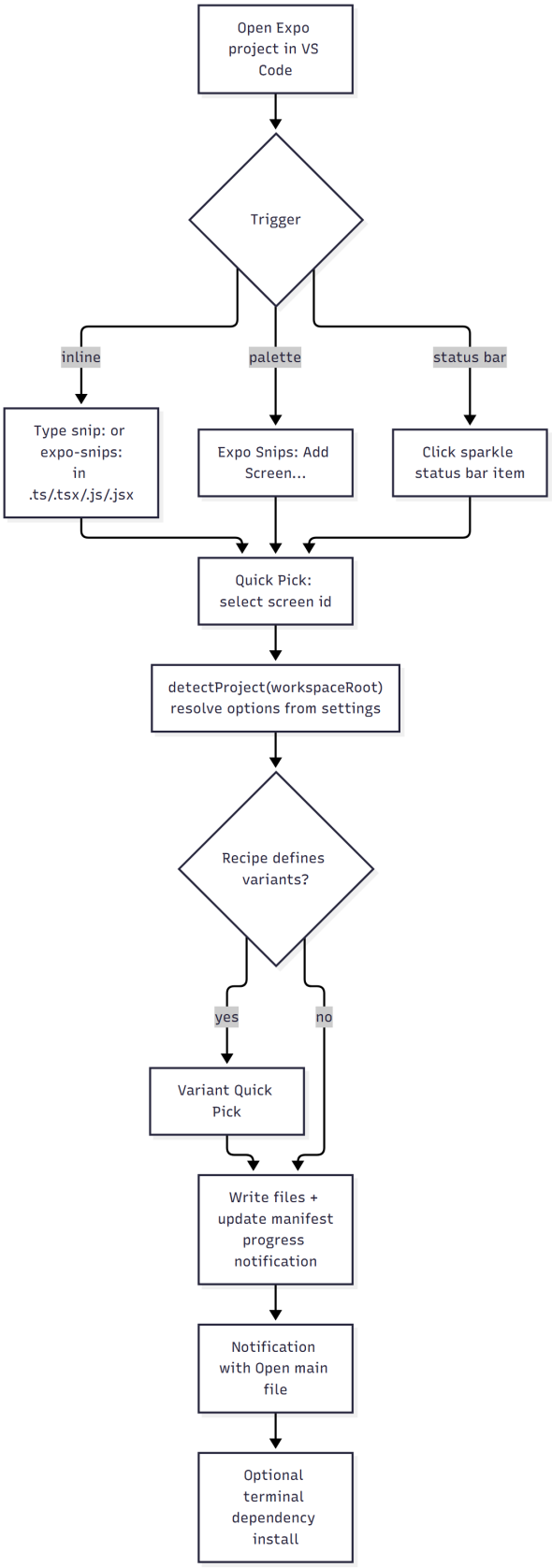


Figure 13.1: VS Code extension interaction flow

Activation

The extension declares four activation events (`packages/vscode-extension/package.json`, lines 74–80): `onCommand:expo-snips.addScreen` and `onLanguage:` for TypeScript, TypeScript React, JavaScript and JavaScript React. It therefore activates when a relevant file is opened or when the command is invoked first.

Surfaces

1. **Inline completion.** Registered for the four language ids and triggered on the `:` character. The provider inspects the text before the cursor; on a match it produces one completion item per catalog entry, labelled with the recipe id and described with the recipe name, documentation and tags. Because `insertText` is empty and the item carries a command, selecting it triggers generation rather than inserting text.
2. **Command palette.** `Expo Snips: Add Screen...` (declared in `contributes.commands`) opens a Quick Pick with matching on description enabled.
3. **Status bar.** A right-aligned item at priority 100, showing the `sparkle` codicon and the text "Expo Snips", tooltip "Add an Expo Snip screen", bound to the add-screen command.
4. **Post-generation notification.** Reports the recipe name and file count and offers a single action, "Open main file", which opens the first written path.

Command identifiers

Table 13.2: VS Code command identifiers

Command id	Declared in `contributes.commands`?
`expo-snips.addScreen`	yes
`expo-snips.generateFromCompletion`	no — registered at runtime only

The second command is intentionally internal (it exists so completion items can dispatch a generation). Because it is not contributed, it is invisible in the command palette, which is the desired behaviour.

Progress and feedback

Generation runs inside `vscode.window.withProgress` with a notification location and the title "Expo Snips: generating {recipe name}...", so file writes and primitive resolution are covered by a progress indicator.

13.4 Shared client concerns

Project detection

Both clients implement the same five probes. The implementations are separate files with near-identical bodies (`cli/src/utils/detectProject.ts` and `vscode-extension/src/project.ts`). Only `dependencies` is inspected; packages present solely under `devDependencies` are not detected.

Registry access

Both clients:

- keep an embedded catalog as the default;
- accept a base URL from different sources (flag/env for the CLI, a setting for the extension);
- normalise the base by trimming and stripping trailing slashes;
- use a 10-second abort timeout;
- fall back to the embedded catalog on any failure.

Differences: the extension caches and backs off; the CLI does not cache.

Manifest

Both clients read and upsert `.expo-snips.json` with the same shape and the same upsert key. The extension does not implement version bumping because it has no update command.

13.5 Presentation layer of generated screens

Generated screens are React Native components. Their structure — components, styles, states and interactions — is documented as a design system in Chapter 22. The templates consistently follow a presentational split:

- a **screen** file (route-level, handles layout, safe area and keyboard avoidance);
- a **form or list item component** (encapsulates field or row rendering);
- a **hook** where async behaviour exists (for example `useLogin`, `useRegister`);
- inline sample data constants (for example `stats`, `upcoming`, product arrays) that the developer replaces.

This is IMPLEMENTED and observable across `packages/registry/templates/**`.

14 CLI Reference

14.1 Invocation

```
expo-snips <command> [options]
```

The binary is declared as `expo-snips` in `packages/cli/package.json` (`bin`), and the program name and description are set in `packages/cli/src/index.ts`:

```
program
  .name("expo-snips")
  .description("Beautiful React Native screens, one command away.")
  .version("0.1.0");
```

Note. The version string is hard-coded in the source rather than read from `package.json`, so it must be kept in sync manually (see Chapter 35).

14.2 Command summary

Table 14.1: Command summary

Command	Argument	Purpose
`init`	none	Create a new Expo project using `create-expo-app`.
`list`	none	List all available screens from the resolved catalog.
`add`	`[screenId]`	Generate a screen into the current project.
`update`	`[screenId]`	Update tracked screens to the latest recipe versions.
`status`	none	Show screens tracked by `.expo-snips.json`.

14.3 `expo-snips init`

```
expo-snips init [--name <name>] [--template <template>]
```

Table 14.2: `init` options

Option	Default	Behaviour
`-name <name>`	unset	Project directory name; when unset the argument `.` is used (current directory).
`-template <template>`	`default`	Passed to `create-expo-app` as `-template <template>`.

Behaviour: if `app.json` already exists in the current working directory the command prints "An Expo project already exists in this directory." and returns without doing anything. Otherwise it executes:

```
npx create-expo-app@latest <name|.> --template <template> --yes
```

with inherited stdio, then prints next-step guidance. No exit code other than 0 is set by this command; a failure inside `create-expo-app` propagates as a process error from `execSync`.

14.4 expo-snips list

```
expo-snips list [--registry <url>]
```

Resolves the catalog, reports the source ("remote"/"local"), and prints one line per recipe: the id padded to 24 characters, the display name, and the tags in square brackets when present.

```
Available screens (24) from local registry:

  app/home                 Home Dashboard Screen [app, home, dashboard]
  auth/login               Login Screen [auth, login, form]
  ...
```

Printing an empty catalog prints "No screens found in the registry." and returns success. See Section 14.8 for the discrepancy between this behaviour and the interactive picker's behaviour.

14.5 expo-snips add

```
expo-snips add [screenId] [options]
```

Table 14.3: `add` options

Option	Value set	Behaviour
`-styling <styling>`	nativewind, stylesheet, tamagui	Explicit styling; rejected when the recipe does not support it.
`-router <router>`	expo-router, react-navigation	Explicit router; rejected when unsupported.
`-ui <ui>`	native, shadcn, custom	Explicit UI primitives; rejected when unsupported.
`-icons <icons>`	lucide, vector-icons, none	Explicit icon library; rejected when unsupported.
`-variant <variant>`	recipe-defined	Screen variant id (for example `glass`).
`-typescript`	flag	Force TypeScript.
`-javascript`	flag	Force JavaScript (takes precedence over `-typescript`).
`-no-install`	flag	Skip dependency installation.
`-registry <url>`	URL	Remote registry base; defaults to `EXPO_SNIPS_REGISTRY`.

Rejected values. A value that the recipe does not list in `supported` aborts with a red error and exit code 1. This applies to `-styling`, `-router`, `-ui` and `-icons`.

Effective-value notices. Selecting `-ui shadcn` prints a note and overrides `styling` to `nativewind` and `icons` to `lucide`, whether or not those were supplied explicitly. A TypeScript recipe generated with `-javascript` prints a warning but proceeds.

Examples

```
# Interactive: pick a screen from a list
expo-snips add

# Named screen, defaults from project detection
expo-snips add auth/login

# Force NativeWind and no dependency installation
expo-snips add app/home --styling nativewind --no-install

# shadcn-style UI (forces nativewind + lucide; adds ui primitives)
expo-snips add auth/register --ui shadcn

# Glassmorphism variant
expo-snips add app/home --variant glass

# JavaScript output
expo-snips add auth/login --javascript

# Point at a locally served registry
expo-snips add app/home --registry http://localhost:3000
```

Variant availability. `-variant glass` has an effect only for recipes that declare a `glass` variant: `app/home`, `auth/login` and `onboarding/welcome`. For any other recipe the flag is accepted but produces default output, because the variant override map is empty.

14.6 expo-snips update

```
expo-snips update [screenId] [--registry <url>] [--yes]
```

Table 14.4: `update` options

Option	Value set	Behaviour
`[screenId]`	recipe id	Update only this recipe; error when no tracked entry matches.
`-registry <url>`	URL	Remote registry base.
`-yes`	flag	Apply without the confirmation prompt.

Behaviour:

- No tracked screens: prints guidance to use `add` and returns.
- Nothing changed: prints "All tracked screens are up to date."
- Changes found: prints the file count, a coloured diff per changed file, then a confirmation prompt defaulting to *no*; declining prints "Update cancelled."

- Applied: rewrites the files and prints the number of updated screens.
- A tracked recipe missing from the registry is skipped with a warning.

```
expo-snips update
expo-snips update auth/login
expo-snips update --yes
```

Important behaviour. `update` compares the newly rendered content with the file currently on disk. If the developer has edited a generated file, the update will overwrite those edits once applied. The diff is the only safeguard.

14.7 expo-snips status

```
expo-snips status
```

Reads `.expo-snips.json` from the current working directory and prints each entry as `<recipeId padded to 24> v<recipeVersion>` with an optional `(variant: <id>)` suffix, followed by a hint to run `expo-snips update`.

```
Tracked screens (2):

  auth/login               v0.3.0
  app/home                 v0.1.0 (variant: glass)

Run expo-snips update to update them to the latest version.
```

With no tracked screens it prints “No screens tracked in this project yet.”

14.8 Exit codes and error messages

Table 14.5: Failure conditions and messages

Condition	Message and exit code
Interactive selection cancelled	no message; exit code 1.
Screen id not found (`add`)	“Screen not found: <id>” plus a pointer to `list`; exit code 1.
Option unsupported by recipe	“Recipe "..." does not support <option> "...".” exit code 1.
No files matched the options	“No files matched the selected options. Nothing generated.”; exit code 1.
No tracked entries for a given `update <id>`	“No installed screens match "<id>".” plus pointer to `status`; exit code 1.
Empty catalog during interactive `add`	guidance to reinstall the CLI or configure a registry; exit code 1.
Empty catalog during `list`	“No screens found in the registry.”; exit code 0.

14.9 Version discrepancy note

The CLI's `-version` output is the literal string `0.1.0` defined in `src/index.ts`. The `package.json` also declares `0.1.0`, so they currently agree; however, changing one without the other would silently diverge. The VS Code extension avoids this class of problem by letting the editor read the version from its manifest.

15 Screen Registry and Recipe Model

15.1 Concept

The registry is a catalogue of *screen recipes*. A recipe is pure data: it declares what a screen is called, which stacks it supports, which npm packages it needs, and which files it produces. Files are produced by rendering Handlebars templates, and a recipe may define multiple entries for the same output path, each guarded by a `when` condition, so that one recipe yields stack-specific output.

15.2 Recipe schema

The schema is defined in `packages/registry/schema.json` (JSON Schema Draft-07) and mirrored by the TypeScript interface in `packages/core/src/types.ts`.

Table 15.1: Recipe fields

Field	Type	Required	Description
`id`	string	yes	Pattern `^[a-z0-9-]+/[a-z0-9-]+$`; must equal the file path relative to `data/` without the extension.
`name`	string	yes	Human-readable name, min length 3.
`description`	string	yes	Short description, min length 10.
`version`	string	yes	Semver pattern `^\d+\.\d+\.\d+$`.
`framework`	enum	yes	Only `"react-native"`.
`runtime`	enum	yes	Only `"expo"`.
`typescript`	boolean	no	Whether the recipe is authored TypeScript-first.
`supported`	object	yes	Four option arrays: styling, router, ui, icons.
`dependencies`	string[]	yes	npm packages installed with `add`.
`devDependencies`	string[]	no	npm packages installed with `add -D`.
`files`	array	yes	File entries (see below), min 1 item.
`variants`	array	no	Named variants, each `{id, label, description?}`.
`tags`	string[]	no	Search keywords.
`primitives`	string[]	no	Primitive recipe ids required when `ui === "shadcn"`.

File entry

Table 15.2: File entry fields

Field	Meaning
`path`	Output path relative to the project root.
`template`	Inline Handlebars template string (used at render time).
`templateFile`	Path to a `.hbs` file, relative to `packages/registry/templates`; inlined into `template` at build time and the field is deleted from the emitted recipe.
`perStyling`	Map from styling option to a template string or `.hbs` path (inlined at build time).
`when`	Condition object with optional `styling`, `router`, `ui`, `icons` arrays and `typescript` boolean.
`variants`	Map from variant id to a template string or `.hbs` path (inlined at build time).

Enumerations

Table 15.3: Option enumerations (from the schema and the core types)

Option	Allowed values
styling	`nativewind`, `stylesheet`, `tamagui`
router	`expo-router`, `react-navigation`
ui	`native`, `shadcn`, `custom`
icons	`lucide`, `vector-icons`, `none`

Implementation gap. `tamagui`, `vector-icons` and `custom` are legal per the schema and the type unions, and are offered in VS Code settings, but *no recipe declares them*, and the CLI prompt lists only `nativewind`/`stylesheet` and `lucide`/`none`. See Chapter 34.

15.3 Catalog statistics

Table 15.4: Catalog composition (verified)

Category	Recipes	Ids
app	5	`app/home`, `app/notifications`, `app/profile`, `app/search`, `app/settings`
auth	4	`auth/login`, `auth/register`, `auth/forgot-password`, `auth/otp-verification`
calendar	1	`calendar/schedule`
chat	1	`chat/conversations`
content	1	`content/article-reader`
dashboard	1	`dashboard/analytics`
ecommerce	2	`ecommerce/products`, `ecommerce/product-details`
map	1	`map/location`
onboarding	3	`onboarding/welcome`, `onboarding/features`, `onboarding/permissions`
ui	3	`ui/button`, `ui/input`, `ui/utils`
utility	2	`utility/empty`, `utility/error`
Total	**24**	—

Templates: **48** `.hbs` files under `packages/registry/templates`, organised by the same category structure.

15.4 Variants

Only three recipes declare a variant, all with the id `glass` and label "Glassmorphism": `app/home` (`app.home.index.glass.hbs`), `auth/login` (`app.login.index.glass.hbs`) and `onboarding/welcome` (`app.welcome.index.glass.hbs`).

The remaining twenty-one recipes declare `"variants": []`.

The glass templates render a dark surface (`#0F172A`) with translucent overlays (`rgba(255,255,255,0.08)`) and, for `auth/login`, import `BlurView` from `expo-blur`; note that `expo-blur` is not declared in that recipe's `dependencies` array (see Chapter 34).

15.5 Template organisation

```
packages/registry/templates/
  app/home/app.home.index.hbs
  app/home/app.home.index.glass.hbs
  auth/login/app.login.index.hbs
  auth/login/app.login.index.shadcn.hbs
  auth/login/app.login.index.glass.hbs
  auth/login/components.login-form.hbs
  auth/login/components.login-form.shadcn.hbs
```

```
auth/login/screens.loginscreen.react-navigation.hbs
auth/login/hooks.use-login.hbs
ui/button.hbs
ui/input.hbs
ui/lib.utils.hbs
...
```

The naming convention is `<path-with-dots>.<variant-suffix>.hbs`, for example `components.login-form.shadcn.hbs` for the shadcn variant of `components/LoginForm.tsx`.

15.6 Build and distribution

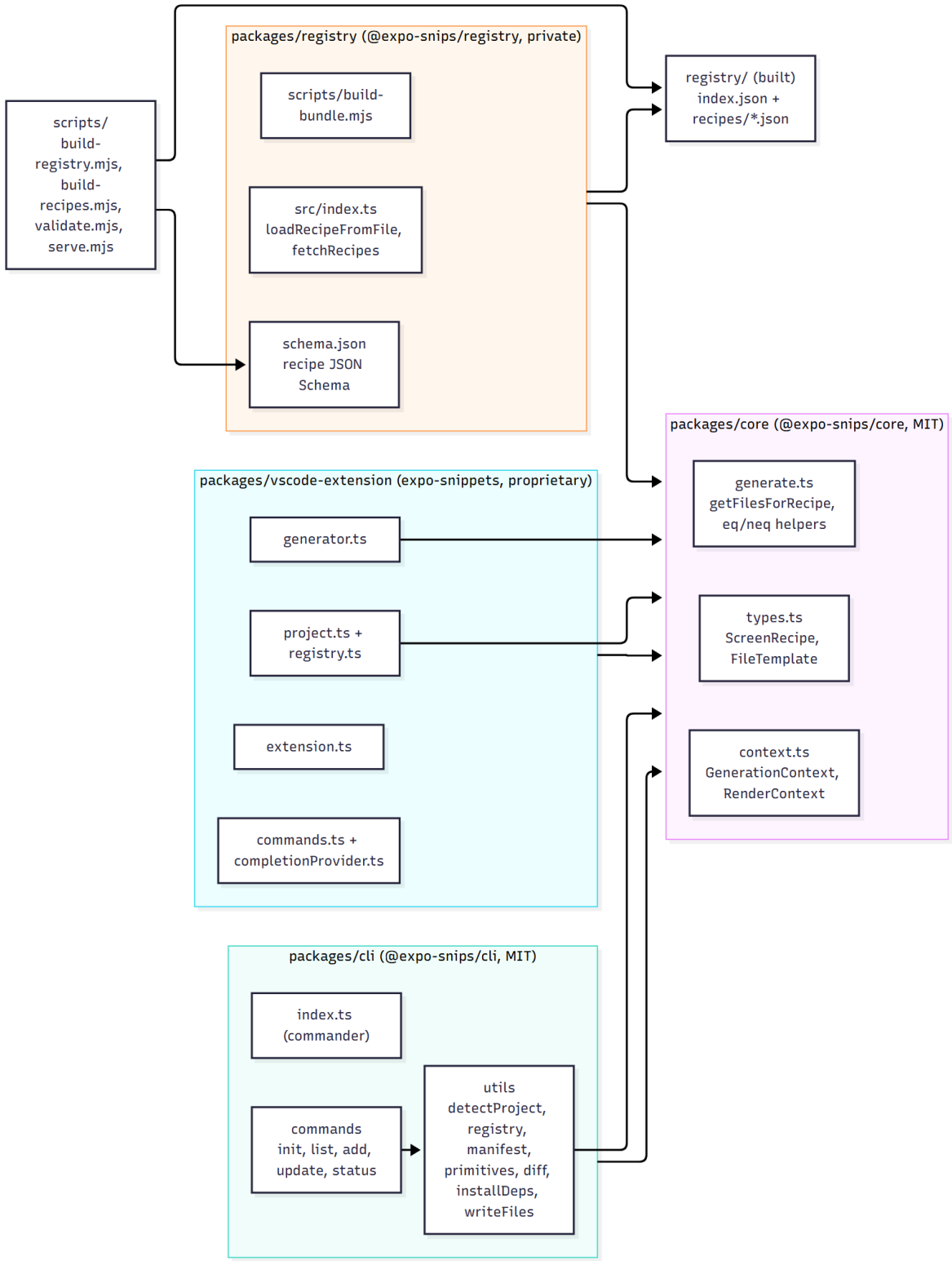


Figure 15.1: Recipe data flows into the built registry and into the embedded client bundles

Table 15.6: Build outputs derived from the recipe sources

Output	Producer and purpose
`registry/index.json`	`scripts/build-registry.mjs`. Index entries for all 24 recipes; served at `{base}/index.json`.
`registry/recipes/<id>.json`	`scripts/build-registry.mjs`. Full recipes with inlined templates; served at `{base}/recipes/<id>.json`.
`packages/registry/bundle.json`	`scripts/build-bundle.mjs`. A `{version:1, name, recipes}` bundle for offline consumption.
`packages/cli/src/generated/registry-data.ts`	`scripts/prepare-registry.mjs`. Embedded catalog for the CLI.
`packages/vscode-extension/src/generated/registry-data.ts`	Same script; embedded catalog for the extension.

All four derived outputs are produced from the single source of truth in `packages/registry/data` and `packages/registry/templates`. The generated per-client modules are git-ignored (`.gitignore`, lines 7–8). The built `registry/` folder is committed and is rebuilt (deleted first) by the build script.

15.7 Validation

`scripts/validate.mjs` (run with `pnpm validate`) implements a hand-written subset of JSON Schema Draft-07 validation. For each file under `data/` it checks:

- all required fields are present;
- `id` matches `^[a-z0-9-]+/[a-z0-9-]+$`;
- `version` matches the three-part semver pattern;
- `framework` is `react-native` and `runtime` is `expo`;
- every `supported.*` value is a legal enum member;
- `files` is a non-empty array and every entry has both `path` and `templateFile`;
- `id` equals the file's path relative to `data/`;
- every referenced template file exists on disk.

It prints a per-recipe pass/fail line and exits non-zero when any error is found. It is the only automated verification in the repository.

Note. Because the emitted recipes replace `templateFile` with `template`, validation must run against the *source* data directory; the validator is invoked by `pnpm validate` precisely for that reason.

16 API Documentation

The system exposes three classes of interface:

1. the **registry HTTP interface** (read-only static JSON);
2. the **programmatic interfaces** of the published packages;
3. the **CLI interface**, documented in Chapter 14.

There is no authenticated API, no write API and no server-side application.

16.1 Registry HTTP interface

Base URL resolution

Table 16.1: Registry base URL resolution

Consumer	Resolution chain
CLI	`-registry <url>` flag → `EXPO_SNIPS_REGISTRY` environment variable → empty (embedded catalog).
Extension	`expo-snips.registryUrl` setting → empty (embedded catalog).
Declared default constant	`https://registry.expo-snips.dev` (`DEFAULT_REGISTRY` in both clients) — exported but not used as an implicit default; an empty base selects the embedded catalog.
Dev server	`pnpm serve` → `http://localhost:3000`.

The base is normalised by trimming whitespace and removing trailing slashes.

`GET /index.json`

Aspect	Detail
Purpose	Return the catalog index used for browsing and selection.
Method	GET
Authentication	None.
Authorization	None (public read).
Headers	None required.
Parameters	None.
Request body	None.
Response	JSON array of index entries.
Timeout	Client-side abort after 10 000 ms.
Status codes	Any non-2xx is treated as failure by the client (`response.ok` check); the client then falls back to the embedded catalog.

Validation	The client accepts the payload only when it is an array with at least one element.
Source	Produced by `scripts/build-registry.mjs` via `toIndexEntry`.

Index entry shape:

```
interface RecipeIndexEntry {
  id: string;          // e.g. "auth/login"
  name: string;        // e.g. "Login Screen"
  description: string;
  version: string;     // semver, e.g. "0.3.0"
  typescript: boolean;
  tags: string[];
  supported: {
    styling: string[];
    router: string[];
    ui: string[];
    icons: string[];
  };
}
```

Example response fragment (from `registry/index.json`):

```
[
  {
    "id": "app/home",
    "name": "Home Dashboard Screen",
    "description": "Home screen with greeting, stats cards and upcoming tasks list.",
    "version": "0.1.0",
    "typescript": true,
    "tags": ["app", "home", "dashboard"],
    "supported": {
      "styling": ["nativewind", "stylesheet"],
      "router": ["expo-router"],
      "ui": ["native"],
      "icons": ["lucide", "none"]
    }
  }
]
```

Note. The index omits `variants`, so clients browsing only the index cannot tell whether a recipe offers variants. Only the full recipe document carries them.

`GET /recipes/{id}.json`

Aspect	Detail
Purpose	Return one complete recipe with templates inlined.
Method	GET
Path parameter	Recipe id, URL-encoded by the client (`encodeURIComponent`); note that the on-disk path uses a slash (`recipes/auth/login.json`) while the client encodes the slash as `%2F`.

Authentication	None.
Response	A `ScreenRecipe` object with `files[].template` strings.
Client acceptance	Accepted only when the payload is an object with a truthy `id`.
Fallback	Embedded catalog lookup by id.
Source	Written by `scripts/build-registry.mjs` to `registry/recipes/<id>.json`.

Example (abridged) response:

```
{
  "id": "auth/login",
  "name": "Login Screen",
  "version": "0.3.0",
  "framework": "react-native",
  "runtime": "expo",
  "typescript": true,
  "supported": {
    "styling": ["nativewind", "stylesheet"],
    "router": ["expo-router", "react-navigation"],
    "ui": ["native", "shadcn"],
    "icons": ["lucide", "none"]
  },
  "dependencies": ["react-hook-form", "zod", "@hookform/resolvers", "lucide-react-
    native"],
  "files": [
    {
      "path": "app/login/index.tsx",
      "when": { "router": ["expo-router"], "ui": ["native"] },
      "variants": { "glass": "import { ... } from 'react-native'; ..." },
      "template": "import { View, Text, ... } from 'react-native'; ..."
    }
  ],
  "variants": [{ "id": "glass", "label": "Glassmorphism", "description": "..." }],
  "tags": ["auth", "login", "form"],
  "primitives": ["ui/utils", "ui/button", "ui/input"]
}
```

Local development server

`scripts/serve.mjs` starts a Node `http` server that serves the built `registry/` folder as static files (default port 3000, overridable with the `PORT` environment variable). It:

- answers `OPTIONS` with `204` and permissive CORS headers (`Access-Control-Allow-Origin: *`, methods `GET, OPTIONS`);
- resolves the URL path against the registry root and rejects any path that escapes it, preventing traversal;
- maps directories and trailing slashes to `index.json`;
- returns `404` with `{"error":"Not found"}` for unknown paths;
- sets a content type from a small extension map (JSON, HBS, PNG, JPG, GIF, SVG) and otherwise `application/octet-stream`.

It refuses to start when `registry/index.json` is missing and instructs the operator to run `pnpm build:registry` first.

16.2 Programmatic interfaces

`@expo-snips/core` (MIT, published)

Table 16.4: Public exports of the core package

Export	Purpose
`renderTemplate(templateStr, context)`	Render one template to a string.
`getFilesForRecipe(recipe, context, resolveFile?)`	Render a recipe into `{path, content}[]`.
`GenerationContext`, `GenerationOptions`, `RenderContext`	Input types for rendering.
`ScreenRecipe`, `FileTemplate`, `ScreenVariant`, `Conditions`	Data model types.
`StylingOption`, `RouterOption`, `UIOption`, `IconOption`	Option unions.

Usage example:

```
import { getFilesForRecipe, GenerationContext, ScreenRecipe } from "@expo-snips/core";

const context: GenerationContext = {
  projectRoot: process.cwd(),
  options: { styling: "nativewind", router: "expo-router", ui: "native", icons: "
    lucide", typescript: true },
};

const files = getFilesForRecipe(recipe as ScreenRecipe, context);
// => [{ path: "app/login/index.tsx", content: "..." }, ...]
```

`@expo-snips/cli` (MIT, published)

The CLI package exposes no documented programmatic API. Its `main` points to `dist/index.js`, which executes the commander program as a side effect when imported, so it is intended for use as a binary rather than as a library.

`@expo-snips/registry` (private)

Table 16.5: Exports of the registry package

Export	Purpose
`loadRecipeFromFile(filePath)`	Parse a recipe and inline `templateFile`, variant and per-styling references.
`getAllRecipes()`	Recursively load and inline every recipe under `data/`.
`getRecipeById(id)`	Load and match a single recipe.
`searchRecipes(query)`	Case-insensitive match on id, name or tags.

Export	Purpose
`fetchRecipes(url)`	Fetch a remote registry document; accepts a bare array, a `{recipes}` bundle, or a single recipe object; throws on a non-OK response or an unrecognised body.

Its filesystem functions resolve paths relative to the package directory using `__dirname`, so they are only meaningful inside the repository or a distribution that includes `data/` and `templates/`.

`expo-snippets` (VS Code extension, proprietary)

Contributes one command and registers a second at runtime.

Table 16.6: Extension commands and settings contributions

Interface	Definition
Command	`expo-snips.addScreen` — title "Expo Snips: Add Screen...".
Command (runtime)	`expo-snips.generateFromCompletion(recipeId?)`.
Setting	`expo-snips.registryUrl` (string, default empty).
Setting	`expo-snips.defaultStyling` (enum, default `nativewind`).
Setting	`expo-snips.defaultRouter` (enum, default `expo-router`).
Setting	`expo-snips.defaultUi` (enum, default `native`).
Setting	`expo-snips.defaultIcons` (enum, default `lucide`).
Setting	`expo-snips.installDependencies` (boolean, default `true`).
Activation	`onCommand:expo-snips.addScreen` and `onLanguage:{typescript,typescriptreact, javascript,javascriptreact}`.

16.3 Failure and error semantics summary

Table 16.7: Error semantics across interfaces

Failure	Signal	Client behaviour
Registry unreachable	Fetch rejects or aborts	Fall back to the embedded catalog; CLI prints a warning when a base was configured.
Registry returns non-OK	`response.ok` false	Same fallback.
Registry returns empty array	Array length 0	Same fallback.

Failure	Signal	Client behaviour
Recipe missing	`getRecipeById` returns null	CLI: error and exit 1. Extension: error message.
Manifest unreadable	JSON parse throws	Treated as empty manifest.
Unknown primitive id	Recipe lookup returns null	Warning; primitive skipped.
Install failure	Child process error	CLI aborts with the process error; files are already written.

17 Authentication

17.1 Product authentication

Status: NOT APPLICABLE.

Expo Snips has no users, accounts, sessions or credentials of its own. It is a local developer tool. A repository-wide inspection found no authentication code, no token handling, no session storage, no OAuth client, no password logic and no identity provider configuration in the product itself.

Consequences that follow directly from this:

- The CLI requires no login and stores no credentials.
- The VS Code extension requires no login and stores no credentials.
- The registry is unauthenticated static hosting.
- `.expo-snips.json` contains no personal data.
- There is no MFA, no token lifecycle, no refresh flow and no session store to document. All of these are **NOT APPLICABLE**.

17.2 Authentication templates generated for user applications

The catalogue *does* contain authentication *screens*, which are generated into the developer's own application. Those screens are not authentication for Expo Snips; they are scaffolding for the developer's app.

The four auth recipes are:

Table 17.1: Authentication recipes

Recipe	Version	UI	Generated files
`auth/login`	0.3.0	native, shadcn	`app/login/index.tsx` (or `screens/LoginScreen.tsx` for react-navigation), `components/LoginForm.tsx`, `hooks/useLogin.ts|.js`; optional `glass` variant.
`auth/register`	0.2.0	native, shadcn	`app/register/index.tsx` (or `screens/RegisterScreen.tsx`), `components/RegisterForm.tsx`, `hooks/useRegister.ts|.js`.
`auth/forgot-password`	0.2.0	native, shadcn	`app/forgot-password/index.tsx`, `components/ForgotPasswordForm.tsx`.
`auth/otp-verification`	0.2.0	native, shadcn	`app/otp-verification/index.tsx`, `components/OtpInput.tsx`.

17.3 Validation model in generated screens

All form-based auth recipes use the same pattern, implemented in the templates:

- `react-hook-form` with the `Controller` component for controlled inputs;
- a `zod` schema and `zodResolver` for validation;
- a typed form data type derived with `z.infer<typeof schema>` in the TypeScript path, and an untyped equivalent in the JavaScript path;
- per-field error text rendered from `formState.errors`.

Example schema, reproduced from `packages/registry/templates/auth/login/components.login-form.hbs`:

```
const schema = z.object({
  email: z.string().email('Invalid email address'),
  password: z.string().min(6, 'Password must be at least 6 characters'),
});

type FormData = z.infer<typeof schema>;
```

The shadcn variant uses slightly different messages (`"Email is required"`, `"Enter a valid email"`, `"Password must be at least 6 characters"`).

17.4 Authentication state and flow

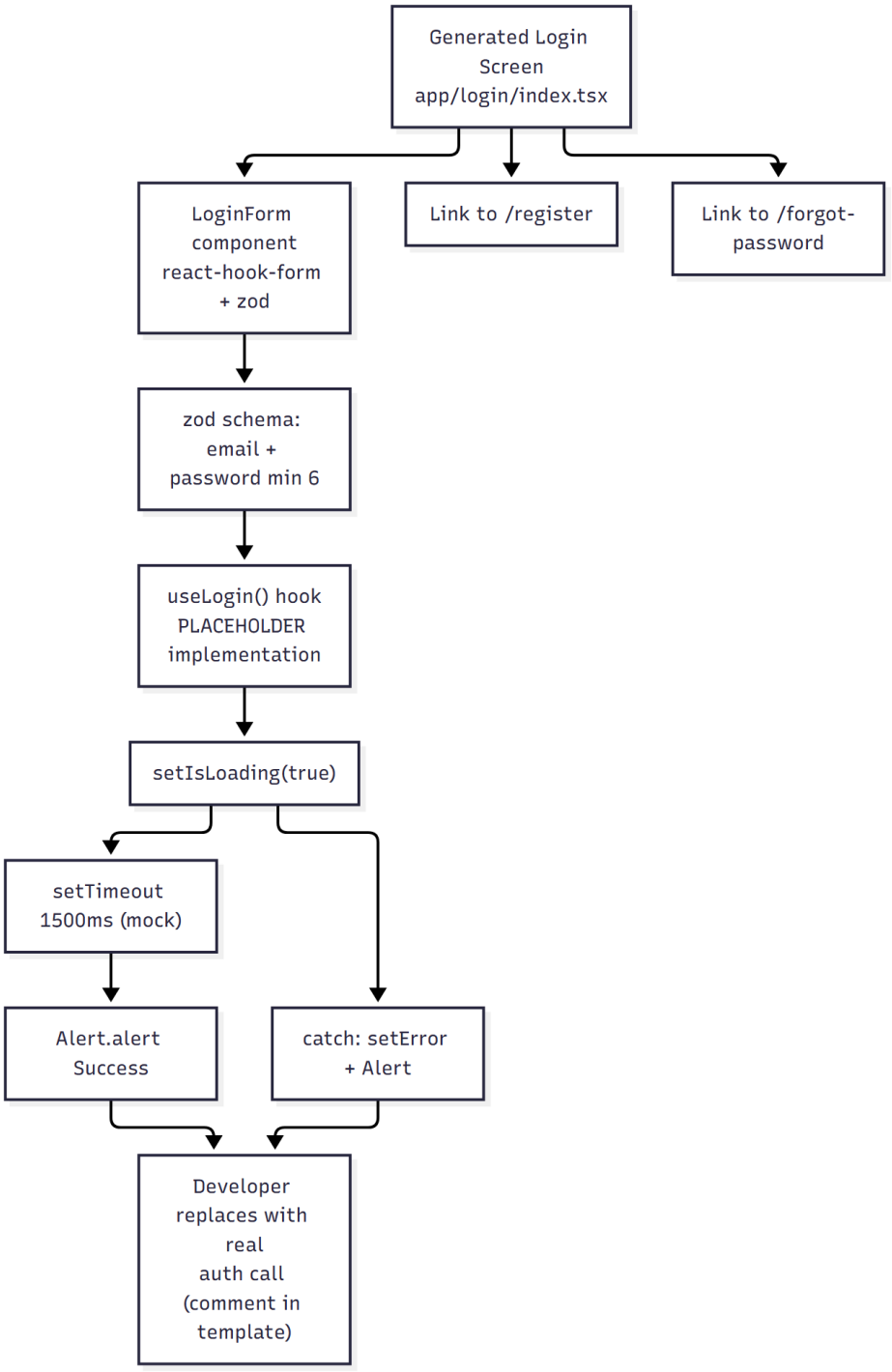


Figure 17.1: Authentication flow encoded in the generated templates (placeholder backend)

Critical characteristic: the authentication backend is a placeholder. `hooks/use-login.hbs` and `hooks/use-register.hbs` implement:

- `isLoading` and `error` state;
- a `console.log` of the submitted credentials;
- a mocked 1,500 ms delay (`new Promise((resolve) => setTimeout(resolve, 1500))`);

- an `Alert.alert('Success', 'Logged in successfully')` on completion;
- a `catch` that stores the error message and shows an alert.

The template carries an explicit instruction to the developer:

```
// Replace with your actual authentication logic
// Example: await supabase.auth.signInWithPassword({ email, password });
```

The shadcn variants differ materially. The shadcn login form's `onSubmit` simply runs `console.log(data)` — it does not call the `useLogin` hook at all (`templates/auth/login/components.login-form.shadcn.hbs`). A developer choosing `-ui shadcn` therefore receives a form with validation but no loading or error handling from the hook. This inconsistency is recorded in Chapter 34.

17.5 Password handling

- The native login form implements a secure-entry toggle: a `showPassword` state selects between `secureTextEntry` and an obscured/plain rendering, with `Eye`/`EyeOff` icons from `lucide-react-native`.
- The register recipe validates a password and a confirmation password with a `min(6)` rule.
- No password is ever persisted, hashed or transmitted by the product.
- The mocked hook logs the credentials to the console for demonstration; a developer should remove that `console.log` before shipping (see Chapter 23).

Status: IMPLEMENTED (templates) / RECOMMENDED (removing the console log).

17.6 Session, token and MFA handling

Status: NOT APPLICABLE.

No session storage, token issuance, refresh logic, secure storage, biometric flow, or multi-factor authentication is present in the product, and none is generated by default. The OTP recipe provides a one-time-code *input experience* (`components/OtpInput.tsx` with auto-advance behaviour) but contains no code verification backend; the developer must supply that. The `auth/otp-verification` recipe depends only on `lucide-react-native`.

17.7 Recommendations for users of the generated auth screens

- Replace the mock `useLogin`/`useRegister` bodies with a real authentication call (the templates suggest Supabase as an example only).
- Remove the credential `console.log`.
- Add secure storage (for example `expo-secure-store`) for any session token — the templates do not include one.
- Wire navigation to the real post-login route; templates link to `/register` and `/forgot-password` by convention and do not perform authentication redirects.

Status: RECOMMENDED.

18 Authorization

18.1 Product authorization

Status: NOT APPLICABLE.

Expo Snips implements no authorization model. There are no roles, permissions, policies, access-control lists, middleware guards or protected resources in the product. Relevant evidence:

- The CLI has no notion of a user or a role; all subcommands are available to anyone who can execute the binary.
- The extension has no `when` activation clauses beyond command and language activation, and no user-dependent behaviour.
- The registry is public, read-only static hosting; every recipe and every template is readable by anyone who can reach the host.
- There is no role/permission data model, no RLS (the product has no database) and no policy engine. All **NOT APPLICABLE**.

18.2 Effective privilege boundaries

Although there is no authorization *model*, the system does have privilege boundaries that matter operationally:

Table 18.1: Effective privilege boundaries

Boundary	What is possible	What is not
Local file writes	Generate files at any path declared by a recipe, relative to the project root.	Read arbitrary files (the tool only reads `package.json`, `tsconfig.json`, the manifest and files it is about to overwrite).
Child process execution	Run the detected package manager; run `npx create-expo-app` during `init`.	Run arbitrary commands from recipe data — recipe `dependencies` strings are inserted into the install command line.
Network access	Fetch JSON from the configured registry base.	Upload data; transmit credentials; authenticate.
Registry content	Any published recipe is trusted and rendered verbatim.	Verify provenance or signature.

18.3 A UI access consideration worth recording

The catalog contains three recipes that are *not* end-user screens but shared UI primitives (`ui/button`, `ui/input`, `ui/utils`). They are listed by `expo-snips list` and are selectable

from the CLI picker and the VS Code Quick Pick exactly like application screens. Nothing in the CLI prevents a user from running:

```
expo-snips add ui/button
```

which writes `components/ui/button.tsx` (and, because `ui/button` declares `primitives: ["ui/utils"]`, also `lib/utils.ts`) directly into the project, with `styling` forced to `nativewind` and `ui` forced to `shadcn` by the primitive-resolution logic.

Assessment (INFERRED). This is not a security problem, but it is a usability and presentation concern: internal primitives are indistinguishable from application screens in every discovery surface. A future revision could filter `ui/*` ids from the user-facing pickers or mark them as internal.

18.4 Recommendation

If a hosted, curated registry is ever offered, authorization will become relevant in at least two places:

1. **Write access** to publish recipes — currently anyone with write access to the repository (and therefore the CI token) can publish, and there is no review gate beyond pull-request review.
2. **Read access** to private/company catalogs — the configurable registry has no support for authentication headers or tokens, so a private registry must currently be reachable without credentials (for example, on a trusted network or behind a proxy).

Status: RECOMMENDED. Neither behaviour exists today.

19 User Flows

19.1 CLI journey

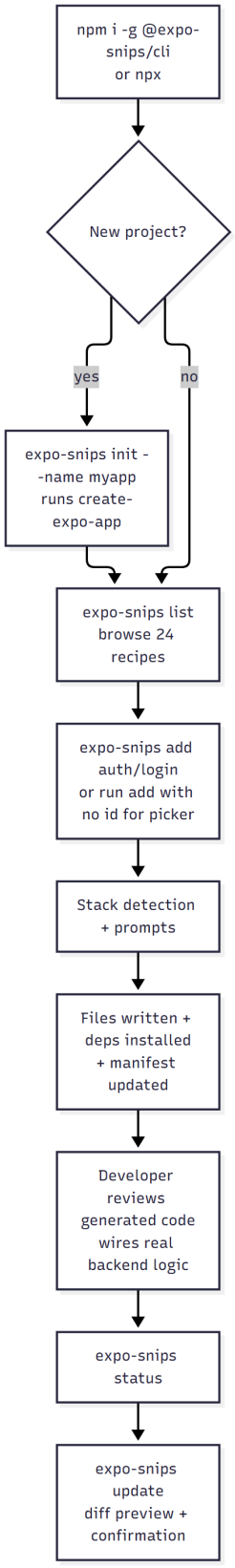


Figure 19.1: CLI journey from installation to keeping screens up to date

Table 19.1: CLI journey steps

#	Step	Detail
1	Install	`npm i -g @expo-snips/cli`, or invoke through `npx`.
2	(Optional) bootstrap	`expo-snips init -name myapp` runs `create-expo-app`.
3	Browse	`expo-snips list` prints 24 recipes with tags.
4	Generate	`expo-snips add auth/login` (or bare `add` for a picker).
5	Detect and prompt	The CLI reports detected values and prompts only for unresolved options.
6	Write and install	Files are created; recipe dependencies are installed.
7	Record	`.expo-snips.json` is updated.
8	Review	The developer edits the generated code and wires real logic.
9	Maintain	`status` to inspect, `update` to pull catalog improvements with a diff preview.

19.2 VS Code journey

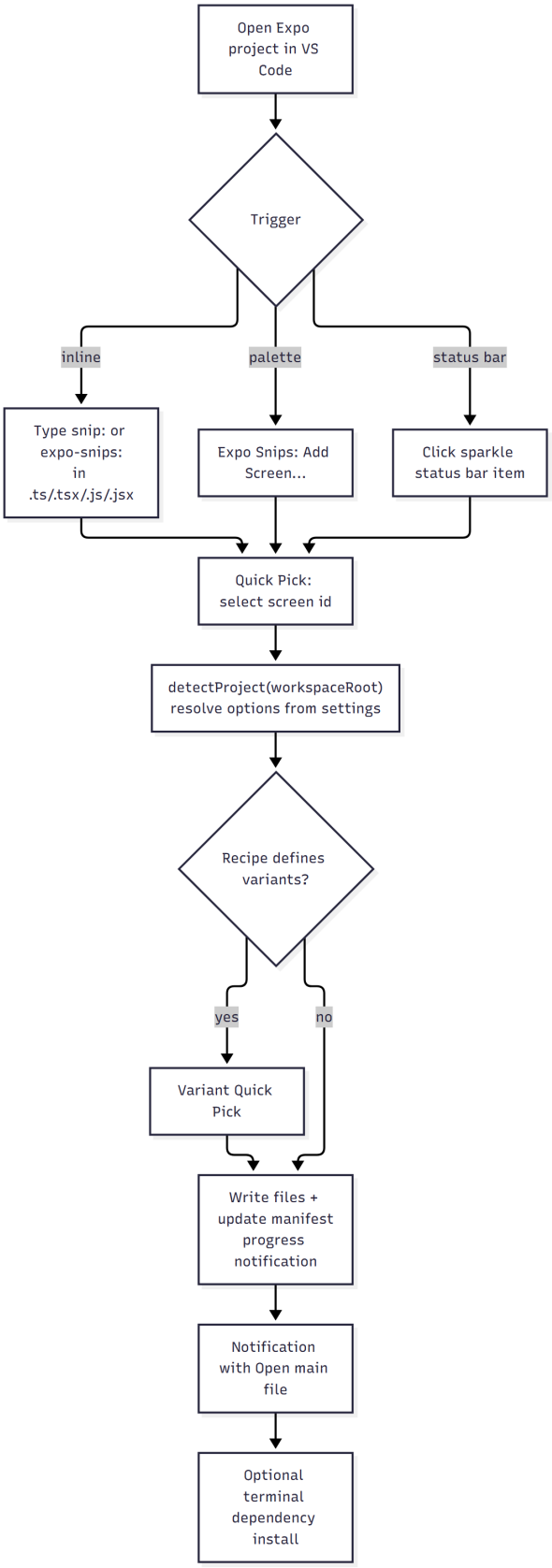


Figure 19.2: VS Code journey using the inline completion trigger

Table 19.2: VS Code journey steps

#	Step	Detail
1	Open a project	The extension activates on a TS/JS language or the command.
2	Trigger	Type `snip:` or `expo-snips:`; or run "Expo Snips: Add Screen..."; or click the status bar item.
3	Select	A Quick Pick lists recipe names with ids and descriptions.
4	Resolve	Options are computed silently from detection and settings; a variant Quick Pick appears only when the recipe declares variants.
5	Generate	Files are written under a progress notification.
6	Complete	An information message reports the file count and offers "Open main file".
7	Install	Dependencies are installed in an "Expo Snips" terminal when the setting is enabled.

19.3 Variant selection flow

1. The CLI accepts `-variant glass`. For a recipe without a glass variant, the flag is accepted and the default template is used.
2. In VS Code, `pickVariant` returns undefined when `recipe.variants` is empty; otherwise it shows a Quick Pick of variant labels and returns the selected id.
3. During rendering, `resolveTemplateString` prefers the variant template over the default for that file entry.

Only `app/home`, `auth/login` and `onboarding/welcome` define variants, so the variant Quick Pick appears only for those three recipes.

19.4 Update flow

The update flow is documented as a state machine in Chapter 20 (Figure 20.4).

19.5 Error and recovery flows

Table 19.3: Recovery flows

Situation	What the user sees	Recovery
Typo in screen id	"Screen not found: <id>" and a pointer to `list`.	Run `list` and retry.
Unsupported option	"Recipe "..." does not support <option> "..."."	Omit the flag; the tool will detect or prompt.

Situation	What the user sees	Recovery
No files matched	"No files matched the selected options. Nothing generated."	Choose an option combination the recipe supports (check `list` output or the recipe document).
Registry unreachable	CLI warning: "Could not reach registry at <base>. Using N built-in screens."	Remove or correct the registry base; generation continues from the embedded catalog.
Option cancelled mid-prompt	Silent exit with code 1.	Re-run and complete the prompts.
Update declined	"Update cancelled."	Re-run when ready; nothing was written.
No manifest	"No screens tracked in this project yet."	Run `add` first.

20 Data Flows

20.1 Principal data flow: generation

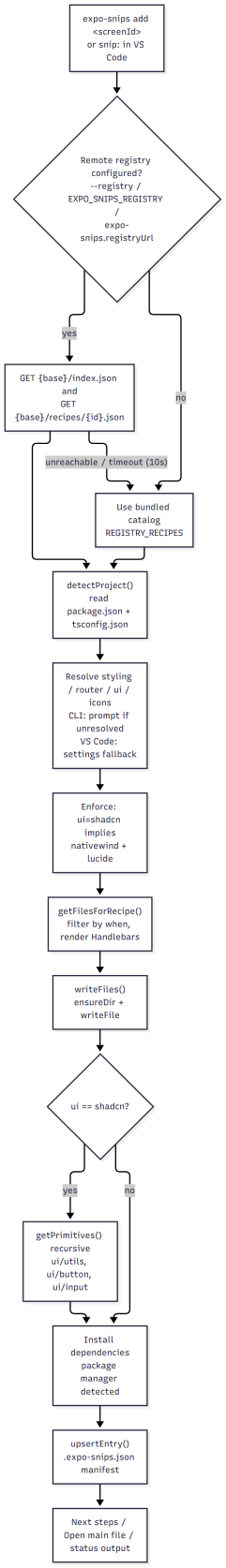


Figure 20.1: Generation data flow: from command input to persisted manifest

Data transformations

Table 20.1: Data transformations in the generation flow

Input	Transformation	Output
Registry base string	`trim()` then strip trailing `/`	Normalised base
Index JSON array	Map to `RecipeIndexEntry[]`	Browsable index
Recipe JSON	Object with `files[].template` strings	`ScreenRecipe`
`package.json` + `tsconfig.json`	Four dependency probes + one file probe	`DetectedProject`
Flags + detection + recipe `supported`	Precedence rules, prompts, shadcn enforcement	`GenerationOptions`
`GenerationOptions` + variant	`buildRenderContext`	`RenderContext`
Recipe + context	`when` filter, template selection, Handlebars compile and execute	`{path, content}[]`
File list + project root	`ensureDir` + `writeFile`	Files on disk
Recipe metadata	`upsertEntry`	Manifest entry

20.2 Registry resolution flow

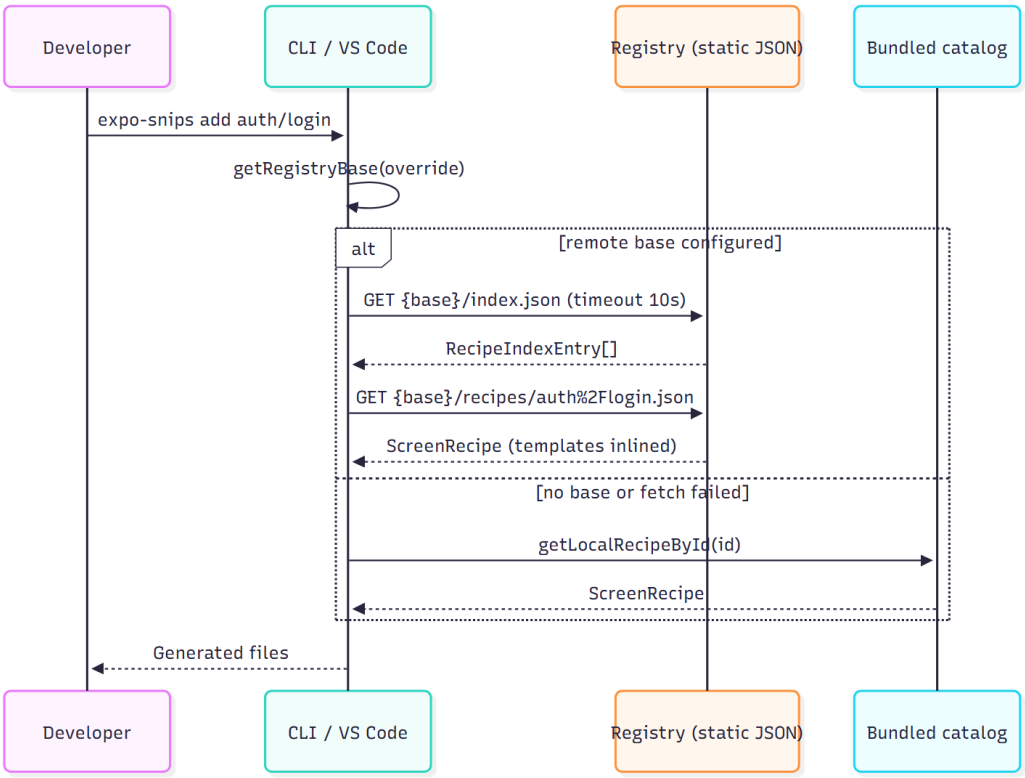


Figure 20.2: Sequence for resolving a recipe, including fallback

20.3 Primitive resolution flow

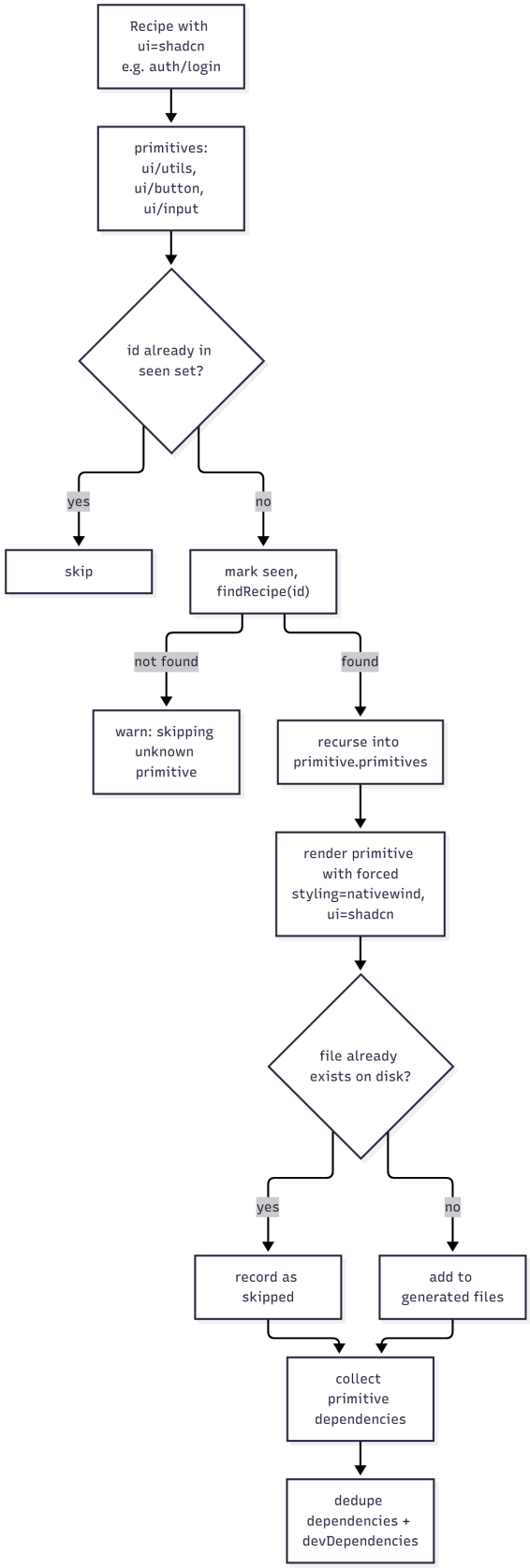


Figure 20.3: Recursive primitive resolution data flow

The recursion is the only graph traversal in the codebase. It is guarded by a `Set<string>` of seen ids, which makes the traversal terminate even if a primitive cycle were introduced. Files whose paths already exist are excluded from the generated set and reported as skipped.

20.4 Update flow

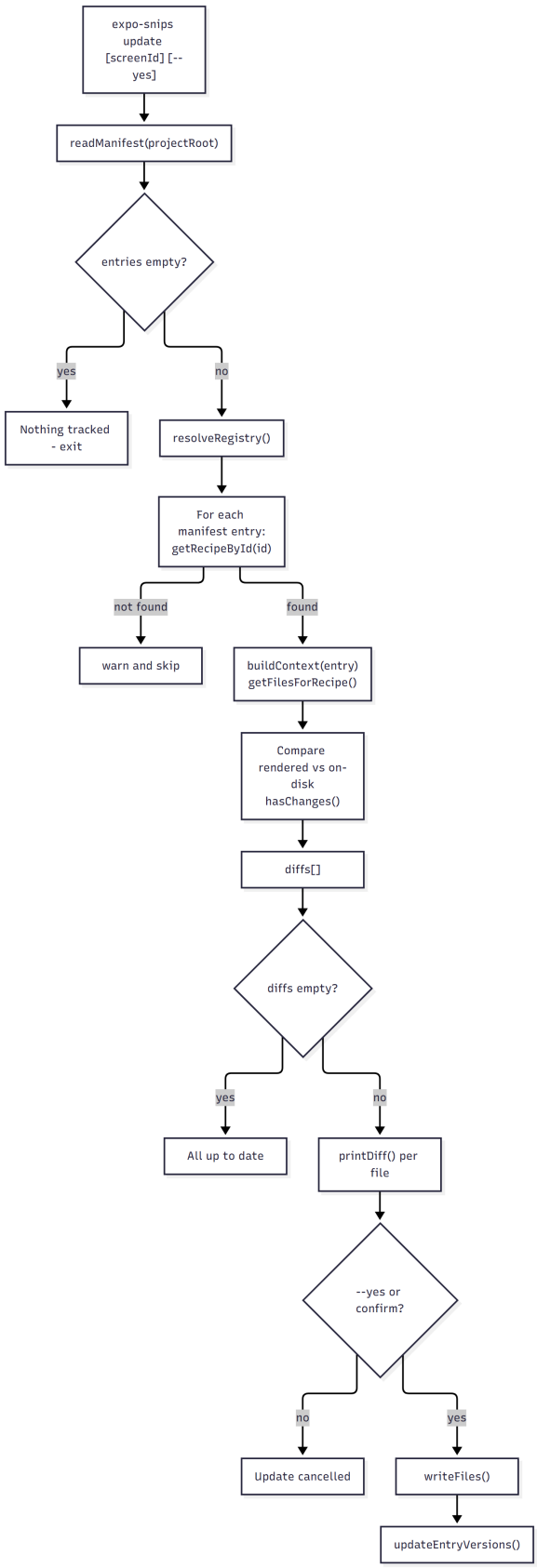


Figure 20.4: Update flow with diff preview and confirmation

Diff data structure

```
interface DiffHunk { type: "add" | "remove" | "context"; line: string; }
```

`diffLines` builds an $(n + 1) \times (m + 1)$ dynamic-programming table over the two line arrays (where n and m are the line counts) and walks it to emit hunks. The algorithm is the classic longest-common-subsequence diff; it has no line-number tracking, no hunk grouping and no context limit, so a large file change prints every line. Colours: green for additions, red for removals, grey for context.

20.5 Migration and data at rest

- **Migrations: NOT APPLICABLE**. There is no database and no schema migration system.
- **Data at rest (1):** `.expo-snips.json` in the user's project. Written with two-space indentation; unreadable content is treated as an empty manifest rather than an error.
- **Data at rest (2):** the generated screen files themselves.
- **Data at rest (3):** the repository's own built registry (`registry/`) and the offline bundles.
- **Data in transit:** HTTP GET of JSON documents. No data leaves the user's machine except the request itself; no telemetry exists.

20.6 Manifest lifecycle

Table 20.2: Manifest lifecycle events

Event	Operation	Effect
First generation	`upsertEntry`	File created with `version: 1` and one entry.
Regeneration of the same recipe and variant	`upsertEntry`	Entry replaced with fresh options, files, version and timestamp.
Generation of the same recipe with a different variant	`upsertEntry`	A second, distinct entry is added (the upsert key includes the variant).
Update applied	`updateEntryVersions`	Matching entries receive the new `recipeVersion`; `installedAt` and `options` are unchanged.
Manifest deleted	—	The tool treats the project as having no tracked screens.

20.7 Data volumes

Table 20.3: Typical data volumes (measured from the repository)

Artefact	Size
Catalog index	686 lines for 24 entries (mostly the `supported` objects).
Single built recipe	The `auth/login` recipe is the largest, with 7 file entries and inlined templates.
Full offline bundle	`packages/registry/bundle.json` is 242,775 bytes for 24 recipes.
Templates	48 `.hbs` files.
Generated output per recipe	1–7 files, each typically under 300 lines.

21 UI / UX

Two distinct experiences must be described: the experience of the *tool* (terminal and editor) and the experience of the *generated screens*.

21.1 Tool experience — terminal

Table 21.1: Terminal interaction design

Element	Design
Colour semantics	Cyan for informational headings (`Generating "Login Screen" (auth/login)...`), green for success (`✓Login Screen added successfully!`), yellow for warnings and notices, red for errors, dim grey for secondary detail (the effective options line, the registry source line, skipped files).
Progressive disclosure	Options are asked for only when they cannot be resolved.
Defaults	Prompts preselect the detected value when the recipe supports it.
Filtering	Prompts list only the values the recipe supports.
Confirmation	The effective option set is printed before generation: `styling=... router=... ui=... icons=... typescript=...`.
Per-file feedback	Each written file is announced as `Created <path>`.
Follow-up guidance	Three concrete next steps after generation.
Safe abort	Any handled error leaves the exit code at 1 and writes nothing further.

21.2 Tool experience — VS Code

Table 21.2: Editor interaction design

Element	Design
Discoverability	Three entry points (completion, command palette, status bar) so the tool is reachable without reading documentation.
Inline trigger	The trigger regex matches only immediately before the cursor (`snip:` or `expo-snips:`), so ordinary typing does not spam the completion list.
Completion items	Labelled with the recipe id (the thing the user must type), detailed with the display name, documented with the description and tags, sorted by id.

Element	Design
Non-destructive insert	The completion inserts no text; selecting it runs the generation command instead. This avoids leaving stray trigger text in the file.
Selection	Quick Pick matches on label and description (`matchOnDescription: true`).
Progress	A progress notification covers the write phase.
Completion feedback	An information message reports the recipe name and file count, and the message detail lists every generated path.
Follow-up action	A single "Open main file" button keeps the next step one click away.
Empty catalog	A dedicated error message with an "Open Settings" action that deep-links to the `expo-snips.registryUrl` setting.

21.3 Generated screen experience

The generated screens share a consistent interaction vocabulary. The following is observed across the templates.

States

Table 21.3: States provided by generated screens

State	Where and how
Loading	`isLoading` in `useLogin`/`useRegister`; a `ActivityIndicator` is rendered in the native form.
Validation error	Per-field error text under the input, from `formState.errors`.
Submission error	`error` state returned by the auth hooks plus `Alert.alert`.
Empty	`utility/empty` — icon, message and a primary action.
Error / retry	`utility/error` — warning icon, message and a retry action.
Refresh	`app/home` wires a `RefreshControl` (its handler logs `'refresh'`).
Unread indicator	`app/notifications` distinguishes read/unread and offers a mark-all-read action.
Selected / unselected	`onboarding/features` cards and `ecommerce` product cards.

Input and validation patterns

- Controlled inputs through `react-hook-form`'s `Controller`.
- Email fields set `keyboardType="email-address"` and `autoCapitalize="none"`.
- Password fields combine a secure entry toggle with `secureTextEntry`.
- The OTP recipe uses a segmented input with auto-advance and auto-submit on completion and an error message plus a resend affordance.
- Labels are rendered above the field in the native variant; the shadcn variant instead pairs an icon with a borderless input inside a row.

Navigation patterns

- Expo Router screens use `Link` with route strings (`/register`, `/forgot-password`, `/`).
- The react-navigation login and register screens use `useNavigation()` and `navigation.goBack()`.
- Back affordances are a left chevron or an arrow-left icon inside a pressable, consistent with the surrounding palette.

21.4 Responsive design

All layouts use flex containers and percentage/flex sizing rather than fixed widths, so they adapt to phone and tablet. Concrete evidence:

- `ecommerce/products` uses a two-column `FlatList` grid.
- `app/home` uses `flex-1` stat cards in a row.
- Forms are wrapped in a `ScrollView` with `keyboardShouldPersistTaps="handled"` and a `KeyboardAvoidingView` whose behaviour switches on platform (`padding` on iOS, `height` on Android).
- Lists use `react-native-safe-area-context`'s `SafeAreaView` in some templates and React Native's own `SafeAreaView` in others — an inconsistency recorded in Chapter 35.

21.5 Animations and interaction feedback

- The use of `Pressable` provides the platform's native press feedback.
- The shadcn-style `Button` adds opacity changes on press (`active:opacity-90`) and web-only hover/focus ring behaviour.
- No template uses `Animated`, `Reanimated`, `Moti` or layout transitions.

Status: IMPLEMENTED (as an observed absence of animation).

21.6 Accessibility

Finding. No generated template sets `accessibilityLabel`, `accessibilityRole`, `accessibilityHint` or `accessibilityState`. Interactive elements are `Pressables` and `TextInputs` without labels for assistive technology. Icon-only buttons (for example the header bell, search and back buttons) expose no text alternative.

Table 21.4: Accessibility finding

ID	Finding	Severity	Evidence
A11Y-01	Icon-only pressables have no accessibility label.	Medium	Every template with an icon button, e.g. `app/home` header actions.
A11Y-02	No accessibility roles or hints are declared anywhere.	Medium	Repo-wide absence across 48 templates.
A11Y-03	Form error text is not associated with its input for assistive technology.	Medium	`components.login-form.hbs` renders errors as sibling `Text`.

ID	Finding	Severity	Evidence
A11Y-04	Colour contrast of the glass variant (`#0F172A` surface with white text) is good; the muted greys (`#9CA3AF`) on white are below WCAG AA for body text.	Low	`#9CA3AF` used for timestamps and placeholders.

Status: RECOMMENDED (add accessibility props and confirm contrast ratios).

21.7 Dark mode

Only the `glass` variant is effectively dark (`#0F172A`); there is no theme switch, no `useColorScheme` integration and no light/dark token set. Screens are light-themed otherwise.

Status: IMPLEMENTED (light only) / RECOMMENDED (theming support).

22 Design System

22.1 Scope

Expo Snips ships design decisions in the form of generated screens rather than as a documented design system. This chapter extracts the system that is actually encoded in the 48 Handlebars templates. Every value below is a literal found in `packages/registry/templates/**/*.hbs`.

Status: IMPLEMENTED (values) / INFERRED (organisation into a system).

22.2 Colour palette

The generated screens use a Tailwind-derived, light, neutral palette with a blue accent. Counts are the number of occurrences across all templates.

Table 22.1: Colour palette derived from template literals

Hex	Tailwind	Count	Role	Usage
`#2563EB`	blue-600	40	Primary	Accent links, active states, primary buttons, check marks, key icons.
`#FFFFFF`	white	74	Surface	Cards, inputs, icon buttons, sheets.
`#F9FAFB`	gray-50	10	Canvas	App background behind cards.
`#F3F4F6`	gray-100	16	Divider	List separators, chart placeholders.
`#E5E7EB`	gray-200	2	Border	Secondary borders.
`#D1D5DB`	gray-300	7	Border	Input and checkbox borders.
`#9CA3AF`	gray-400	28	Muted text	Timestamps, placeholders, chevrons.
`#6B7280`	gray-500	45	Secondary text	Body copy, labels, secondary icons.
`#4B5563`	gray-600	19	Icon	Default icon colour.
`#374151`	gray-700	18	Strong body	Article body, menu labels.
`#111827`	gray-900	65	Primary text	Headings and titles.
`#EFF6FF`	blue-50	5	Tint	Selected rows, light blue fills.
`#DBEAFE`	blue-100	3	Tint	Avatar placeholder background.
`#BFDBFE`	blue-200	1	Tint	Product placeholder block.
`#93C5FD`	blue-300	3	Accent	Decorative accents.
`#60A5FA`	blue-400	7	Accent	Chart bars, highlights.
`#3B82F6`	blue-500	1	Accent	Scheduled event colour.

Hex	Tailwind	Count	Role	Usage
#0F172A	slate-900	3	Dark surface	Glass-variant background only.
#10B981	emerald-500	4	Success	Positive trend indicators.
#16A34A	green-600	1	Success	Success status.
#15803D	green-700	1	Success text	Success copy.
#D1FAE5	emerald-100	1	Success fill	Success background.
#DCFCE7	green-100	1	Success fill	Success background.
#F0FDF4	green-50	1	Success fill	Success background.
#F59E0B	amber-500	9	Rating / warning	Star ratings.
#FEF3C7	amber-100	1	Warning fill	Warning background.
#EF4444	red-500	21	Error	Error text, destructive icons, negative trends.
#DC2626	red-600	1	Error text	Destructive action label.
#FEE2E2	red-100	1	Error fill	Error background.
#FEF2F2	red-50	1	Error fill	Error surface.
#8B5CF6	violet-500	2	Data accent	Secondary chart series and event colour.
#E0E7FF	indigo-100	1	Data accent	Placeholder block.
#FCE7F3	pink-100	1	Data accent	Placeholder block.

Semantic mapping (recommended)

For a future brand or product site, the following mapping is consistent with the templates' own usage. **Status:** RECOMMENDED.

Table 22.2: Recommended semantic colour mapping

Role	Value	Tokens observed in templates
Primary / accent	`#2563EB`	`bg-primary`, `text-primary`
Primary on accent	`#FFFFFF`	`text-primary-foreground`
Canvas	`#F9FAFB`	`bg-gray-50`
Surface	`#FFFFFF`	`bg-white`, `bg-background`
Text primary	`#111827`	`text-gray-900`, `text-foreground`
Text secondary	`#6B7280`	`text-gray-500`, `text-muted-foreground`
Border	`#D1D5DB`	`border-input`, `border-gray-300`
Destructive	`#EF4444`	`bg-destructive`, `text-destructive`
Success	`#10B981`	trend indicators
Warning	`#F59E0B`	rating stars

22.3 shadcn-style design tokens

When `ui=shadcn` is selected, templates switch from literal colour values to semantic utility tokens. The tokens actually referenced in templates are:

Table 22.3: Tokens referenced by shadcn-style templates

Token	Where used
`bg-background`, `text-foreground`	Screen containers and titles in `app.*.index.shadcn.hbs`
`text-muted-foreground`	Subtitles and secondary text
`text-destructive`	Validation error messages
`bg-primary`, `text-primary-foreground`	Default button variant
`bg-primary/10`	Selected OTP cell fill
`border-input`	Input and outline-button borders
`bg-secondary`, `text-secondary-foreground`	Secondary button variant
`bg-accent`, `text-accent-foreground`	Outline/ghost hover states
`ring-ring`	Web focus ring on buttons
`rounded-md` (inputs, buttons)	Control radius

Important gap. No Tailwind or NativeWind theme configuration is present in the repository (`no tailwind.config.*` or NativeWind theme file exists). The tokens above require such a configuration to resolve to concrete colours. This is recorded as a known issue and a high-priority technical-debt item (Chapters 34 and 35).

22.4 Typography

- **Font family:** none is specified. All templates inherit the React Native platform default (system font). No `fontFamily` appears in any template.
- **Weights:** `font-bold` for titles and statistic values, `font-medium` for labels and button text, regular for body copy.

Table 22.4: Type scale (Frontend)

Style	Class	px	Use
Page title	`text-3xl`	30	Auth and onboarding headings (shadcn variant)
Page title	`text-2xl`	24	Home greeting, header titles
Section title	`text-lg`	18	Section headings, card titles
Body	`text-base`	16	Form values, list titles
Body small	`text-sm`	14	Labels, secondary copy, buttons
Meta	`text-xs`	12	Timestamps, deltas, captions

StyleSheet variants encode the same scale numerically (`fontSize: 12, 14, 16, 18, 24`) with the same weight conventions.

22.5 Spacing

Spacing is a multiple-of-four scale, expressed either as utility classes or as numeric StyleSheet values.

Table 22.5: Spacing scale observed in templates

Value (px)	Class	Typical use
4	`gap-1`, `mt-1`	Label-to-error spacing
8	`gap-2`, `mb-2`	Icon-to-text gaps
12	`gap-3`, `ml-3`	List item internal spacing
16	`gap-4`, `p-4`	Card padding, form field gaps
20	`px-5`	Screen horizontal padding
24	`p-6`, `mt-6`	Section separation, screen padding
32	`mt-8`	Major section separation

Screen padding is consistently 20 px horizontally for list/dashboard screens and 24 px for auth and onboarding screens.

22.6 Border radius

Table 22.6: Radius scale

Class	px	Use
`rounded-2xl`	16	Cards, stat tiles, task rows
`rounded-xl`	12	Secondary containers
`rounded-lg`	8	Inputs and buttons (native variant)
`rounded-md`	6	Inputs and buttons (shadcn variant)
`rounded-full`	—	Icon buttons (40×40 with radius 20), avatars, checkbox dots

22.7 Elevation, borders and shadows

- **No shadow or elevation properties appear in any template.** Depth is conveyed by white surfaces on a grey canvas rather than by shadows.
- Borders are used sparingly: `borderWidth: 2` for checkboxes and OTP cells, single-pixel top/bottom borders for list dividers (`#F3F4F6`).
- The glass variant substitutes translucency (`rgba(255,255,255,0.08)`) for elevation.

Status: IMPLEMENTED (absence of shadows is verifiable).

22.8 Component inventory

Table 22.7: Design-system components demonstrated by recipes

Component	Characteristic styling
Card	White, 16 px radius, 16 px padding.
Stat tile	Card with a small icon-plus-label row, a 24 px bold value and a 12 px delta line.
Button (native)	Full-width pressable with an accent fill and white label.
Button (shadcn)	`cva`-driven variants: default, destructive, outline, secondary, ghost, link; sizes default/sm/lg/icon; forwardRef wrapper around `Pressable` with automatic `Text` child wrapping.
Input (native)	Label above field, bordered row, optional leading icon, error text below.
Input (shadcn)	`TextInput` with tokenised classes, placeholder colour `#9ca3af`, native height override.
Icon button	40×40 circle, white, centred icon (20–24 px).
OTP input	Segmented cells, selected cell tinted, auto-advance, auto-submit.
List row	Icon or avatar, title, subtitle, chevron or badge.
Search bar	Rounded input with a leading search icon.
Toggle row	Label plus a `Switch`, grouped in a white block with dividers.

Component	Characteristic styling
Chip	Recent-search pill; category filter.
Rating	Amber star row with a numeric value.
Avatar	Circle with initials or a tinted background.
Bottom sheet	Location detail sheet over a map.
Empty state	Centred muted icon, title, message and a primary action.

22.9 Icons

- **Library:** `lucide-react-native`, declared as a dependency by almost every recipe.
- **Sizes:** 12–24 px, with 16 and 20 most common; colour is set explicitly, usually `#4B5563` for inactive icons, `#9CA3AF` for chevrons and `#2563EB` for active state.
- **Degradation:** template conditionals remove icon imports and icon elements entirely when `icons` is `none`, leaving text-only layouts.

22.10 Breakpoints and responsive units

There is no breakpoint system. Layouts are flex-based and therefore inherently fluid. The only explicit responsiveness mechanism is NativeWind's `web:` prefix, used in the shadcn primitives for web-only hover, focus and sizing behaviour (for example `web:focus-visible:ring-2` and `native:h-12`).

22.11 Animation

No template uses an animation library, transition or layout animation. Press feedback is provided by the platform's `Pressable`. The button primitive adds opacity transitions on press. This is the extent of the motion design.

22.12 Design-system gap summary

Table 22.8: Design-system gaps

Gap	Detail
No theme configuration	shadcn tokens are referenced but no Tailwind/NativeWind theme defines them.
No atomic primitives for the native path	Native-variant screens hand-roll inputs and buttons instead of drawing on shared primitives.
No accessibility props	See Section 21.
No dark mode	Only the glass variant is dark, and it is a one-off template.
No icon abstraction	lucide icons are imported directly per template.
Spacing tokens not centralised	Values are repeated literally, not defined once.

Status: IMPLEMENTED (gaps are verifiable) / RECOMMENDED (centralising the system in a theme file).

23 Security

23.1 Method and scope

The review covers the repository's own source, templates, build scripts and CI configuration. Findings are stated in a fixed format: finding, severity, evidence, impact and recommendation. Severity reflects impact in the product's real deployment model — a locally executed developer tool that writes files into the user's project and may fetch content from a configurable registry.

23.2 Secrets

Table 23.1: Secret-related findings

ID	Finding	Severity	Evidence	Status
SEC-01	No secrets are committed. A recursive search for `.env` files found none, and no credential literals appear in source, templates or workflows.	—	Absence verified across the tree.	PASS
SEC-02	CI uses repository secrets for publishing (`NPM_TOKEN`) and the workflow default token (`GITHUB_TOKEN`), both referenced indirectly rather than inlined.	—	`publish.yml`, `deploy-registry.yml`.	PASS
SEC-03	The CLI has no mechanism to store credentials; it therefore cannot leak them.	—	`registry.ts` performs unauthenticated GET only.	PASS

23.3 Injection

Table 23.2: Injection findings

ID	Finding	Severity	Evidence	Status
SEC-04	**Shell command construction from unvalidated user input** in `init`. The project name is interpolated directly into a shell string that is executed with inherited stdio. A name containing shell metacharacters or quote breakout would be interpreted by the shell.	High	`packages/cli/src/commands/init.ts`, lines 20–23.	OPEN
SEC-05	**Dependency names are interpolated into an executed shell command.** Recipe `dependencies` strings are joined and passed to `execSync`. With a bundled catalog this is trusted data; with a remote registry, recipe content becomes an execution vector (see SEC-08).	High	`packages/cli/src/utils/installDeps.ts`, lines 25 and 35.	OPEN
SEC-06	No SQL, NoSQL or ORM access exists, so SQL injection is **NOT APPLICABLE**.	—	No database in the project.	N/A

ID	Finding	Severity	Evidence	Status
SEC-07	Template rendering is not a code-execution path: Handlebars is used in default (non-helper-registering) mode with only the `eq` and `neq` helpers added, and generated output is written to files rather than evaluated.	Low	`packages/core/src/generate.ts`, lines 5–6.	ACCEPTED

SEC-04 / SEC-05 — details and recommendation

```
// packages/cli/src/commands/init.ts (excerpt)
const nameArg = name ? `${name}` : ".";
const cmd = `npx create-expo-app@latest ${nameArg} --template ${template} --yes`;
execSync(cmd, { cwd: process.cwd(), stdio: "inherit" });
```

Both `name` and `template` come from argv. Neither is validated or quoted.

Impact: a crafted project name (for example one containing `;`, `&&` or a backtick) executes arbitrary commands with the user's privileges. The risk is partly self-inflicted (the user types their own name), but the command may also be supplied by a script, CI job or documentation snippet.

Recommendation: use `execFileSync` (or `spawnSync`) with an argument array so no shell is involved, and validate the project name against a conservative pattern (`^[A-Za-z0-9._-]+$`). Apply the same treatment to the dependency installer, and validate dependency strings against npm package-name grammar.

23.4 Trust boundary: the registry

Table 23.3: Remote-content findings

ID	Finding	Severity	Evidence	Status
SEC-08	**Remote recipes are rendered and written verbatim, with no verification.** A configured registry can return any `files[].path` and any template content; the client writes those paths into the user's project and installs declared dependencies.	High	`getRecipeById` (both clients) plus `writeFiles`, `installDeps`.	OPEN
SEC-09	Path traversal is not validated on generated paths. A recipe path containing `..` would resolve outside the project root because paths are joined without containment checks.	High	`writeFiles.ts` line 14; `generator.ts` line 215.	OPEN
SEC-10	HTTPS is not enforced. A registry base of `http://` is accepted without warning.	Medium	`getRegistryBase` accepts any string.	OPEN
SEC-11	There is no integrity check (hash or signature) on fetched recipe documents.	Medium	`fetchJSON` returns parsed JSON directly.	OPEN

Overall assessment. Because the default configuration uses the embedded catalog, the system is safe out of the box. The moment a remote registry is configured — which the product actively documents and supports — the registry becomes a trusted code-distribution channel with none of the controls a package manager applies (no signature, no hash, no review, no path containment). This is the single most important security consideration in the product.

Recommendation (RECOMMENDED):

- reject or normalise any generated path that escapes the project root;
- validate dependency strings against the npm name grammar;
- warn or refuse when a non-HTTPS registry is configured;
- consider publishing a manifest of recipes with content hashes and verifying it client-side;

- document the trust implications of `-registry` prominently.

23.5 Generated-code findings

Table 23.4: Findings in generated screens

ID	Finding	Severity	Evidence	Status
SEC-12	**Credentials are logged to the console.** The generated login and register hooks execute `console.log('Login attempt', { email, password })`, and the shadcn form logs the whole form payload.	Medium	`hooks.use-login.hbs`; `components.login-form.shadcn.hbs`.	OPEN
SEC-13	**Authentication is a placeholder.** Submission always succeeds after a fixed delay and shows a success alert. If a developer ships without replacing it, the app reports successful login for any credentials.	High	`hooks.use-login.hbs`, `hooks.use-register.hbs`.	OPEN (documented)
SEC-14	No secure storage is generated for any session token; the templates perform no session persistence. This is a gap rather than a vulnerability, but it means a developer following the scaffolding must add secure storage themselves.	Low	Absence across auth templates.	INFO

ID	Finding	Severity	Evidence	Status
SEC-15	Sample data is embedded (statistics, product lists, notification lists, placeholder names). No real data or personal data is present.	—	`packages/registry/templates/**`.	PASS

Recommendation: remove the credential logging from the auth hook templates (or gate it behind a development-only check), and state in the generated file's leading comment and in user-facing documentation that the hook must be replaced before the screen is functional.

23.6 Input validation

Table 23.5: Input validation posture

Input	Validation
CLI option values (styling, router, ui, icons)	Checked against the recipe's `supported` arrays; rejected with an error.
CLI screen id	Looked up in the catalog; rejected when absent.
CLI variant id	*Not* validated. An unknown variant is silently ignored and the default template is used.
CLI project name and template (`init`)	None. See SEC-04.
Registry URL	Trimmed and de-slash-normalised; otherwise unvalidated.
Remote response shape	Partially validated: index must be a non-empty array; recipe must be an object with a truthy `id`. No field-level validation.
Recipe field values	Validated only by the build-time validator, not by clients.
Generated file paths	Not validated for containment. See SEC-09.
Recipe dependency strings	Not validated. See SEC-05.

23.7 Cross-site, CORS and web concerns

NOT APPLICABLE for the product. There is no browser application, no HTML rendering of remote content, no cookies and no session-bearing endpoints.

Related observations:

- The development server sets `Access-Control-Allow-Origin: *`. This is appropriate for a localhost JSON fixture server but would be inappropriate if it were ever exposed publicly with sensitive content. It serves read-only static JSON, so the exposure is limited.

- The development server implements path-traversal protection by resolving the request against the registry root and rejecting escapes — a good control, and notably absent from the file-writing path.
- The extension renders recipe descriptions in a `MarkdownString` in the completion list. VS Code sanitises documentation markdown by default; a malicious registry could still attempt content injection through that channel. Severity low, and only reachable with a configured untrusted registry.

23.8 Dependency posture

- No dependency vulnerability scanning, audit step, Dependabot configuration or lockfile-integrity check exists in CI.
- Versions are declared with caret ranges (for example `commander ^12`, `handlebars ^4.7.8`, `esbuild ^0.21.5`), and a `pnpm-lock.yaml` is committed.
- `pnpm-workspace.yaml` explicitly allows build scripts for `esbuild` and `@vscode/vsce-sign`, and disables them for `keytar` — a positive, deliberate posture-control measure.

Status: PARTIAL. **Recommendation:** add `pnpm audit` (or an equivalent) to CI and enable dependency update automation.

23.9 Positive security controls

- Network fetches are time-bounded (10 s), so a hostile or hung endpoint cannot stall the client indefinitely.
- The development server defends against path traversal.
- Regeneration is diff-previewed and confirmation-gated by default.
- Primitives skip existing files, reducing accidental overwrites.
- Recipe validation runs before publication and checks structure and template existence.
- No telemetry means no unintended data egress.
- The offline-by-default design means most users never touch the trust boundary described in Section 23.

Status: IMPLEMENTED.

23.10 Data integrity finding

ID	Finding	Severity	Evidence	Status
SEC-16	`add` overwrites any existing file at a recipe path without warning or backup (only primitives are skip-aware).	Medium	`writeFiles.ts` line 16.	OPEN

ID	Finding	Severity	Evidence	Status
SEC-17	If dependency installation fails, files have already been written and the manifest is not updated, leaving a project whose files and manifest disagree.	Low	`add.ts`, lines 238–272 order.	OPEN

Recommendation (RECOMMENDED): prompt before overwriting a file that was not written by a previous generation (that is, one not listed in the manifest), and record the manifest entry before installing dependencies (or roll back file writes on install failure).

23.11 Security summary

Table 23.7: Findings by severity

Severity	Findings
High	SEC-04 (init command injection), SEC-05 (dependency-name interpolation), SEC-08 (unverified remote recipes), SEC-09 (no path containment), SEC-13 (placeholder authentication).
Medium	SEC-10 (HTTPS not enforced), SEC-11 (no integrity check), SEC-12 (credential logging in generated code), SEC-16 (silent overwrite).
Low	SEC-07 (template execution surface — accepted), SEC-14 (no secure storage generated), SEC-17 (partial-failure consistency).
Pass	SEC-01, SEC-02, SEC-03, SEC-15 and the absence of SQL/CSRF/XSS surfaces.

Status: RECOMMENDED (all remediations). None of the high-severity findings is exploitable without a deliberate action by the user (supplying a crafted project name, or configuring an untrusted registry), except SEC-12 and SEC-13, which are properties of the code that the tool *generates* and will reach users by default.

24 Performance

24.1 Method and evidence boundary

Status: IMPLEMENTED (code paths) / UNKNOWN (measured runtimes).

No benchmark, profiler run or performance test exists in the repository, and no timing instrumentation is present in the code. The analysis below is therefore derived from *static* properties of the code — algorithmic complexity, bundle sizes and workload sizes — plus artefact sizes measured directly from the working tree. No wall-clock figure in this chapter is claimed as measured on the user's machine unless it is an artefact size.

24.2 Measured artefact sizes

All values below were read from the working tree at analysis time.

Table 24.1: Measured artefact sizes

Artefact	Size	Path
VS Code extension bundle (esbuild, dev sourcemap)	568,423 B	`packages/vscode-extension/out/extension.js`
CLI generated offline catalog (TypeScript)	249,874 B	`packages/cli/dist/generated/registr data.js`
CLI generated offline catalog (source)	239,885 B	`packages/cli/src/generated/registry data.ts`
Registry offline bundle	242,775 B	`packages/registry/bundle.json`
Registry published index	12,748 B	`registry/index.json`
Packaged extension (VSIX 0.1.3)	502,508 B	`expo-snippets-0.1.3.vsix`

About 95 % of the CLI's compiled JavaScript and roughly 40 % of the extension bundle is the inlined recipe catalog (templates and metadata), not logic. This is the single dominant size factor in the product.

24.3 Workload sizes

- 24 recipes in the catalog; the largest recipe (`auth/login`) has 7 file entries.
- 48 Handlebars templates.
- A typical generation writes 1–7 files, each below roughly 300 lines.
- A primitive resolution pass touches at most a handful of recipes (`ui/button`, `ui/input`, `ui/utils`).

24.4 Algorithmic characteristics

Table 24.2: Complexity of the principal routines

Routine	Complexity	Notes
Catalog load	$O(1)$ per lookup	The catalog is a module constant in the CLI and extension; a lookup is an array `find`.
Remote index fetch	$O(1)$ request	One HTTP GET of a 12.7 KB document.
Remote recipe fetch	$O(1)$ request	One HTTP GET per selected recipe.
Template render	$O(n)$ per template	Handlebars compile plus execute, where n is template length.
Primitive resolution	$O(r \cdot f)$	r = primitives transitively referenced, f = files per primitive; guarded against cycles.
Diff	$O(n \cdot m)$ time, $O(n \cdot m)$ space	Longest-common-subsequence table over two files; n, m are line counts.

The only super-linear routine is the update diff, and its inputs are single generated source files of a few hundred lines, so the practical cost is negligible (a few hundred thousand table cells).

24.5 Observed efficiency measures

- **Offline-first design.** The catalog ships inside each client, so the common path performs *no* network work at all. This is the most significant performance property of the product.
- **Bounded network calls.** Every fetch carries a 10-second timeout, so a slow or dead registry degrades gracefully instead of hanging.
- **Extension caching.** The extension caches a remote catalog for 5 minutes, avoiding repeated index fetches during a working session.
- **Packaging.** The extension is bundled (not shipped as many files); `.vscodeignore` excludes sources and dev material from the VSIX.
- **Build ordering.** The registry is prepared once per build and inlined, so runtime never reads registry files from disk.

Status: IMPLEMENTED.

24.6 Optimisation opportunities

Table 24.3: Performance findings

ID	Finding	Evidence	Recommendation
PERF-01	Templates are recompiled on every render; there is no compiled-template cache.	`core/src/generate.ts` line 9 calls `Handlebars.compile` inside `renderTemplate`.	Cache compiled templates by template string, or precompile at build time with `handlebars`. Impact is small at current volumes.

ID	Finding	Evidence	Recommendation
PERF-02	The catalog is duplicated: it exists as JSON (`bundle.json`), as a generated `.ts` module per client, and inside each client bundle.	Three copies measured above.	Acceptable; a shared generated package would remove duplication if the catalog grows.
PERF-03	The diff builds a full $(n+1) \times (m+1)$ table and has no context grouping, so a large change prints every line.	`cli/src/utils/diff.ts`.	Add a context limit or switch to a streaming diff for very large files.
PERF-04	No performance budget, benchmark or regression guard is defined.	Absence in CI and tests.	Introduce a lightweight generation-time benchmark in CI (RECOMMENDED).

24.7 Perceived performance (user experience)

Because generation is local and synchronous, the dominant perceived cost is the *dependency install* step, which shells out to `npm`/`pnpm` and is inherently network- and disk-bound. The tool mitigates this by making installation optional (`-no-install` in the CLI, `expo-snips.installDependencies` in the extension) and by running it in a visible terminal in the extension so the user sees progress.

Status: INFERRED from code; no timing data available.

24.8 Summary

Table 24.4: Performance posture

Dimension	Assessment
Correctness of complexity	All hot paths are linear or constant except the diff, which is small-input.
Cold-start cost	Dominated by loading the inlined catalog (≈240 KB of JavaScript).
Network cost	Zero on the default path; one index request plus one recipe request on the remote path.
Largest user-visible delay	Dependency installation.
Measurement maturity	None — this is the principal gap.

Status: IMPLEMENTED (design) / RECOMMENDED (measurement).

25 Scalability

25.1 Scope of the question

Expo Snips is a developer tool distributed to individual workstations and an editor. It is not a multi-tenant online service, so conventional *horizontal* web-scale concerns (load balancing, database sharding, request fan-out) are **NOT APPLICABLE**. Scalability here means three concrete things:

1. **Catalog growth** — does the product still work as the number of recipes and templates grows?
2. **Consumer base growth** — does the delivery mechanism (npm, GitHub Pages, VS Code Marketplace) absorb many users?
3. **Team growth** — can more contributors add recipes without conflicts?

25.2 Catalog growth

Table 25.1: Catalog growth analysis

Dimension	Assessment	Reasoning
Lookup	Good	In-memory array; negligible at hundreds of entries.
Build time	Good	Linear in recipe count; `build:registry` walks the data directory once.
Offline bundle size	Concern	Each client inlines the whole catalog. At 24 recipes the CLI's catalog module is ≈240 KB; it grows linearly, so a catalog of several hundred recipes would bloat the npm package and the extension bundle proportionally.
Template duplication	Concern	shadcn and per-styling overrides duplicate template text per recipe rather than sharing components.
Delivery size limits	Risk	npm enforces package size guidance; very large catalogs would need to move to the remote-registry path by default.

INFERRED: the current architecture scales comfortably to a few dozen recipes but the inline-everything decision becomes the limiting factor at scale. The product already contains the escape hatch (the remote registry and `-registry`/`registryUrl`), but it is off by default.

RECOMMENDED: if the catalog is expected to exceed roughly one hundred recipes, move the default to a lighter client that fetches an index and loads individual recipes on demand, and keep the full bundle only as an offline fallback.

25.3 Consumer base growth

- **npm distribution:** the public registry is designed for very high read concurrency; the two published packages (`@expo-snips/core`, `@expo-snips/cli`) impose no server-side burden on the project.
- **Registry hosting:** the registry is a set of static JSON files deployed to GitHub Pages, which is a CDN-backed static host. Serving `index.json` and per-recipe documents scales without any application server. There is no dynamic backend to saturate.
- **VS Code extension:** distribution and updates are handled entirely by the Marketplace; the extension makes no call to project-owned infrastructure unless the user configures a registry URL.
- **No telemetry:** because nothing reports back, traffic growth has no analytics or write path to contend with.

Status: IMPLEMENTED. The product's delivery model is effectively infinitely scalable for its purpose because it is static and distributed.

25.4 Team growth

- Recipes are independent JSON documents under `packages/registry/data/`; two contributors adding different recipes touch different files, which is a low-conflict shape for git.
- The `validate` script enforces structure per recipe, and the `build:registry` script fails on a missing template, giving contributors fast feedback.
- There is no shared registry lock file or generated index committed per change beyond the build output, so merge conflicts on the index are avoided.

Status: IMPLEMENTED. **Gap:** the absence of automated tests and the absence of a root `README` or contribution guide make onboarding rely on reading source (see Chapter 35).

25.5 Known scalability limits

Table 25.2: Scalability limits and risks

ID	Limit	Trigger
SCALE-01	Entire catalog inlined into every client bundle.	Catalog grows past a few dozen recipes.
SCALE-02	No pagination or search index for the remote registry; the client fetches and filters the whole index.	Very large catalogs over a remote registry.
SCALE-03	Single static registry host with no versioning strategy documented for breaking recipe changes.	Multiple concurrent catalog versions in the wild.
SCALE-04	Dependency installation is serial and user-visible.	Recipes with many or heavy dependencies.

25.6 Summary

Table 25.3: Scalability posture

Axis	Verdict
Runtime concurrency	NOT APPLICABLE (single-user local tool).
Delivery / distribution	IMPLEMENTED and inherently scalable (static CDN + npm + Marketplace).
Catalog growth	INFERRED to be the real constraint; bounded by the inline bundle.
Team growth	IMPLEMENTED (independent recipe files) with onboarding gaps.

Status: IMPLEMENTED (delivery) / INFERRED (catalog limit) / RECOMMENDED (remote-first default at scale).

26 Testing

26.1 Current state

Status: NOT IMPLEMENTED (no automated tests exist).

A recursive search of the working tree for test or specification files (`*.test.ts`, `*.spec.ts` and JavaScript equivalents) returns no results, and no test runner is declared in any package. The root `package.json` defines a `test` script (`pnpm -r test`), but no workspace package defines a `test` script for it to run. The VS Code extension README lists "Testing infrastructure" as an unfinished roadmap item and "No tests" under known limitations, and the CHANGELOG contains no test-related entries.

What *does* exist and acts as a lightweight correctness gate:

- `scripts/validate.mjs` — a recipe validator (see below).
- `pnpm build` across all packages, which performs TypeScript compilation under `strict` mode and therefore catches type errors.
- `pnpm -filter expo-snips typecheck`, an explicit no-emit type check for the extension.
- `scripts/build-registry.mjs`, which fails if a recipe references a missing template.

These are build-time checks, not tests. They do not execute behaviour.

26.2 The validator as the de-facto quality gate

`scripts/validate.mjs` is the closest thing to an automated test. It is an inline, hand-written validator (it reproduces a subset of Draft-07 JSON Schema rather than using a schema library) and checks each recipe for:

Table 26.1: Checks performed by `validate.mjs`

Check	Detail
Required fields	Every field in `schema.required` must be present and non-null.
Id format	Must match `^[a-z0-9-]+/[a-z0-9-]+$`.
Version format	Must match `^\d+\.\d+\.\d+$`.
Enums	`framework` must be `react-native`; `runtime` must be `expo`.
Supported option values	Each entry of `supported.styling/router/ui/icons` must be one of the schema enums.
Files present	`files` must be a non-empty array; each entry needs `path` and `templateFile`.
Id matches path	The recipe id must equal its path relative to `data/` (cross-check between content and location).
Template existence	Every referenced `.hbs` file must exist on disk.

Observations:

- The validator is genuinely useful: it enforces identity, structure and referential integrity of the catalog.
- It does *not* validate the `variants`, `primitives`, `tags` or `dependencies` fields beyond their presence, and it does not validate nested `when` conditions.
- Because it re-implements schema validation by hand, it can drift from `schema.json`; the schema itself is not executed.
- It is invoked by the developer (or by the bake pipeline), not wired into CI (see Chapter 28).

26.3 Rendering is untested

The generation engine (`packages/core`) — the part most likely to regress and the part both clients depend on — has no tests. There is no assertion that a given recipe, options set and variant produce the expected file paths and content, no test of the `when` filtering logic, and no test of primitive recursion. A change to a template or to `getFilesForRecipe` is verified only by building and, at best, running the tool by hand.

26.4 Test coverage by area

Table 26.2: Coverage assessment by area

Area	Coverage	Comment
Recipe data	Structural only	`validate.mjs` checks shape and template existence, not rendered output.
Core engine	None	No unit tests for rendering, filtering or recursion.
CLI commands	None	No command-level tests.
CLI utilities	None	Diff, detection, manifest and install logic untested.
VS Code extension	None	No integration or unit tests; no `@vscode/test-electron` harness.
Scripts	None	The build, bake and serve scripts are not tested.
Type safety	Partial	`strict` TypeScript catches type-level errors at build.

26.5 Recommended test strategy

Status: RECOMMENDED. In order of value per unit of effort:

1. **Golden-file tests for the core engine.** For a representative set of recipes and option combinations, assert that `getFilesForRecipe` returns a fixed set of paths and that rendered content matches a stored snapshot. This is the highest-value test because it protects the shared engine and the templates simultaneously.
2. **Variant and `when` matrix tests.** Assert that each supported styling/router/ui/icons combination selects the intended template, and that a variant overrides the default.

3. **Primitive recursion tests.** Assert that a shadcn recipe pulls in the expected primitive files, that existing files are skipped, and that a synthetic cycle terminates.
4. **Manifest tests.** Assert `upsertEntry` semantics for the same recipe with different variants, and `updateEntryVersions` behaviour.
5. **CLI smoke test.** Run `add` against a temporary directory and assert the resulting file tree and manifest, using `-no-install` to avoid network and package-manager side effects.
6. **Schema-driven validation.** Replace or supplement the hand-written checks with a real JSON Schema validator so `schema.json` is the single source of truth.
7. **Extension integration test.** A minimal `@vscode/test-electron` run that activates the extension and generates into a scratch workspace.

A suitable stack is already implied by the codebase: `vitest` or the Node built-in test runner for the Node packages, and `@vscode/test-electron` plus Mocha for the extension (the VS Code convention). No test framework is currently installed, so this would be a new dependency.

26.6 Summary

Table 26.3: Testing posture

Dimension	Assessment
Unit tests	None.
Integration tests	None.
Structural validation	IMPLEMENTED (`validate.mjs`).
Type checking	IMPLEMENTED (`tsc -strict`, extension `typecheck`).
CI test gate	None (see Chapter 28).
Manual verification	The only current method for behaviour.

Status: NOT IMPLEMENTED / RECOMMENDED.

27 Deployment

27.1 Deployment inventory

The project deploys three distinct things through three different channels.

Table 27.1: Deployment inventory

Artefact	Channel	Automation	Status
`@expo-snips/core`	npm public registry	`publish.yml` on `v*` tags	IMPLEMENTED
`@expo-snips/cli`	npm public registry	`publish.yml` on `v*` tags	IMPLEMENTED
Screen registry (JSON)	GitHub Pages	`deploy-registry.yml` on `main`/`master`	IMPLEMENTED
VS Code extension (`.vsix`)	Manual / Marketplace	None	NOT AUTOMATED
`packages/registry` (npm)	—	`private: true`	NOT PUBLISHED

27.2 Registry deployment

The registry is a folder of static JSON: `index.json` plus one document per recipe under `recipes/`. Its deployment is:

1. Checkout the repository.
2. `node scripts/build-registry.mjs` — this *deletes* the existing `registry/` directory, rebuilds it from `packages/registry/data` with every template inlined, and writes `index.json` plus `recipes/<id>.json`.
3. Copy the built `registry/` folder into `public/registry/`.
4. Publish `public/` to the `gh-pages` branch with `peaceiris/actions-gh-pages@v3`, using the workflow's `GITHUB_TOKEN`.

The workflow runs on every push to `main` or `master` and can be triggered manually via `workflow_dispatch`. It declares `permissions: contents: write`, which the Pages action needs to push the `gh-pages` branch.

Client default. Clients resolve a registry base from configuration; when none is supplied, the CLI's built-in constant points at `https://registry.expo-snips.dev` and the VS Code extension defaults to its *embedded* catalog (empty registry URL). The repository remote is `github.com/unkown812/expo-snips`, and the deployed Pages site would be served under that account's GitHub Pages domain.

INFERRED: the hard-coded default host `registry.expo-snips.dev` is not configured anywhere in this repository (no CNAME file, no DNS configuration). Whether it resolves to the GitHub Pages deployment is **UNKNOWN** from the repository alone.

27.3 npm package deployment

`publish.yml` triggers on tags matching `v*` and publishes the two public packages:

```
- run: pnpm install --frozen-lockfile
- run: pnpm build
- run: pnpm --filter @expo-snips/core publish --no-git-checks --access public
  env: { NODE_AUTH_TOKEN: ${{ secrets.NPM_TOKEN }} }
- run: pnpm --filter @expo-snips/cli publish --no-git-checks --access public
  env: { NODE_AUTH_TOKEN: ${{ secrets.NPM_TOKEN }} }
```

Both packages declare `"publishConfig": {"access": "public"}` and a `files: ["dist"]` allow-list, so only compiled output is uploaded. The CLI's build step first runs `scripts/prepare-registry.mjs`, which inlines the catalog into `src/generated/registry-data.ts` before `tsc` compiles it — so the published npm package is self-contained and works offline.

Note: the workflow publishes only `core` and `cli`. The `registry` package is marked `private` and is never published.

27.4 VS Code extension deployment

Status: NOT AUTOMATED.

Packaging is manual:

```
pnpm --filter expo-snips package   # esbuild + vsce package
```

This produces `expo-snippets-<version>.vsix`. Four packaged extensions (0.1.0 through 0.1.3) are present in the working tree under `packages/vscode-extension/` (they are excluded from git by a `*.vsix` rule in `.gitignore`), which indicates the current distribution practice: build locally and share or install the artefact manually ("Install from VSIX"). There is no publish step to the VS Code Marketplace and no workflow that builds or attaches the VSIX to a GitHub release.

RECOMMENDED: add a workflow that packages the extension on a tag and either publishes to the Marketplace (`vsce publish`) or attaches the VSIX to a GitHub release, so the artefact is reproducible rather than committed.

27.5 Runtime requirements and prerequisites

Table 27.2: Runtime prerequisites

Component	Requirement
Node.js (CLI and build)	≥ 18 (declared in `engines` and in the esbuild `target: node18`). CI uses Node 20.
pnpm	9 (workflow pins `version: 9`); 9+ recommended.
VS Code (extension)	≥ 1.80.0 (declared in `engines.vscode`).
Target project	Any Expo / React Native project; detection reads `package.json` and `tsconfig.json`.

Component	Requirement
Package manager (for deps)	Any of npm, pnpm, yarn or bun; the extension detects which and the CLI delegates to the project's installer.

27.6 What is absent

- **No containerisation.** There is no `Dockerfile`, `docker-compose` file or container registry configuration.
- **No infrastructure-as-code.** Hosting is GitHub Pages; there is no Terraform, Pulumi or cloud configuration.
- **No environment-specific build.** Builds do not branch on an environment; there is a single production shape.
- **No rollback procedure.** Nothing documents how to revert a bad registry publish. Because registry state is rebuilt from the repository, reverting the commit and re-running the workflow restores the previous state — but this is INFERRED, not documented.
- **No release notes automation.** The extension has a manual CHANGELOG; the npm packages have no CHANGELOG.

27.7 Deployment summary

Table 27.3: Deployment posture

Dimension	Assessment
Registry delivery	IMPLEMENTED and automated (GitHub Pages, static JSON).
npm delivery	IMPLEMENTED and automated (tag-triggered).
Extension delivery	NOT AUTOMATED; artefact committed by hand.
Environments	Single (production); no staging or preview.
Containerisation / IaC	NOT APPLICABLE.
Rollback	INFERRED only; not documented.

Status: IMPLEMENTED (registry, npm) / NOT AUTOMATED (extension) / RECOMMENDED (extension release automation, rollback documentation).

28 CI/CD

28.1 Overview

Status: PARTIAL.

The repository contains two GitHub Actions workflows in `.github/workflows/`. There is no CI workflow in the conventional sense: nothing runs on pull requests or pushes to validate the code. Both workflows are *delivery* workflows triggered by merge or tag.

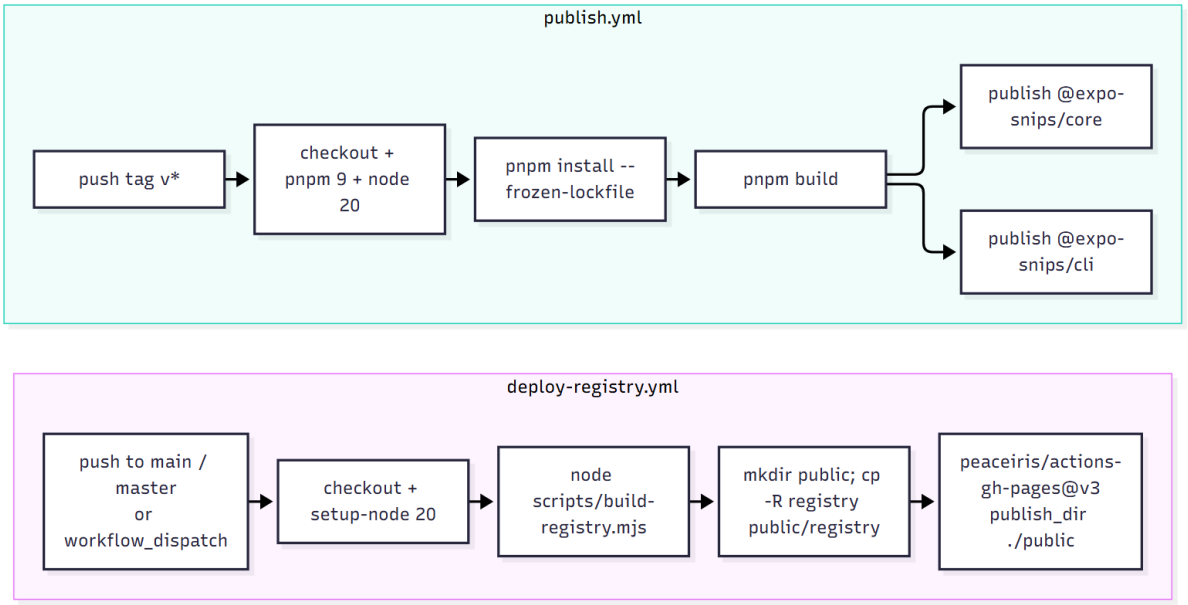


Figure 28.1: Continuous delivery pipelines present in the repository

28.2 Workflow inventory

Table 28.1: GitHub Actions workflows

File	Trigger	Runner	Purpose
`publish.yml`	push to tags `v*`	ubuntu-latest, Node 20, pnpm 9	Build all packages and publish `@expo-snips/core` and `@expo-snips/cli` to npm.
`deploy-registry.yml`	push to `main`/`master`, `workflow_dispatch`	ubuntu-latest, Node 20	Build the registry and deploy it to GitHub Pages.

28.3 Pipeline detail: `publish.yml`

1. `actions/checkout@v4`
2. `pnpm/action-setup@v4` (version 9)
3. `actions/setup-node@v4` (Node 20, `registry-url: https://registry.npmjs.org`)
4. `pnpm install -frozen-lockfile` — reproducible install from the committed lockfile.
5. `pnpm build` — recursive build; this runs `prepare-registry.mjs` for both clients before compilation.
6. Publish `core`, then publish `cli`, each authenticated with `NPM_TOKEN`.

Notes: publishing uses `-no-git-checks` (permitting a publish from a detached HEAD, which is normal on a tag build) and `-access public` (reinforcing the package's own `publishConfig`). There is no headless `npm whoami` check, no provenance/attestation step, and no dry-run gate.

28.4 Pipeline detail: `deploy-registry.yml`

1. `actions/checkout@v4`
2. `actions/setup-node@v4` (Node 20; no dependency install step because the script uses only Node built-ins)
3. `node scripts/build-registry.mjs` — rebuilds `registry/`
4. `mkdir public` and `cp -R registry public/registry`
5. `peaceiris/actions-gh-pages@v3` publishes `./public` to `gh-pages` using `GITHUB_TOKEN`, under `contents: write` permission.

28.5 What the pipelines do not do

Table 28.2: Missing pipeline capabilities

Capability	Present?	Comment
Linting	No	No linter is configured anywhere in the repo.
Automated tests	No	No tests exist; no test step is present.
Recipe validation in CI	No	`validate.mjs` exists but is not invoked by either workflow.
Build on pull requests	No	Neither workflow runs on `pull_request`.
Type checking	Indirect	`pnpm build` runs `tsc`, so type errors fail the delivery build — but only after merge or on a tag, not on a PR.
Dependency audit	No	No `pnpm audit`, Dependabot or CodeQL.
Extension packaging/publish	No	The VSIX is built and committed by hand.
Caching	No	No `actions/cache` or pnpm store cache configured.
Provenance / attestation	No	Plain `npm publish`.
Version bumping	No	Version numbers are edited manually in `package.json`.

28.6 Consequences

- **The pipeline cannot prevent a broken main.** Because no workflow runs on pull requests, a change that fails to build or violates the recipe schema is only discovered when someone builds locally — or when a tag is pushed and publishing fails.
- **The registry is rebuilt on every push to the default branch.** This is good for freshness but means an unreviewed change to a template reaches the published registry immediately after merge.
- **The extension lags the rest of the product.** The CLI and registry are automated while the extension, the most user-facing artefact, is manual.
- **No concurrency control on `deploy-registry.yml`.** Two rapid pushes could race to push the `gh-pages` branch; there is no `concurrency` group.

28.7 Recommended pipeline

Status: RECOMMENDED. A minimal, high-value CI workflow on `pull_request` and `push` to the default branch would:

1. check out, install with `-frozen-lockfile`;
2. run `node scripts/validate.mjs` (recipe integrity);
3. run `pnpm build` (type-check and compile all packages);
4. run tests once they exist (see Chapter 26);
5. add a linter and formatter check;
6. enable dependency update automation.

Then add a `concurrency` group to the registry deployment workflow and a tag-triggered job that packages the extension.

28.8 CI/CD summary

Table 28.3: CI/CD posture

Dimension	Assessment
Continuous integration	NOT IMPLEMENTED (no PR validation).
Continuous delivery — npm	IMPLEMENTED (tag-triggered).
Continuous delivery — registry	IMPLEMENTED (push-triggered).
Continuous delivery — extension	NOT IMPLEMENTED (manual).
Quality gates in CI	None beyond compilation.

29 Integrations

29.1 Integration map

Expo Snips integrates with external systems in three directions: build-time tooling, distribution channels, and the developer projects it generates code into. There is no server-side integration surface (no webhooks, no message queues, no third-party APIs called at runtime other than an optional registry fetch).

Table 29.1: External integrations

Integration	Direction	Status	Notes
npm registry	Outbound (install, publish)	IMPLEMENTED	Dependency install of generated screens; publishing of the two public packages.
GitHub Pages	Outbound (deploy)	IMPLEMENTED	Static hosting of the recipe registry.
VS Code Extension API	Bidirectional (host)	IMPLEMENTED	Commands, completion provider, status bar, settings, terminals.
Remote recipe registry	Inbound (HTTP GET JSON)	IMPLEMENTED (optional, off-by-default)	Configurable base URL; 10 s timeout; extension caches 5 min.
GitHub Actions	Platform	IMPLEMENTED	Two delivery workflows.
VS Code Marketplace	Outbound (distribution)	NOT IMPLEMENTED	No publish step; the VSIX is packaged locally.
Package managers in target project	Outbound (shell)	IMPLEMENTED	Detects npm/pnpm/yarn/bun and shells out to install.
Analytics / telemetry	—	NOT APPLICABLE	None present.
Authentication providers	—	NOT APPLICABLE	No accounts, tokens or OAuth.
Databases	—	NOT APPLICABLE	No database.

29.2 Build-time library integrations

These are libraries the project itself depends on, not external services.

Table 29.2: Third-party libraries used by the product

Library	Used by	Role
`handlebars` ^4.7.8	`core`	Template compilation and rendering; the generation engine.
`commander` ^12	`cli`	Command and option parsing.
`prompts` ^2.4	`cli`	Interactive question prompts.
`chalk` ^4.1	`cli`	Terminal colour output.
`fs-extra` ^11.2	`cli`, extension	Directory creation and file writing.
`esbuild` ^0.21	extension	Bundling the extension to CommonJS.
`@vscode/vsce` ^2.24	extension	Packaging the `.vsix`.
`typescript` ^5.5	all	Compilation and strict type checking.

Note on dependency shape: the CLI and core declare runtime dependencies; the extension declares its runtime helpers (`fs-extra`, `@expo-snips/core`) as *devDependencies* because esbuild bundles them into the shipped artefact. This is correct for a bundled extension but means the extension's declared dependency set is not a runtime contract.

29.3 Libraries the generated screens depend on

When a recipe is generated, the tool installs the recipe's declared dependencies. Across the whole catalog the union is:

Table 29.3: Dependencies declared by recipes

Package	Used for
`react-hook-form`	Form state management in auth and input screens.
`@hookform/resolvers`	Bridges `zod` schemas into `react-hook-form`.
`zod`	Schema validation for forms.
`lucide-react-native`	Icons (selected when `icons=lucide`).
`react-native-maps`	The `map/location` recipe.
`expo-notifications`	The notifications flow.
`class-variance-authority`, `clsx`, `tailwind-merge`	shadcn-style UI primitives and the `cn` class-merging utility.

The generated screens also import modules that are expected to exist in any Expo project and are therefore not installed by the tool: `react`, `react-native`, `react-native-safe-area-context`, `expo-router` (the default router) or `@react-navigation/native` (when that router is selected), and `nativewind` (default styling).

Integration gap: `expo-blur`

Three templates — the `glass` variants of `app/home`, `auth/login` and `onboarding/welcome` — import `BlurView` from `expo-blur`:

```
import { BlurView } from 'expo-blur';
```

However, no recipe declares `expo-blur` in its `dependencies` array, so generating a glass variant does not install the package. A developer selecting a glass variant would get a module-resolution error until they installed `expo-blur` manually.

Status: IMPLEMENTED (templates) / GAP (declared dependency). **Recommendation:** declare `expo-blur` on the three recipes that offer a glass variant.

29.4 The remote registry as an integration

The registry integration is the only network-facing contract and is worth treating as a formal API (see Chapter 16). Its properties:

- **Protocol:** plain HTTPS (or HTTP) GET returning JSON.
- **Endpoints:** `{base}/index.json` and `{base}/recipes/{id}.json`, where the id contains a slash and is therefore URL-encoded by the client.
- **Authentication:** none.
- **Configuration:** CLI `-registry` or `EXPO_SNIPS_REGISTRY`; extension `expo-snips.registryUrl`.
- **Resilience:** 10-second timeouts; the extension caches for 5 minutes and falls back to its embedded catalog on failure; the CLI falls back to its embedded catalog when no base is configured.
- **Failure mode:** a non-2xx response or malformed shape causes an error on the CLI and a silent fallback on the extension.

29.5 Integration risks

- A user-configured registry becomes a trusted code channel with no signature or hash verification (see Chapter 23).
- The hard-coded default host `registry.expo-snips.dev` is not represented in the repository by any hosting configuration, so its liveness is **UNKNOWN**.
- The extension's "fallback to embedded catalog" behaviour is helpful but means a misconfigured registry silently produces the built-in catalog rather than an error, which can mask configuration mistakes.

29.6 Summary

Status: IMPLEMENTED (build tooling, npm, GitHub Pages, VS Code API, optional registry) / NOT IMPLEMENTED (Marketplace publishing) / NOT APPLICABLE (analytics, auth providers, databases).

30 Configuration

30.1 Configuration surfaces

Expo Snips has no configuration file of its own and no server-side configuration. Configuration exists at four points:

1. **CLI flags and environment variables** (command line / process environment).
2. **VS Code settings** (editor settings, per user or workspace).
3. **Recipe definitions** (JSON data that configures generation behaviour per screen).
4. **Per-project state** (the `.expo-snips.json` manifest and the detected stack of the user's project).

There is no `.env` file in the repository and none is read by the code.

30.2 Environment variables

Table 30.1: Environment variables

Variable	Default	Effect
`EXPO_SNIPS_REGISTRY`	empty (embedded catalog)	Overrides the registry base URL used by the CLI. An empty value selects the embedded catalog.
`PORT`	`3000`	Port for the development registry server (`scripts/serve.mjs`).
`NPM_TOKEN`	—	CI only. Authenticates npm publishing in `publish.yml`. Never read by application code.
`GITHUB_TOKEN`	—	CI only. Used by the Pages deploy action. Never read by application code.

Precedence (CLI): an explicit `-registry` flag takes precedence over `EXPO_SNIPS_REGISTRY`; when neither is set, the built-in default registry constant applies. An empty string in either place means "use the catalog bundled with the package".

30.3 CLI options

Global and per-command options are defined with `commander`. The generation-relevant options are:

Table 30.2: CLI generation options

Option	Values	Effect
`-styling`	`nativewind`, `stylesheet`, `tamagui`	Selects the styling template branch; validated against the recipe's `supported.styling`.
`-router`	`expo-router`, `react-navigation`	Selects the navigation template branch; validated against `supported.router`.
`-ui`	`native`, `shadcn`, `custom`	Selects the UI primitive branch; `shadcn` additionally forces `styling=nativewind` and `icons=lucide` and pulls in primitive recipes.
`-icons`	`lucide`, `vector-icons`, `none`	Selects the icon template branch.
`-variant`	*recipe-defined*	Selects a named template variant. Not validated against the recipe.
`-typescript` / `-javascript`	flag	Forces the language branch (overrides detection).
`-no-install`	flag	Skips dependency installation.
`-registry <url>`	URL	Overrides the registry base URL.

When an option is omitted, the value is inferred from the project (see below), then falls back to a default, then to an interactive prompt if the terminal is interactive.

30.4 VS Code settings

The extension contributes a configuration section titled "Expo Snips" with these properties:

Table 30.3: VS Code settings

Setting	Type	Default	Effect
`expo-snips.registryUrl`	string	*empty*	Remote registry base. Empty uses the embedded catalog.
`expo-snips.defaultStyling`	enum	`nativewind`	Fallback styling when detection fails. Values: `nativewind`, `stylesheet`, `tamagui`.
`expo-snips.defaultRouter`	enum	`expo-router`	Fallback router. Values: `expo-router`, `react-navigation`.
`expo-snips.defaultUi`	enum	`native`	Fallback UI primitives. Values: `native`, `shadcn`, `custom`.
`expo-snips.defaultIcons`	enum	`lucide`	Fallback icon library. Values: `lucide`, `vector-icons`, `none`.
`expo-snips.installDependencies`	boolean	`true`	Install recipe dependencies in a terminal after generation.

Configuration mismatch to note: the settings enums advertise `tamagui`, `vector-icons` and `custom`, but no recipe in the catalog currently implements them (every recipe's `supported` array excludes them). Selecting such a fallback would have no recipe to match. The extension README acknowledges this. This is a documentation/configuration inconsistency rather than a runtime defect, but it can mislead users.

30.5 Project detection (configuration by inference)

The tool configures itself from the target project rather than requiring input. Detection reads:

Table 30.4: Detection probes

Setting	Probe
Router	`expo-router` present in dependencies selects `expo-router`; otherwise `@react-navigation/native` selects `react-navigation`.
Styling	`nativewind` in dependencies selects `nativewind`; otherwise `stylesheet`.
UI primitives	`@shadcn/ui` or `react-native-reusables` selects `shadcn`; otherwise `native`.
Icons	`lucide-react-native` selects `lucide`; otherwise `none`.
TypeScript	Presence of `tsconfig.json`.

Status: IMPLEMENTED. Detected values are used as defaults; explicit flags (CLI) or settings fallbacks (extension) take precedence.

30.6 Recipe-level configuration

Recipes are the richest configuration surface. A recipe declares:

- its identity and version (`id`, `name`, `description`, `version`);
- the options it *supports* (`supported.styling/router/ui/icons`) — the tool will only generate combinations a recipe declares;
- its `dependencies` and `devDependencies`;
- its `files`, each with a target `path`, a `templateFile`, an optional `when` condition, and optional `variants` and `perStyling` template overrides;
- optional `variants` (named style alternatives), `tags` (search keywords) and `primitives` (UI primitive recipes required when `ui=shadcn`).

The authoritative machine-readable definition is `packages/registry/schema.json` (JSON Schema Draft-07), reproduced in Chapter 15.

30.7 Project state: the manifest

```
// .expo-snips.json  (written into the user's project root)
interface Manifest {
  version: number;
  screens: Array<{
    recipeId: string;
```

```
    recipeVersion: string;
    variant?: string;
    options: GenerationOptions;
    files: string[];
    installedAt: string;
  }>;
}
```

The manifest is created by the tool, written with two-space indentation, and tolerates absent or unreadable content by treating the project as having no tracked screens. The upsert key for a screen is the recipe id *plus* the variant, so the same recipe can be tracked once per variant.

30.8 Build and development configuration

Table 30.5: Build-related configuration

File	Purpose
`pnpm-workspace.yaml`	Declares `packages/*` and an `allowBuilds` policy that enables build scripts for `esbuild` and `@vscode/vsce-sign` and disables them for `keytar`.
`tsconfig.base.json`	Shared TypeScript compiler options extended by each package.
Per-package `tsconfig.json`	Package-specific compilation settings.
`esbuild.mjs`	Bundle configuration for the extension (CJS, Node 18 target, `vscode` external, sourcemap on).
`.vscodeignore`	Excludes sources and dev material from the packaged VSIX.
`.gitignore`	Excludes build output and generated files from version control.

There is no `eslint`, `prettier` or `editorconfig` configuration in the repository.

30.9 Configuration summary

Table 30.6: Configuration posture

Surface	Assessment
CLI flags / env	IMPLEMENTED; documented in Chapter 14.
Editor settings	IMPLEMENTED; three values advertise unimplemented options.
Recipe data	IMPLEMENTED; schema-governed.
Detection	IMPLEMENTED; four probes plus a TypeScript probe.
Config files	NOT APPLICABLE (none exists).
Secrets	None in the application; CI secrets only.

31 SEO — Search Engine Optimisation

31.1 Applicability

Classic web SEO: NOT APPLICABLE. The project publishes no website, no landing page, no HTML documentation site and no marketing URL. There is no `index.html`, no `robots.txt`, no `sitemap.xml` and no rendered content for a search engine to index. The only HTTP surface the project owns is the recipe registry, which serves machine-readable JSON.

Discoverability SEO: APPLICABLE. The product is discovered through three **listing** channels that function as its search surface, and these are where optimisation is meaningful:

1. the npm registry (for `@expo-snips/core` and `@expo-snips/cli`);
2. the VS Code Marketplace listing (for the `expo-snippets` extension);
3. GitHub repository search and general web search of repository content.

This chapter therefore analyses the metadata that governs discovery in those channels.

31.2 In-repository SEO signals

Table 31.1: SEO-relevant metadata found in the repository

Artefact	Signal	Content
Extension `package.json`	`displayName`	"Expo Snippets".
Extension `package.json`	`description`	"Generate complete React Native screens with a single abbreviation."
Extension `package.json`	`keywords`	Over 40 keywords covering `react-native`, `expo`, `screen`, `template`, `codegen`, `expo-router`, `nativewind`, `shadcn`, `vscode`, `expo-snips` and more.
Extension `package.json`	`categories`	`Snippets`, `Other`.
Extension `package.json`	`repository.url`	Links to `github.com/unkown812/expo-snips`.
Extension `package.json`	`icon`	`assets/logo.png`.
Extension README	Rich Markdown	Headings, tables, feature lists, quick-start instructions, badges.
CLI `package.json`	`description`	"Beautiful React Native screens, one command away."
Core `package.json`	`description`	"Shared types and generation engine for Expo Snips screens."

Artefact	Signal	Content
Repository	Topic titles	Recipe names and descriptions provide natural-language keywords (`auth/login`, `onboarding/welcome`, ...).

31.3 Strengths

- The extension keyword list is unusually broad and covers the phrases a developer would actually search (`react-native screen generator`, `expo template`, `codegen`, framework names). Keyword coverage is the strongest discovery asset the project has.
- The extension README is substantial and structured, which is favourable both to the Marketplace listing and to GitHub search.
- The `repository` field and README links create a coherent cross-channel trail from listing to source.

Status: IMPLEMENTED.

31.4 Gaps

Table 31.2: SEO findings

ID	Finding	Evidence	Recommendation
SEO-01	No website or landing page exists, so every search result resolves to a repository or a listing rather than a product page.	Absence of any HTML in the repo.	If a product identity is intended, publish a small static site with descriptive copy.
SEO-02	No root README. The top-level repository view shows no descriptive text, which weakens GitHub search relevance and first impressions.	Absence of `README.md` at the repository root.	Add a root README that describes the monorepo and links the packages.
SEO-03	npm packages carry only the `description` field as prose; there is no `README` referenced from the npm listing and no keywords field on the npm packages.	`packages/*/package.json`	Add `keywords` and ensure a README is published with each npm package.

ID	Finding	Evidence	Recommendation
SEO-04	Listing metadata is inconsistent: the extension's repository/author (`unkown812`) differs from the owner host implied by some README links (`github.com/expo-snips/expo-snips`).	`package.json` vs README links.	Standardise the canonical repository URL everywhere.
SEO-05	The registry serves JSON only; nothing about the recipes is human-browsable or indexable.	`scripts/serve.mjs`, `registry/`.	Optionally publish a human-readable catalog page generated from the index.

31.5 Summary

Table 31.3: SEO posture

Dimension	Assessment
Website SEO	NOT APPLICABLE (no website).
Listing keyword coverage	IMPLEMENTED and strong (extension keywords).
npm discoverability	PARTIAL (descriptions only).
Repository discoverability	PARTIAL (no root README).
Cross-channel consistency	Needs work (repository URL mismatch).

Status: NOT APPLICABLE (web SEO) / PARTIAL (listing SEO) / RECOMMENDED (root README, canonical URLs).

32 AEO — Answer Engine Optimisation

32.1 Applicability

Answer Engine Optimisation concerns how readily a product's content is selected and quoted by answer engines — AI assistants, conversational search and featured-snippet surfaces. As with SEO (Chapter 31), there is no project-owned website, so the question reduces to: *how well does the repository and its listings feed answer engines?*

32.2 Content available to answer engines

Answer engines draw on: the README files, the package metadata, the recipe descriptions, the extension CHANGELOG, and any prose in the repository. All of these are plain text or Markdown, which is exactly the format answer engines can consume and quote.

Table 32.1: Answer-engine-relevant content

Source	Format	Value for answer engines
Extension README	Markdown	Explains what the tool does, the problem it solves, and how to use it, in natural language.
`ProjectDescription.md`	Markdown	A product description with framing that answers "what is this?".
Recipe `description` fields	Plain text	One-line answers to "is there a screen for X?".
Extension CHANGELOG	Markdown	Dated, factual statements suitable for quoting.
Package descriptions	Plain text	Short capability statements.

32.3 Answer-readiness of the content

Table 32.2: AEO findings

ID	Finding	Evidence	Recommendation
AEO-01	The repository contains clear "what it is / what problem" prose in the extension README, which is well suited to being quoted verbatim.	Extension README sections "What is Expo Snips?" and "The Problem".	IMPLEMENTED; keep this structure.

ID	Finding	Evidence	Recommendation
AEO-02	FAQ-shaped content is absent. Answer engines favour explicit question-and-answer pairings that can be lifted directly.	No FAQ section anywhere.	RECOMMENDED: add a short FAQ ("Does it work offline?", "Which Expo SDK?", "Is it free?").
AEO-03	`documents/ProjectDescription.md` contains *outdated* statements: it references a recipe count and variant support that the code no longer matches.	`documents/ProjectDescription.md`	Update it, because an answer engine may quote the stale figure. See Chapter 34.
AEO-04	Authoritative one-sentence definitions are duplicated in several places (README, package descriptions) with slightly different wording, which can produce inconsistent answers.	`vscode-extension/package.json` description vs README headline.	Standardise a single canonical one-liner.
AEO-05	No structured data (schema.org `SoftwareApplication` or `SoftwareSourceCode`) is published, because there is no site to host it.	Absence of any HTML.	NOT APPLICABLE unless a site is created; then add JSON-LD.
AEO-06	The README uses license and version badges that are useful, unambiguous facts for an answer engine to extract.	Badge block in the extension README.	IMPLEMENTED.

32.4 Factual accuracy as an AEO risk

The single most important AEO property is accuracy; an answer engine that quotes a wrong fact damages trust more than one that quotes nothing. Two facts about this repository are stated inconsistently across sources:

- **Recipe count:** `ProjectDescription.md` understates the catalog relative to the code, where the count is 24.
- **Variant support:** `ProjectDescription.md` implies variants are unsupported, while three recipes (`app/home`, `auth/login`, `onboarding/welcome`) implement a `glass` variant.

Status: INFERRED from comparison of the document with the code. **Recommendation:** correct the description document (RECOMMENDED).

32.5 Summary

Table 32.3: AEO posture

Dimension	Assessment
Quotable capability prose	IMPLEMENTED (extension README).
FAQ content	ABSENT (recommended).
Structured data	NOT APPLICABLE (no site).
Factual consistency	GAP (outdated description document).

Status: PARTIAL / RECOMMENDED.

33 GEO — Generative Engine Optimisation

33.1 Applicability and scope

Generative Engine Optimisation concerns whether a product is represented and cited accurately by generative systems: large language models and the assistants built on them, including code-generating assistants. For a developer tool, GEO matters because a common way a developer learns about a library is to ask an assistant "how do I scaffold an Expo login screen?".

There is no project-owned website to optimise, so classic GEO tactics (rich pages, citations, entity markup) are **NOT APPLICABLE**. What is applicable is the *corpus footprint*: whether the project's own material is present, self-consistent and machine-consumable where generative systems look.

33.2 Corpus footprint

Table 33.1: Sources a generative system could learn from

Source	Availability	Content
Public GitHub repository	Public (master branch)	Source, recipes, templates, workflows, READMEs.
npm packages	Public	`@expo-snips/core`, `@expo-snips/cli` metadata and (if a README is published) documentation.
VS Code Marketplace listing	Public (when listed)	Extension metadata, README, keywords.
Recipe registry JSON	Public	Structured catalog with descriptions and tags.
Answering-assistant context	Not controlled	Unknown whether any model has been trained on or cites this project.

33.3 Generative-readiness properties

- **Structured, self-describing data:** the recipe catalog is JSON Schema-conformant, with `id`, `name`, `description` and `tags` per recipe. This is precisely the shape a generative system can reason over, and it is a genuine strength.
- **Machine-readable API documentation:** the `@expo-snips/core` package ships TypeScript declarations, and the generated `registry-data.ts` is typed, giving models accurate signatures.
- **Versioned, dated history:** the extension CHANGELOG is dated and factual.
- **No telemetry or private corpus:** nothing about the tool's usage is captured, so there is no private signal to leak or misrepresent.

Status: IMPLEMENTED.

33.4 Representation risks

Table 33.2: GEO findings

ID	Finding	Evidence	Recommendation
GEO-01	Inconsistent identity across sources: the extension publishes under publisher `ukown812` and repository `github.com/unkown812/expo-snips`, while some README links point at `github.com/expo-snips/expo-snips`. A generative system may conflate these into a wrong citation.	Extension `package.json` vs README links.	Canonicalise one repository URL and apply it everywhere.
GEO-02	Name collision risk: the extension is published as `expo-snippets`, the CLI as `@expo-snips/cli`, and the product as "Expo Snips". Models may treat these as three products.	`package.json` names.	Keep the "Expo Snips" brand string adjacent to each package name in its description.
GEO-03	Stale prose in `ProjectDescription.md` can propagate an incorrect fact (recipe count, variant support) into a generated answer.	`documents/ProjectDescription.md`	Correct the document; keep a single source of truth for counts.
GEO-04	No license clarity for the extension beyond "Private License", which a generative system cannot interpret as a known SPDX license.	Extension `package.json` `license` field.	Use an explicit, recognisable license expression if the intent allows.
GEO-05	No canonical, quotable "what is this" one-liner shared across sources.	Three differing descriptions.	Publish one canonical sentence and reuse it.

33.5 Why GEO matters for this product specifically

The product's value proposition — "generate a complete, correct screen for a given stack" — is exactly the kind of question developers pose to coding assistants. If a model has never seen the project or cites it inconsistently, the assistant will instead suggest generic manual boilerplate, which is the alternative the tool exists to replace. GEO is therefore economically meaningful even without a website: the goal is that a generative system, asked for an Expo screen, can name the tool and be correct about what it does.

33.6 Summary

Table 33.3: GEO posture

Dimension	Assessment
Website/entity optimisation	NOT APPLICABLE (no site).
Structured corpus	IMPLEMENTED (schema-valid JSON, typed declarations).
Identity consistency	GAP (publisher/repo/brand mismatch).
Factual consistency	GAP (stale description document).
Model citations	UNKNOWN (not observable from the repository).

Status: PARTIAL / RECOMMENDED (canonical identity, accurate descriptions).

34 Known Issues

34.1 Purpose and method

This chapter records defects, inconsistencies and behavioural surprises that are *present in the code as analysed*. It does not duplicate security findings, which are catalogued in Chapter 23, nor the maintenance backlog in Chapter 35. Each issue is given an identifier, a category, a severity and an evidence pointer. Severity reflects impact on a user of the product: **High** means wrong or broken generated output in a plausible workflow, **Medium** means confusion or a manual workaround, **Low** means cosmetic or documentation-only.

Some issues are documented by the authors themselves (the extension README's "Known Limitations" and the CHANGELOG), and these are marked accordingly.

34.2 Issue register

Table 34.1: Known issues

ID	Category	Severity	Status	Description and evidence
KI-01	Generated styling	High	Open	shadcn-generated screens use design tokens such as `bg-background`, `text-foreground`, `border-border` and `ring-ring`, but the repository ships no Tailwind/NativeWind theme configuration defining those tokens. A generated shadcn screen can therefore render unstyled until the user adds the token definitions. Evidence: shadcn templates under `packages/registry/templates`; no `tailwind.config.*` in the repository.
KI-02	Missing dependency	High	Open	The `glass` variants of `app/home`, `auth/login` and `onboarding/welcome` import `expo-blur`, which no recipe declares in `dependencies`; the package is not installed. Evidence: `*.glass.hbs` template imports; recipe JSON dependency arrays.

ID	Category	Severity	Status	Description and evidence
KI-03	Interactive generation	High	Open (documented)	Auth hooks generated by `auth/login` and `auth/register` are placeholders: they wait a fixed delay and then always report success. The shadcn login form only `console.log`s the submitted data and does not call the hook at all. Shipping without replacement yields a non-functional login. Evidence: `hooks.use-login.hbs`, `hooks.use-register.hbs`, `components.login-form.shadcn.hbs`.
KI-04	Advertised-but-unimplemented options	Medium	Open (documented)	`tamagui`, `vector-icons` and `custom` are offered in the extension settings and exist in the schema, but no recipe implements them. Selecting one finds no matching recipe. Evidence: extension `package.json` enums vs recipe `supported` arrays.
KI-05	CLI version reporting	Medium	Open	The CLI's `-version` output is a hard-coded literal rather than the package version, so it will drift from `package.json` on release. Evidence: `packages/cli/src/index.ts`.
KI-06	Silent overwrite	Medium	Open	`add` overwrites any existing file at a recipe path without confirmation or backup; only primitive files are skip-aware. Evidence: `cli/src/utils/writeFiles.ts`. (Also logged as SEC-16.)
KI-07	Variant validation	Medium	Open	An unknown `-variant` value is silently ignored and the default template is produced, with no warning. Evidence: `core/src/generate.ts` `resolveTemplateString`.
KI-08	Silent fallback	Medium	Open	When a configured remote registry is unreachable, the extension silently uses its built-in catalog; the user cannot tell that their configured registry was ignored. Evidence: extension `registry.ts` fallback path.

ID	Category	Severity	Status	Description and evidence
KI-09	Stale product document	Medium	Open	`documents/ProjectDescription.md` understates the catalog and implies variants are unsupported, contradicting the code (24 recipes; variants present). Evidence: document vs `packages/registry/data/**` and `registry/index.json`.
KI-10	Partial-failure consistency	Low	Open	If dependency installation fails, generated files have already been written and the manifest may not be updated, leaving files and manifest disagreeing. Evidence: ordering in `cli/src/commands/add.ts`. (Also logged as SEC-17.)
KI-11	Credential logging	Low	Open	Generated auth code logs submitted credentials to the console. Evidence: auth hook templates. (Also logged as SEC-12.)
KI-12	Manifest tolerance	Low	Open	A malformed or unreadable `.expo-snips.json` is treated as an empty manifest rather than reported, so corruption is silent. Evidence: `cli/src/utils/manifest.ts`.
KI-13	Generated artefacts retained in the tree	Low	Open	Two build outputs are kept in the working tree: the built registry (`registry/`, 25 files) and the offline bundle (`packages/registry/bundle.json`) are *tracked* by git, while four packaged extensions (`expo-snippets-0.1.0..0.1.3.vsix`) sit untracked in the extension directory (excluded by a `*.vsix` rule in `.gitignore`). Tracking generated files invites drift from source. Evidence: `git ls-files`, `.gitignore`.
KI-14	Registry host unverifiable	Low	Unknown	The CLI's default registry host `registry.expo-snips.dev` has no hosting configuration in the repository, so its existence and ownership cannot be verified here. Evidence: registry resolution constants; no CNAME or pages config.

ID	Category	Severity	Status	Description and evidence
KI-15	Generated registry tracked in git	Low	Open	Because `registry/` is tracked *and* regenerated on every push to the default branch, the deployed output and the committed copy can diverge if the deploy workflow's build differs from a manual build. The single source of truth is `packages/registry/data/`. Evidence: `git ls-files registry`; `deploy-registry.yml`.

34.3 Author-acknowledged limitations

The extension README's "Known Limitations" section explicitly states: no tests; no CI/CD for the extension (packaged manually); limited icon/styling options (`tamagui`, `vector-icons`, `custom` unsupported); offline by default; and VS Code only (the extension and CLI are independently installed). These align with KI-02 through KI-05 above and with the findings in Chapter 26 and Chapter 28.

Status: IMPLEMENTED (documented by the authors). This is a positive practice — the limitations are stated rather than hidden.

34.4 Corrections required to prior documentation

Two specific corrections are needed to bring the pre-existing `ProjectDescription.md` into agreement with the code:

Table 34.2: Documentation corrections

Claim as written	Code reality	Correction
An earlier recipe count (stated as 19).	24 recipes in `packages/registry/data`.	Update the count to 24.
Variants described as unsupported.	`app/home`, `auth/login` and `onboarding/welcome` define a `glass` variant.	Document variant support and the three recipes that use it.

34.5 Issue density by area

Table 34.3: Issues by category

Category	Count	Highest severity
Generated output correctness	3 (KI-01, KI-02, KI-03)	High
Configuration / UX	4 (KI-04, KI-05, KI-07, KI-08)	Medium
Data integrity	3 (KI-06, KI-10, KI-12)	Medium
Documentation / hygiene	5 (KI-09, KI-11, KI-13, KI-14, KI-15)	Medium

34.6 Summary

Status: the product is a functioning, coherent tool, but three high-severity issues affect the *output* it produces: shadcn styling tokens without theme configuration (KI-01), an undeclared `expo-blur` import on glass variants (KI-02), and placeholder authentication code that a developer might ship unmodified (KI-03). These are the items to address first because they undermine the product's core promise of "production-quality screens".

35 Technical Debt

35.1 Definition and method

Technical debt here means maintenance cost accepted in the current codebase: missing safety nets, duplication, hard-coded values, and structure that will resist change. Each item has an identifier, an estimate of the interest it accrues (how much it will slow future work), the evidence, and a recommended action. Items are ordered by the value of paying them down.

This is distinct from Chapter 34, which describes observable defects. Some findings appear in both where a defect is also a structural weakness.

35.2 Debt register

Table 35.1: Technical debt register

ID	Item	Interest	Evidence and consequence
TD-01	No automated tests anywhere.	High	Absence of any test file. Every change to the shared core engine or to a template is unverified except by hand; regressions reach users. The core engine is used by two clients, so a single mistake is amplified.
TD-02	No linting or formatting configuration.	Medium	No `eslint`, `prettier` or `editorconfig`. Style is upheld by convention only; diffs carry incidental noise and code quality is inconsistent.
TD-03	No continuous integration on pull requests.	High	Neither workflow runs on `pull_request`. Broken builds and invalid recipes are discovered late (on merge or at release). See Chapter 28.
TD-04	Catalog inlined into every client.	Medium	The catalog is duplicated as `bundle.json`, per-client generated TypeScript and code inside each bundle (≈240 KB each). It grows linearly with the recipe count and inflates both the npm package and the extension. See Chapter 25.
TD-05	Templates are recompiled on each render.	Low	`renderTemplate` calls `Handlebars.compile` every time. A compiled-template cache is a small, safe improvement.
TD-06	Hard-coded CLI version string.	Low	Two authorities for the version number will diverge on release. Read from `package.json` instead.

ID	Item	Interest	Evidence and consequence
TD-07	Hand-written schema validation.	Medium	`validate.mjs` re-implements a subset of JSON Schema, so it can drift from `schema.json` and it is not exercised in CI. Use a real validator.
TD-08	Duplicated generation logic across the two clients.	High	The CLI (`commands/add.ts` and utilities) and the extension (`generator.ts`) each implement detection, option resolution, file writing, manifest handling and primitive resolution independently. Fixes must be applied twice, and the two paths can diverge.
TD-09	No root README or contribution guide.	Medium	The repository root has no `README.md`. Onboarding a contributor requires reading source. See Chapter 38.
TD-10	Generated artefacts tracked in version control.	Low	The built registry (`registry/`, 25 files) and the offline bundle (`packages/registry/bundle.json`) are committed even though they are derived from `packages/registry/data/`. The four local `.vsix` files are correctly excluded by `.gitignore` but remain in the working tree. Derived files in history invite drift and noisy diffs.
TD-11	Registry package lacks a licence declaration.	Low	`packages/registry/package.json` has no `license` field, while the other packages do. Ambiguous reuse terms.
TD-12	No error-classification or logging convention.	Medium	Errors are surfaced with ad-hoc strings; there is no shared logging layer or set of error types. Diagnostics are hard to improve consistently.
TD-13	Silent tolerance of corrupt state.	Low	An unreadable manifest is treated as empty. Convenient, but it hides corruption.
TD-14	No dependency-update automation or audit.	Medium	No Dependabot, Renovate or `pnpm audit` in CI; caret ranges mean drift without signal.
TD-15	Divergent shell execution across platforms.	Low	Generators shell out for installation; the approach is not wrapped behind a single abstraction, complicating testing and cross-platform reasoning.

35.3 The dominant debt: two clients, one engine, duplicated orchestration

`@expo-snips/core` correctly centralises *rendering*. What it does *not* centralise is everything around rendering: project detection, option precedence, the shadcn enforcement rules, primitive

resolution, file writing, manifest updates and dependency installation. Both the CLI and the extension implement these separately.

The evidence:

- `packages/cli/src/utils/detectProject.ts` and `packages/vscode-extension/src/project.ts` both probe `package.json` and `tsconfig.json`.
- `packages/cli/src/utils/primitives.ts` and the download/primitive logic in `packages/vscode-extension/src/generator.ts` both resolve primitive recipes.
- `packages/cli/src/utils/registry.ts` and `packages/vscode-extension/src/registry.ts` both fetch and fall back.
- `packages/cli/src/utils/manifest.ts` and the manifest functions in `project.ts` both read and write `.expo-snips.json`.

Consequence: a behavioural change (for example, "shadcn always implies NativeWind and Lucide") must be made in more than one place, and there is currently no test that would catch a divergence.

Recommendation (RECOMMENDED): promote the orchestration layer into `@expo-snips/core` (or a new shared package) so that both clients call one implementation, leaving only UI/terminal concerns in the clients.

35.4 Debt heat map

Table 35.2: Debt by theme and interest

Theme	Interest	Items
Verification (tests, CI, lint)	High	TD-01, TD-03, TD-02, TD-07, TD-14
Shared architecture	High	TD-08, TD-04
Documentation / onboarding	Medium	TD-09, TD-11
Robustness	Medium	TD-12, TD-13
Hygiene	Low	TD-10, TD-06, TD-05, TD-15

35.5 Priority

1. **Introduce CI with validation and build** (TD-03) — nearest-term, protects everything else.
2. **Add golden-file tests for the core engine** (TD-01) — makes the shared engine safe to refactor.
3. **Unify the orchestration layer** (TD-08) — removes the highest structural interest.
4. **Address the output-correctness issues** in Chapter 34 (KI-01, KI-02, KI-03) — these are product promises, not just hygiene.
5. **Pay down the remaining low-interest items** opportunistically.

35.6 Summary

Status: the debt is concentrated and identifiable rather than diffuse. The core rendering engine is small, well-typed and reasonably factored; the debt lives in verification (no tests, no

PR CI) and in duplicated orchestration across the two clients. Both are addressable without rearchitecting, and both have high leverage because the engine is shared.

36 Roadmap

36.1 Basis

This roadmap is reconstructed from three sources in the repository: the extension README's "Roadmap" checklist, the extension CHANGELOG's `[Unreleased]` section, and gaps established elsewhere in this document. Where an item is drawn from the repository it is labelled accordingly; items this analysis proposes are labelled RECOMMENDED. Nothing in this chapter should be read as a commitment by the authors.

36.2 Repository-stated roadmap

The extension README lists these items as outstanding (unchecked):

Table 36.1: Items the repository itself lists as future work

ID	Item	Source	Reality check
RM-01	`expo-snips update` — update installed screens to the latest recipe versions.	Extension README	**Already implemented** in the CLI (`commands/update.ts`). The README checklist is stale.
RM-02	`expo-snips status` — inspect installed screens and available updates.	Extension README	**Already implemented** in the CLI (`commands/status.ts`). Stale.
RM-03	Tamagui styling support.	README	Genuinely outstanding; appears in the schema and settings but has no recipe.
RM-04	Vector Icons support.	README	Genuinely outstanding; enum exists, no recipe uses it.
RM-05	Custom UI primitive support.	README	Genuinely outstanding; enum exists, no recipe uses it.
RM-06	Expanded screen catalog.	README	Ongoing by nature.
RM-07	Testing infrastructure.	README	Outstanding. See Chapter 26.
RM-08	CI/CD pipeline for the VS Code extension.	README	Outstanding. See Chapter 28.

Note: RM-01 and RM-02 are already delivered, so the README roadmap should be corrected. This is a small but user-visible inaccuracy.

36.3 Near-term recommendations

Status: RECOMMENDED. These are the items with the highest ratio of value to effort, derived from the findings chapters.

Table 36.2: Near-term recommendations

ID	Recommendation	Addresses	Rationale
RN-01	Add a pull-request CI workflow that runs recipe validation, build and (later) tests.	TD-03, TD-07	Cheapest, highest-leverage safety net.
RN-02	Add golden-file tests for the core rendering engine.	TD-01, Chapter 26	Protects the shared engine and all templates at once.
RN-03	Ship a NativeWind/Tailwind theme defining the shadcn tokens (`background`, `foreground`, `border`, `ring`, ...).	KI-01	Without it, generated shadcn screens may not style correctly.
RN-04	Declare `expo-blur` on the three recipes that offer a glass variant.	KI-02	One-line fixes that make the glass variants functional.
RN-05	Make generated auth hooks obviously stubbed and non-logging: remove credential `console.log`, and mark the hook as a placeholder in its comment.	KI-03, KI-11	Reduces the chance of shipping non-functional auth.
RN-06	Add a root `README` and a contribution guide.	TD-09, Chapter 38	Onboarding and discoverability.
RN-07	Canonicalise the repository/brand identity across all metadata.	SEO-04, GEO-01/02/05	Removes citation ambiguity.
RN-08	Add extension packaging to CI (package on tag; optionally publish or attach the VSIX).	RM-08	Removes the manual release step and the reliance on local builds.

36.4 Mid-term recommendations

Table 36.3: Mid-term recommendations

ID	Recommendation	Rationale
RM-10	Extract the shared orchestration layer (detection, option resolution, primitives, file writing, manifest) into the core package.	Removes the highest structural debt (TD-08); both clients call one implementation.
RM-11	Replace hand-written validation with a real JSON Schema validator, or generate validation from the schema.	Makes `schema.json` the single source of truth (TD-07).

ID	Recommendation	Rationale
RM-12	Implement one of the advertised-but-missing options end to end (Tamagui is the largest and most requested).	Closes a configuration inconsistency (KI-04, RM-03).
RM-13	Introduce a compiled-template cache.	Small, safe performance win (TD-05, PERF-01).
RM-14	Harden the registry trust boundary: path containment, dependency-name validation, HTTPS enforcement and content hashes.	Addresses high-severity security findings (SEC-05, SEC-08, SEC-09, SEC-10, SEC-11).
RM-15	Add dependency-update automation and an audit step.	Supply-chain hygiene (TD-14).

36.5 Long-term / architectural directions

Status: RECOMMENDED (strategic; not required for the current product).

- **Remote-first catalog option.** If the catalog grows well beyond today's size, offer a client that fetches an index and loads recipes on demand, keeping the full inline bundle as an offline fallback (Chapter 25, SCALE-01/02).
- **A human-readable catalog site.** Derive a browsable page from the registry index to serve discoverability (SEO-05) and answer engines (Chapter 32).
- **Recipe authoring tooling.** A generator and preview for new recipes would lower the barrier to expanding the catalog (RM-06).
- **Cross-language reach.** The engine is TypeScript/Handlebars bound; a documented registry contract already exists, so other front-ends could consume it.

36.6 Sequencing

Table 36.4: Suggested sequencing

Horizon	Focus	Items
Immediate	Correctness and safety nets	RN-01, RN-02, RN-03, RN-04, RN-05
Short	Onboarding and consistency	RN-06, RN-07, RN-08, TD-10
Mid	Architecture and features	RM-10, RM-11, RM-12, RM-13
Long	Reach and hardening	RM-14, RM-15, remote-first catalog, catalog site

36.7 Summary

The repository's own roadmap is small and mostly honest, marred only by listing two already-implemented CLI features as outstanding. The highest-value work is not new features but the verification and correctness items above: a CI gate, a golden-file test suite, and the three output-correctness fixes that protect the product's core promise.

37 Development Guidelines

37.1 Purpose

No formal style guide, linter configuration or `CONTRIBUTING` file exists in the repository. This chapter records the *implicit* conventions that the existing code follows, so that new code can be consistent with it. Conventions derived from the code are labelled INFERRED; those stated by the authors are labelled IMPLEMENTED.

37.2 Repository layout and tooling

- The repository is a pnpm workspace monorepo with packages under `packages/*` (`core`, `cli`, `registry`, `vscode-extension`).
- Node $\geq$ 18 is required; CI uses Node 20; pnpm 9 is pinned in CI.
- Shared root scripts: `build`, `dev`, `test`, `validate`, `build:registry`, `serve` (all recursive where applicable).
- Build output directories (`dist`, `out`) and the generated registry modules are excluded by `.gitignore`.

37.3 TypeScript conventions

The shared compiler configuration (`tsconfig.base.json`) is the primary contract:

Table 37.1: TypeScript settings and their implications

Setting	Value	Implication for contributors
`strict`	`true`	No implicit `any`; null checks enforced. Code must be fully typed.
`target` / `module`	ES2020 / CommonJS	Use ES2020 syntax; output is CommonJS for the Node packages.
`moduleResolution`	`node`	Classic Node resolution; import workspace packages by name.
`esModuleInterop`	`true`	Default imports from CommonJS modules are permitted.
`declaration` / `declarationMap`	`true`	Public `core` API is published with types.
`resolveJsonModule`	`true`	JSON can be imported as a module.

INFERRED coding style: interfaces and type aliases are preferred for public shapes (`ScreenRecipe`, `GenerationOptions`, `DetectionResult`); functions are small and single-purpose; utilities live in a `utils/` folder within a package; the core exposes a small public surface via `index.ts`. No barrel files beyond the package entry points are used.

37.4 Extension conventions (INFERRED)

- The extension is bundled with esbuild to CommonJS, targeting Node 18, with `vscode` marked external and source maps enabled.
- Runtime libraries the extension needs are bundled, so they are declared as `devDependencies` rather than `dependencies`.
- The extension entry point is `src/extension.ts`; commands, completion provider, generation and project logic are separated into their own modules.
- The offline catalog is regenerated at build time by `scripts/prepare-registry.mjs`; the generated file is never edited by hand.

37.5 Recipe and template conventions (INFERRED + IMPLEMENTED)

Recipes are the project's main "content" authoring surface, and the conventions here are strong and consistent.

Recipe identity

- An id is `<category>/<name>`, lowercase, hyphen-separated (`^[a-z0-9-]+/[a-z0-9-]+`) — for example `auth/login`.
- The recipe file must live at a path that matches the id: `packages/registry/data/auth/login.json`. The validator enforces this.
- The recipe `version` is semver of the recipe itself (`auth/login` is at 0.3.0).

Recipe content

- `supported` must list only options the recipe actually implements; the clients reject unsupported combinations.
- Every output file is a separate entry in `files`, with a target `path` and a `templateFile`. Multiple entries can target the same path under different `when` conditions (for example TypeScript vs JavaScript, or native vs shadcn).
- `when` selects a file entry by styling, router, ui, icons and/or TypeScript.
- `variants` at the recipe level names alternative styles; a file maps a variant to its template via `files[].variants`.
- `primitives` lists UI primitive recipe ids required when `ui=shadcn` (for example `ui/utils`, `ui/button`, `ui/input`).
- `dependencies` must include every third-party package the templates import. (The `glass` variants currently violate this; see Chapter 34.)

Template naming

Templates live under `packages/registry/templates/<category>/<name>/` and follow a dotted convention that mirrors the output location, with optional modifier segments before the extension:

Table 37.2: Template naming examples

Template file	Meaning
`auth/login/app.login.index.hbs`	Expo Router `index` screen (default).
`auth/login/app.login.index.shadcn.hbs`	`shadcn` variant of the same screen.
`auth/login/app.login.index.glass.hbs`	`glass` variant of the same screen.
`auth/login/components.login-form.hbs`	A component file.
`auth/login/hooks.use-login.hbs`	A hook file (shared by TS and JS outputs).
`auth/login/screens.loginscreen.react-navigation.hbs`	`react-navigation` screen variant.

Template content rules (INFERRED)

- Templates are TypeScript/JavaScript source, not markup; they render to `.tsx`, `.ts` or `.js` files.
- Handlebars is used only with the two registered helpers (`eq`, `neq`); contributors should not assume custom helpers exist.
- New recipe files must be referenced by path from a recipe document; an orphaned `.hbs` file is not shipped.

37.6 Versioning and change control (INFERRED)

- The public packages are versioned in their `package.json` and released by pushing a `v*` tag.
- The extension follows the CHANGELOG convention (Keep a Changelog plus Semantic Versioning), as stated at the top of its `CHANGELOG.md`.
- Recipe changes are versioned per recipe, independently of the packages.

37.7 Recommended additions to these guidelines

Status: RECOMMENDED.

- Add a linter and formatter (for example ESLint and Prettier) with a committed configuration, and run them in CI.
- Add an `.editorconfig` to fix indentation and line endings across editors.
- Document the separation of concerns explicitly: rendering belongs in `core`; orchestration should also be shared (see Chapter 35); clients should hold only environment-specific code.
- Require a recipe to declare every package its templates import, with an automated check.

37.8 Summary

Status: INFERRED (most conventions) / IMPLEMENTED (recipe identity, schema, TypeScript settings, extension bundling). The codebase is internally consistent and easy to imitate; the gap is that none of this is written down and none of it is enforced automatically.

38 Contribution Guide

38.1 Basis and honesty note

The repository contains **no** `CONTRIBUTING.md`, no `CODE_OF_CONDUCT.md`, no issue or pull-request templates and no `GOVERNANCE` file. There is therefore no author-stated contribution process to document. What follows is derived from the repository's structure, scripts and workflows: it is a factual description of how work can be contributed, labelled INFERRED, followed by recommended additions labelled RECOMMENDED. It is not a substitute for a real contribution policy.

38.2 Prerequisites (INFERRED)

- Node.js 18 or newer (20 recommended; CI uses 20).
- pnpm 9 or newer.
- Git.
- Optional: the VS Code Extension Development Host for working on the extension.

38.3 Getting started (INFERRED)

```
git clone <repository>
cd expo-snips
pnpm install
pnpm build            # build all packages
pnpm validate         # check every recipe against the schema
```

To run the registry locally:

```
pnpm serve            # builds the registry and serves it on PORT (default 3000)
```

To work on the extension:

```
pnpm --filter expo-snips build       # bundle to out/extension.js
pnpm --filter expo-snips watch       # rebuild on change
pnpm --filter expo-snips typecheck   # no-emit type check
```

Press F5 with the extension package open to launch an Extension Development Host.

38.4 Contribution paths

There are two natural kinds of contribution, and they differ substantially.

1. Adding or changing a screen recipe (most common)

1. Create or edit a JSON document under `packages/registry/data/<category>/<name>.json`. The file path must match the recipe id, and the document must satisfy `packages/registry/schema.json`.

2. Add the templates under `packages/registry/templates/<category>/<name>/`, using the dotted naming convention (see Chapter 37).
3. Declare every third-party package the templates import in the recipe's `dependencies`.
4. Run `pnpm validate` to confirm the recipe is well-formed and all referenced templates exist.
5. Run `pnpm build:registry` (or `pnpm serve`) to confirm the recipe builds and to update the committed `registry/` output.

The validator gives fast, specific feedback (id format, version format, enum values, template existence, id/path agreement), so this workflow is contributor friendly.

2. Changing engine or client code

1. Make the change in the relevant package (`core` for the shared engine, `cli` or `vscode-extension` for a client).
2. Run `pnpm build` to type-check and compile.
3. Manually verify generation, since **no automated tests exist** (Chapter 26).
4. If the behaviour is user-visible, note that the same behaviour may need to be applied in both clients, because orchestration is duplicated (Chapter 35, TD-08).

38.5 Change and release process (INFERRED)

- Commits are made to `master`. Existing history shows small, direct commits (ten commits at the time of analysis) rather than a feature-branch/PR flow.
- The registry is redeployed automatically on every push to `main` or `master`.
- The npm packages are released by pushing a tag matching `v*`.
- Version numbers are edited by hand in the relevant `package.json`.
- The extension is packaged by hand (`pnpm -filter expo-snips package`).

Consequence: contributing directly to the default branch is effectively the current practice, and there is no automated gate before a change reaches the deployed registry. A contributor should therefore run `pnpm validate` and `pnpm build` locally before pushing, because CI will not.

38.6 What to include in a contribution

Table 38.1: Contribution expectations (INFERRED)

Change type	Expected accompanying work
New recipe	JSON document, templates, dependency declarations, validator run, registry rebuild.
Recipe change	Version bump of the recipe's own `version` if the change is behavioural; note whether existing projects need to regenerate.
Core engine change	Manual verification across several recipes and both clients; consider a test once tests exist.
Client change	Verify the same behaviour is not required in the other client.

Change type	Expected accompanying work
Extension release	Update `CHANGELOG.md`; the top of the file establishes Keep a Changelog and SemVer.
Breaking change	No deprecation policy exists; document the impact explicitly (see below).

38.7 Missing policy areas

Status: RECOMMENDED. The following should be created before the project invites external contributors:

1. A `CONTRIBUTING.md` capturing the above, plus branch and PR conventions.
2. A `CODE_OF_CONDUCT.md`.
3. Issue and pull-request templates.
4. A documented deprecation and recipe-migration policy, since recipes are versioned independently and users may have generated screens from old versions.
5. A root `README` that orients a newcomer (currently absent).
6. A licence decision for the extension: it currently ships under a "Private License", which will deter external contribution. This is a deliberate-looking author choice, but it should be stated plainly for contributors.

38.8 Contribution-friendly features already present

- The recipe validator provides immediate, actionable feedback.
- Recipes are independent files, so two contributors rarely conflict.
- The offline build means a contributor can fully test generation without any network access.
- The extension CHANGELOG already follows a recognised convention.
- The extension README documents the source layout and a build/debug workflow.

Status: IMPLEMENTED.

38.9 Summary

Status: INFERRED (all process descriptions) / RECOMMENDED (formal contribution policy). The technical workflow for adding a recipe is genuinely good; the social and policy scaffolding for outside contributors does not yet exist.

39 Glossary

39.1 Purpose

This glossary defines terms as they are used *in this document* and in the Expo Snips repository. Where a term is specific to the project it is marked **Project**; where it is an ecosystem term it is marked **General**. Definitions are grounded in the repository code.

39.2 Domain terms

Table 39.1: Glossary of domain terms

Term	Type	Definition
Recipe	Project	A JSON document describing one screen: its identity, version, supported options, dependencies, output files, optional variants, tags and primitives. Lives in `packages/registry/data/`.
Template	Project	A Handlebars (`.hbs`) file that renders to a source file in the target project. Lives in `packages/registry/templates/`.
Registry	Project	The published catalog of recipes: `index.json` plus one JSON document per recipe. Static, hosted on GitHub Pages.
Index entry	Project	The summary of a recipe used for browsing and quick picks: id, name, description, version, typescript flag, tags and the supported options.
Catalog	Project	Collectively, all recipes. Shipped inside each client as an offline bundle.
Offline bundle	Project	The inlined catalog (`bundle.json` or the generated `registry-data.ts`) that lets a client work with no network.
Variant	Project	A named alternative style for a screen (for example `glass`). Declared at the recipe level; a file maps a variant to its template.
Primitive	Project	A small reusable UI recipe (for example `ui/button`, `ui/input`, `ui/utils`) required when `ui=shadcn`, resolved recursively and skipped if already present.
Generation	Project	The act of turning a recipe plus options into written files.
Generation options	Project	The resolved choices that shape output: `styling`, `router`, `ui`, `icons` and `typescript`.
Detection	Project	Inferring options from the target project by probing `package.json` and `tsconfig.json`.

Term	Type	Definition
Manifest	Project	`.expo-snips.json` in the target project, recording which recipes (and variants) have been generated, their versions, options and files.
Upsert key	Project	The identity used to record a generated screen: recipe id plus variant.

39.3 Ecosystem and technology terms

Table 39.2: Glossary of technology terms

Term	Type	Definition
Expo	General	A framework and toolchain for building React Native applications; the runtime target of every generated screen.
React Native	General	A framework for building native mobile apps with React; the platform target.
Expo Router	General	File-based routing for Expo apps; the default router option.
React Navigation	General	An alternative navigation library; the other router option.
NativeWind	General	A Tailwind-style utility-class system for React Native; the default styling option.
Tamagui	General	A UI and styling library; listed as a styling option but not yet implemented in any recipe.
shadcn	General/Project	Here, a set of copy-in UI primitives (Button, Input, and a `cn` utility) and design tokens; selectable as a UI option and backed by primitive recipes.
lucide / lucide-react-native	General	The default icon library.
Vector Icons	General	An icon library listed as an option but not implemented in any recipe.
safe area	General	The screen region not obscured by device notches, status bars or home indicators; generated screens handle it via `react-native-safe-area-context`.
Handlebars	General	The template engine used by the shared core.
esbuild	General	The bundler used for the VS Code extension.
vsce / VSIX	General	The VS Code packaging tool and the packaged extension format.
pnpm workspace	General	The monorepo tooling; packages are linked from `packages/*`.
Monorepo	General	A single repository containing multiple packages.
Longest-common-subsequence diff	General	The algorithm behind the update preview; compares two files line by line.

Term	Type	Definition
Golden-file test	General	A test that compares generated output against a stored expected file; recommended for this project.

39.4 Documentation and status terms

Table 39.3: Glossary of documentation terms

Term	Type	Definition
IMPLEMENTED	This doc	The behaviour is present in the code, with evidence.
INFERRED	This doc	A reasonable conclusion drawn from code or structure, not stated explicitly.
RECOMMENDED	This doc	A proposed improvement; not present in the code.
UNKNOWN	This doc	Cannot be determined from the repository.
NOT APPLICABLE	This doc	The concept does not exist in this product.
OPEN / ACCEPTED / PASS	This doc	Status of a finding: unaddressed, knowingly accepted, or satisfied.
Finding / Severity / Evidence / Impact / Recommendation	This doc	The fixed format used for security findings in Chapter 23.
BRD	This doc	Business Requirements Document.
PRD	This doc	Product Requirements Document.
SEO	This doc	Search Engine Optimisation.
AEO	This doc	Answer Engine Optimisation.
GEO	This doc	Generative Engine Optimisation.

39.5 Abbreviations used in this document

Table 39.4: Abbreviations

Abbreviation	Expansion
CLI	Command-Line Interface.
CI / CD	Continuous Integration / Continuous Delivery.
CORS	Cross-Origin Resource Sharing.
DX	Developer Experience.
IaC	Infrastructure as Code.
JSON-LD	JavaScript Object Notation for Linked Data.
LCS	Longest Common Subsequence.
N/A	Not Applicable.
PASS	A security control that is satisfied.
RBAC	Role-Based Access Control (not applicable to this project).

Abbreviation	Expansion
SPDX	Software Package Data Exchange (a licence identifier standard).
VSIX	VS Code extension package format.

40 References

40.1 Purpose

This chapter lists the primary evidence used to produce this documentation. It is not a bibliography of external literature; it is a manifest of the repository files and configuration that each claim is grounded in. All paths are relative to the repository root `C:\Dev\expo-snips`.

40.2 Primary repository sources

Table 40.1: Manifest and configuration sources

Source	Used for
`package.json`	Root scripts and workspace name/version.
`pnpm-workspace.yaml`	Workspace layout and `allowBuilds` policy.
`tsconfig.base.json`	Shared TypeScript compiler settings.
`.gitignore`	Which artefacts are excluded from version control.
`packages/*/package.json`	Per-package identity, versions, licences, dependencies, build scripts.
`packages/*/tsconfig.json`	Per-package compilation settings.
`packages/vscode-extension/esbuild.mjs`	Extension bundling configuration.
`packages/vscode-extension/.vscodeignore`	VSIX exclusion rules.

Table 40.2: Source code sources

Source	Used for
`packages/core/src/types.ts`	Shared type definitions.
`packages/core/src/context.ts`	Generation context shapes.
`packages/core/src/generate.ts`	Rendering engine, conditions, helpers, variant resolution.
`packages/core/src/index.ts`	Public surface of the core package.
`packages/cli/src/index.ts`	CLI command registration and version string.
`packages/cli/src/commands/*.ts`	`init`, `list`, `add`, `update`, `status` behaviour.
`packages/cli/src/utils/*.ts`	Detection, diff, install, manifest, primitives, registry resolution, write files.
`packages/vscode-extension/src/*.ts`	Extension activation, commands, completion, generation, project detection, registry client.

Table 40.3: Data, scripts and automation sources

Source	Used for
`packages/registry/schema.json`	Recipe JSON Schema (structure, enums, required fields).
`packages/registry/data/**`	The 24 recipes.
`packages/registry/templates/**`	The 48 Handlebars templates.
`packages/registry/bundle.json`	Offline catalog bundle and its size.
`registry/index.json` and `registry/recipes/**`	Built catalog output (tracked).
`scripts/build-registry.mjs`	Registry build behaviour.
`scripts/validate.mjs`	Recipe validation rules.
`scripts/serve.mjs`	Development server, CORS, path-traversal protection, `PORT`.
`scripts/lib/build-recipes.mjs`	Template inlining and index-entry derivation.
`packages/registry/scripts/build-bundle.mjs`	Bundle construction.
`packages/*/scripts/prepare-registry.mjs`	Offline catalog generation for clients.
`.github/workflows/publish.yml`	npm release pipeline.
`.github/workflows/deploy-registry.yml`	Registry deployment pipeline.

40.3 Documentation sources

Table 40.4: Existing documentation consulted

Source	Used for
`packages/vscode-extension/README.md`	Feature description, settings, project structure, roadmap, known limitations.
`packages/vscode-extension/CHANGELOG.md`	Release history and recent fixes.
`packages/vscode-extension/LICENSE`	Extension licensing.
`documents/ProjectDescription.md`	Pre-existing product description (found to contain outdated claims; see Chapter 34).

40.4 External references

These are the ecosystems the product targets. They are referenced for context only; no external content was used as a source of claims about the repository.

Table 40.5: External references

Reference	Relevance
Expo documentation	The target runtime of generated screens.
React Native documentation	The target platform.
Handlebars documentation	The template engine.
JSON Schema (Draft-07)	The specification the recipe schema follows.
VS Code Extension API	The host API for the extension.
Keep a Changelog / Semantic Versioning	The conventions named in the extension CHANGELOG.

40.5 Verified facts ledger

The following figures appear throughout this document and were verified directly from the repository. They are listed here so they can be re-checked.

Table 40.6: Verified figures

Figure	Value and source
Recipes in the catalog	24 (`packages/registry/data`, `registry/index.json`).
Handlebars templates	48 (`packages/registry/templates`).
Packages in the workspace	4 (`packages/*`).
Recipes defining a variant	3 (`app/home`, `auth/login`, `onboarding/welcome`; the `glass` variant).
Recipes with no variants	21.
Offline bundle size	242,775 bytes (`packages/registry/bundle.json`).
Published index size	12,748 bytes (`registry/index.json`).
Extension bundle size	568,423 bytes (`packages/vscode-extension/out/extension.js`).
CLI catalog module size	249,874 bytes (`packages/cli/dist/generated/registry-data.js`).
CLI commands	5 (`init`, `list`, `add`, `update`, `status`).
Core and CLI version	0.1.0 (MIT).
Registry package	private, version 0.1.0, no licence field.
Extension version	0.1.3 (`expo-snippets`, publisher `ukown812`, “Private License”).
Commits / branch	10 commits on `master` at time of analysis.
Registry endpoints	`GET {base}/index.json`, `GET {base}/recipes/{id}.json`.

A Catalog Reference

A.1 Overview

This appendix is the complete, verified catalog at the time of analysis: 24 recipes. Data is derived directly from `packages/registry/data/**/*.json` and cross-checked against `registry/index.json`.

A.2 Recipe identity

Table A.1: All recipes: identity and shape

Id	Name	Version	Files	Variant	Primitives
app/home	Home Dashboard	0.1.0	1	glass	—
app/notifications	Notifications	0.1.0	1	—	—
app/profile	Profile	0.1.0	1	—	—
app/search	Search	0.1.0	1	—	—
app/settings	Settings	0.1.0	1	—	—
auth/forgot-password	Forgot Password	0.2.0	4	—	utils, button, input
auth/login	Login	0.3.0	7	glass	utils, button, input
auth/otp-verification	OTP Verification	0.2.0	4	—	utils, button, input
auth/register	Register	0.2.0	7	—	utils, button, input
calendar/schedule	Calendar Schedule	0.1.0	2	—	—
chat/conversations	Chat Conversations	0.1.0	2	—	—
content/article-reader	Article Reader	0.1.0	1	—	—
dashboard/analytics	Analytics Dashboard	0.1.0	1	—	—
ecommerce/product-details	Product Details	0.1.0	1	—	—
ecommerce/products	Product Listing	0.1.0	1	—	—
map/location	Map Location	0.1.0	2	—	—
onboarding/features	Features Onboarding	0.1.0	1	—	—
onboarding/permissions	Permissions	0.2.0	2	—	utils, button
onboarding/welcome	Welcome	0.2.0	2	glass	utils, button

Id	Name	Version	Files	Variant	Primitives
ui/button	Button primitive	0.1.0	1	—	utils
ui/input	Input primitive	0.1.0	1	—	utils
ui/utils	`cn` utility	0.1.0	1	—	—
utility/empty	Empty State	0.1.0	1	—	—
utility/error	Error State	0.1.0	1	—	—

A.3 Supported options

Legend — Styling: **nw** = nativewind, **ss** = stylesheet, **tg** = tamagui. Router: **er** = expo-router, **rn** = react-navigation. UI: **n** = native, **s** = shadcn, **c** = custom. Icons: **l** = lucide, **v** = vector-icons, – = none.

Table A.2: Options each recipe supports

Id	Styling	Router	UI	Icons
app/home	nw, ss	er	n	l, –
app/notifications	nw, ss	er	n	l, –
app/profile	nw, ss	er	n	l, –
app/search	nw, ss	er	n	l, –
app/settings	nw, ss	er	n	l, –
auth/forgot-password	nw, ss	er	n, s	l, –
auth/login	nw, ss	er, rn	n, s	l, –
auth/otp-verification	nw, ss	er	n, s	l, –
auth/register	nw, ss	er, rn	n, s	l, –
calendar/schedule	nw, ss	er	n	l, –
chat/conversations	nw, ss	er	n	l, –
content/article-reader	nw, ss	er	n	l, –
dashboard/analytics	nw, ss	er	n	l, –
ecommerce/product-details	nw, ss	er	n	l, –
ecommerce/products	nw, ss	er	n	l, –
map/location	nw, ss	er	n	l, –
onboarding/features	nw, ss	er	n	l, –
onboarding/permissions	nw, ss	er	n, s	l, –
onboarding/welcome	nw, ss	er	n, s	l, –
ui/button	nw	er	s	l, –
ui/input	nw	er	s	l, –
ui/utils	nw	er	s	l, –
utility/empty	nw, ss	er	n	l, –
utility/error	nw, ss	er	n	l, –

A.4 Catalog observations

- **No recipe supports Tamagui** (`tg`) or Vector Icons (`v`), and none supports `custom` UI — despite all three being advertised in the extension settings (see Chapter 30).
- **Router support is almost entirely `expo-router`**; only `auth/login` and `auth/register` additionally support `react-navigation`.
- **Only three recipes define a variant** (`glass`): `app/home`, `auth/login`, `onboarding/welcome`. The other 21 have an empty variants list.
- **shadcn support is concentrated in the auth and onboarding flows**, which are the recipes that declare primitive dependencies.
- **Every recipe is TypeScript-first** (`typescript: true`), but `auth/login` and `auth/register` also emit JavaScript output files when the TypeScript option is off.
- **Recipe versions vary** (0.1.0 through 0.3.0) and are independent of the package versions (0.1.0) and the extension version (0.1.3).

A.5 Tags

Every recipe carries search tags, which the clients use for filtering. The full tag set represents the catalog's natural-language index:

```
app, home, dashboard, notifications, list, profile, account, search,
settings, preferences, auth, forgot-password, form, login, otp,
verification, register, signup, calendar, schedule, events, planner, chat,
messages, conversations, content, article, reader, blog, detail, analytics,
stats, charts, metrics, ecommerce, product, details, products, catalog,
map, location, navigation, places, onboarding, features, intro, permissions,
welcome, ui, button, input, primitive, util, utility, empty, state, error
```

Status: IMPLEMENTED (all data verified against the repository).

B Environment and Toolchain

B.1 Application runtime environment

Table B.1: Required runtime environment for the product

Item	Requirement / value
Node.js	$\geq$ 18 (declared in the CLI and core `engines`; esbuild target `node18`).
Node.js in CI	20 (`actions/setup-node`).
pnpm	9 (pinned in both workflows).
VS Code	$\geq$ 1.80.0 (`engines.vscode`).
Target framework	Expo / React Native.
Operating systems	Not constrained by code; the CLI shells out to a package manager and the extension uses VS Code's terminal, so npm/pnpm/yarn/bun are all supported.

B.2 TypeScript and build configuration

Table B.2: Compiler and bundler settings

Setting	Value	Where
Target	ES2020	`tsconfig.base.json`; extension `tsconfig.json`
Module	CommonJS	`tsconfig.base.json`; extension
Module resolution	node	all
Strict	true	`tsconfig.base.json`; extension
Declarations	enabled (with maps)	`tsconfig.base.json`
Source maps	enabled	`tsconfig.base.json`; esbuild
esbuild target	node18	`esbuild.mjs`
esbuild format	CJS	`esbuild.mjs`
esbuild externals	`vscode`	`esbuild.mjs`
Extension output	`out/extension.js`	`esbuild.mjs`
CLI/Core output	`dist/`	per-package `tsconfig.json`

B.3 Declared dependency versions

Table B.3: Third-party dependencies by package

Package	Version	Used by
`handlebars`	^4.7.8	`@expo-snips/core` (runtime)
`commander`	^12.0.0	`@expo-snips/cli`
`prompts`	^2.4.2	`@expo-snips/cli`
`chalk`	^4.1.2	`@expo-snips/cli`
`fs-extra`	^11.2.0	`@expo-snips/cli`; `vscode-extension` (dev)
`typescript`	^5.5.0	all packages (dev)
`@types/node`	^20.0.0	cli, registry, extension (dev)
`@types/fs-extra`	^11.0.4	cli, extension (dev)
`@types/prompts`	^2.4.9	cli (dev)
`@types/vscode`	^1.80.0	extension (dev)
`esbuild`	^0.21.5	extension (dev)
`@vscode/vsce`	^2.24.0	extension (dev)

B.4 Environment variables

Table B.4: Environment variables

Variable	Scope	Meaning
`EXPO_SNIPS_REGISTRY`	CLI runtime	Override registry base URL.
`PORT`	Dev server	Port for `scripts/serve.mjs` (default 3000).
`NPM_TOKEN`	CI	npm publish authentication.
`GITHUB_TOKEN`	CI	GitHub Pages deployment.

No `.env` file exists in the repository.

B.5 Documentation build environment

This documentation suite was produced and validated with the following tools. These are the tools used to generate the *document*, not requirements of the product.

Table B.5: Tooling used to produce this documentation

Tool	Role
pdfLaTeX (MiKTeX)	Compiles `main.tex` to PDF and resolves references.
Pandoc	Converts `main.tex` to Microsoft Word (`.docx`).
Mermaid CLI (`mmdc`)	Renders the `.mmd` diagrams to PNG images.

The document is a single LaTeX `report` with one chapter per document type, deliberately using only standard constructs (headings, tables, lists, figures, captions, code blocks) so that it converts cleanly to Word.

B.6 Version landscape (a source of confusion)

Three independent version lines coexist and should not be conflated:

Table B.6: Version lines

Artefact	Version	Notes
Root workspace	0.1.0	private.
`@expo-snips/core`	0.1.0	published, MIT.
`@expo-snips/cli`	0.1.0	published, MIT.
`@expo-snips/registry`	0.1.0	private.
`expo-snippets` (extension)	0.1.3	published artefact, “Private License”.
Recipes	0.1.0–0.3.0	versioned per recipe.

Status: IMPLEMENTED (all values read from the repository).

C Classification and Coverage

C.1 Evidence labels

Every substantive claim in this documentation carries one of five labels. The labels are used literally as **Status:** markers throughout.

Table C.1: Evidence labels

Label	Meaning
IMPLEMENTED	The behaviour exists in the repository and the claim is supported by a specific file, configuration or artefact.
INFERRED	A conclusion drawn from structure or behaviour that is not stated explicitly. The reasoning is given.
RECOMMENDED	A proposed improvement that is *not* present. Never presented as if it existed.
UNKNOWN	The claim cannot be determined from the repository (for example external hosting state).
NOT APPLICABLE	The concept does not exist in this product. Never used to hide a gap — gaps are labelled as such.

Supporting conventions: security findings use *Finding / Severity / Evidence / Impact / Recommendation*; maintenance items use *Open / Accepted / Pass* or *Open / Unknown*; maintenance registers use identifiers (SEC, KI, TD, PERF, SCALE, SEO, AEO, GEO, RM, RN).

C.2 Coverage of requested document types

The table below records every document type that was requested and states how it was handled. Section types marked **NOT APPLICABLE** were deliberately *not* written as standalone chapters, because writing them would mean padding a chapter with no repository evidence. Their non-applicability is stated explicitly so nothing is silently omitted.

Table C.2: Requested document types and their handling

Requested type	Handling	Location / reason
Project overview	Written	Chapter 1
Project description	Written	Chapter 3
Executive summary	Written	Chapter 2
Business requirements (BRD)	Written	Chapter 4
Product requirements (PRD)	Written	Chapter 5
Functional requirements	Written	Chapter 6

Requested type	**Handling**	**Location / reason**
Non-functional requirements	Written	Chapter 7
Technical requirements	Written	Chapter 8
Software architecture	Written	Chapter 10
System architecture	Written	Chapter 9
High-level design	Written	Chapter 11
Low-level design	Written	Chapter 12
Frontend architecture	Written	Chapter 13
Backend architecture	NOT APPLICABLE	No backend or server application exists in the repository; the only server is a local static JSON fixture (`scripts/serve.mjs`).
Database design	NOT APPLICABLE	There is no database, ORM or persistence layer. The only persisted state is a JSON manifest in the user's project.
API documentation	Written	Chapter 16 (registry HTTP contract and the core TypeScript API).
Authentication	Written	Chapter 17 (no real auth in the product; the *generated* auth is analysed).
Authorization	Written	Chapter 18 (largely not applicable; records the absence of roles/permissions).
User flows	Written	Chapter 19
Data flows	Written	Chapter 20
UI/UX	Written	Chapter 21
Design system	Written	Chapter 22
Security	Written	Chapter 23
Performance	Written	Chapter 24
Scalability	Written	Chapter 25
Testing	Written	Chapter 26
Deployment	Written	Chapter 27
CI/CD	Written	Chapter 28
Integrations	Written	Chapter 29
Configuration	Written	Chapter 30
SEO	Written	Chapter 31
AEO	Written	Chapter 32
GEO	Written	Chapter 33
Known issues	Written	Chapter 34
Technical debt	Written	Chapter 35
Roadmap	Written	Chapter 36
Development guidelines	Written	Chapter 37
Contribution guide	Written	Chapter 38

Requested type	Handling	Location / reason
Glossary	Written	Chapter 39

C.3 Additional chapters written

Two additional chapters were written because the repository contains material that the requested list did not cover explicitly and that would otherwise go undocumented:

- **CLI reference** (Chapter 14) — the CLI is a primary product surface with five commands and a precise option set.
- **Registry and recipes** (Chapter 15) — the recipe schema and catalog are the product's core data model.

Three appendices were added: the catalog reference (Chapter A), the environment and toolchain (Chapter B), and this classification appendix. A file index follows (Chapter D).

C.4 Applicability summary

Table C.3: Areas that are not applicable to this product

Area	Verdict	Basis
Backend / server application	NOT APPLICABLE	No server code.
Database / migrations	NOT APPLICABLE	No database.
User accounts / tenancy	NOT APPLICABLE	No accounts.
Role-based access control	NOT APPLICABLE	No roles or permissions.
Web / browser UI	NOT APPLICABLE	No web application.
Public website / landing page	NOT APPLICABLE	No site; registry is JSON.
Telemetry / analytics	NOT APPLICABLE	None present (and none is a gap to fill).
Payment / billing	NOT APPLICABLE	Product is free/open-source packages plus a privately licensed extension; no transactions in code.
Containerisation / IaC	NOT APPLICABLE	Static hosting only.
Real-time / streaming	NOT APPLICABLE	No such feature.

C.5 Reading guide

- For a business reader: Chapters 1–5, plus this appendix and Chapter 34.

- For an engineer integrating or extending: Chapters 10–13, 14–20, and Chapters 37–38.
- For an assessor: Chapters 23, 24, 25, 26, 28, and Chapters 34–36.

Status: IMPLEMENTED (this appendix describes the documentation itself and the applicability of each area, all verifiable from the repository).

D File Index

D.1 Purpose

This appendix is an annotated index of the repository's hand-written files, grouped by package. Generated files and dependency trees are listed only where they matter (for example, build outputs referenced for size). Paths are relative to the repository root.

Line counts are given where they were measured during analysis and are marked *approx.* where they are estimates; they are provided to convey size, not as exact facts.

D.2 Root

Table D.1: Root-level files

Path	Purpose
`package.json`	Workspace root: name, private flag, recursive scripts (`build`, `dev`, `test`, `validate`, `build:registry`, `serve`).
`pnpm-workspace.yaml`	Declares `packages/*` and the `allowBuilds` policy (`esbuild` and `@vscode/vsce-sign` allowed; `keytar` denied).
`pnpm-lock.yaml`	Committed lockfile; the install anchor in CI.
`tsconfig.base.json`	Shared TypeScript options (`strict`, ES2020, CommonJS, declarations, source maps, `resolveJsonModule`).
`.gitignore`	Ignores `node_modules/`, `dist/`, `out/`, `*.tsbuildinfo`, `*.vsix`, `.turbo/` and the generated registry modules.
`documents/ProjectDescription.md`	Pre-existing product description (tracked; contains outdated claims — see Chapter 34).

D.3 `packages/core` — shared engine and types

Table D.2: Core package files

Path	Lines	Purpose
`src/types.ts`	—	Shared types: `ScreenRecipe`, `FileTemplate`, styling/router/UI/icon unions.
`src/context.ts`	—	`GenerationOptions`, `GenerationContext`, `RenderContext`.

Path	Lines	Purpose
src/generate.ts	≈70	Rendering engine: registers eq and neq helpers, compiles and executes templates, applies when conditions and variant overrides.
src/index.ts	—	Public surface re-exports.
package.json	—	@expo-snips/core 0.1.0, MIT, dependency handlebars.

D.4 packages/cli — command-line interface

Table D.3: CLI package files

Path	Lines	Purpose
src/index.ts	—	Registers the five commands; declares the hard-coded -version.
src/commands/init.ts	—	Scaffolds a project via create-expo-app (shell execution; SEC-04).
src/commands/list.ts	—	Lists catalog recipes.
src/commands/add.ts	—	Main generation flow: detection, option resolution, primitive resolution, file writing, manifest update, dependency install.
src/commands/update.ts	—	Diff-preview and confirmation for regenerating installed screens.
src/commands/status.ts	—	Reports installed screens and available updates.
src/utils/detectProject.ts	—	Probes package.json and tsconfig.json.
src/utils/resolveRegistry.ts	—	Resolves the registry base and the embedded-catalog fallback.
src/utils/registry.ts	—	Fetches index and recipes with a 10 s timeout.
src/utils/primitives.ts	—	Recursive primitive resolution with a seen-set guard.
src/utils/writeFiles.ts	—	Creates directories and writes generated files (no containment check; SEC-09).
src/utils/installDeps.ts	—	Shells out to install dependency strings (SEC-05).
src/utils/manifest.ts	—	Reads/upserts .expo-snips.json.
src/utils/diff.ts	—	Line diff (LCS) for update previews.

Path	Lines	Purpose
src/generated/registry-data.ts	≈240 KB	Generated offline catalog (git-ignored).

D.5 packages/registry — recipes, templates and schema

Table D.4: Registry package files

Path	Lines	Purpose
schema.json	151	JSON Schema (Draft-07) for a screen recipe.
data/**/*.json	24 files	The recipes (one document each). Full detail in Chapter A.
templates/**/*.hbs	48 files	Handlebars templates.
bundle.json	242,775 B	Offline catalog bundle (tracked).
scripts/build-bundle.mjs	—	Builds bundle.json.
src/index.ts	—	Registry package entry.
package.json	—	@expo-snips/registry 0.1.0, private, no licence field.

D.6 packages/vscode-extension — editor integration

Table D.5: Extension package files

Path	Lines	Purpose
src/extension.ts	17	Activates commands, completion provider and status bar item.
src/commands.ts	49	addScreen and generateFromCompletion handlers.
src/completionProvider.ts	45	Provides snip: / expo-snips: completions.
src/generator.ts	268	Orchestrates detection, rendering, writing, manifest and install (duplicates CLI orchestration; TD-08).
src/project.ts	103	Project detection and manifest management.
src/registry.ts	≈120	Embedded catalog plus optional remote client with 5-minute cache and fallback.
scripts/prepare-registry.mjs	—	Inlines the catalog at build time.
esbuild.mjs	30	Bundles to CJS, Node 18 target, vscode external.

Path	Lines	Purpose
`package.json`	—	`expo-snippets` 0.1.3, publisher `ukown812`, "Private License", commands and settings contributions.
`README.md`	344	Product documentation, settings, structure, roadmap, limitations.
`CHANGELOG.md`	37	Keep-a-Changelog history (0.1.0 and 0.1.3).
`LICENSE`	—	Proprietary licence.
`assets/logo.png`	—	Extension icon.
`.vscodeignore`	—	VSIX exclusion rules.
`.vscode/launch.json`, `tasks.json`	—	Debug configuration and build tasks.

D.7 `scripts` — workspace tooling

Table D.6: Build and tooling scripts

Path	Lines	Purpose
`lib/build-recipes.mjs`	105	Shared recipe loader: inlines every template, derives index entries, writes the generated TypeScript module.
`build-registry.mjs`	28	Rebuilds `registry/` from `packages/registry/data`.
`validate.mjs`	124	Validates every recipe and template reference.
`serve.mjs`	86	Static development server for the registry with CORS and path-traversal protection.

D.8 `.github/workflows` — automation

Table D.7: Workflows

Path	Lines	Purpose
`publish.yml`	27	Publishes `core` and `cli` to npm on `v*` tags.
`deploy-registry.yml`	28	Builds and deploys the registry to GitHub Pages on pushes to the default branch.

D.9 Generated and derived artefacts

Table D.8: Generated artefacts and their disposition

Artefact	Tracked?	Regenerated by
`registry/index.json`, `registry/recipes/**`	Yes	`pnpm build:registry` / deploy workflow.
`packages/registry/bundle.json`	Yes	`build:bundle` in the registry package.
`packages/cli/src/generated/registry-data.ts`	No (ignored)	CLI build (`prepare-registry.mjs`).
`packages/vscode-extension/src/generated/registry-data.ts`	No (ignored)	Extension build (`prepare-registry.mjs`).
`packages/*/dist`, `out`	No (ignored)	Builds.
`*.vsix`	No (ignored)	`pnpm -filter expo-snips package`.

Status: IMPLEMENTED (index derived from the working tree and `git ls-files`).